

spintent

SPANISH PARSE INTENTS

vo.98 2026-08-19^{*}

©2026 by Pablo González L[†]

CTAN: <https://www.ctan.org/pkg/spintent>

 <https://github.com/pablgonz/spintent>

Resumen

El paquete **spintent** proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crear documentos PDF *accesibles* (*tagged* PDF) en español usando Lua $\text{\LaTeX}$.

Índice

1 Descripción	1	3.3 El comando <code>\spmoney</code>	6
1.1 Carga del paquete	1	3.3.1 Llaves para <code>\spmoney</code>	7
1.2 El paquete <code>babel</code>	2	3.3.2 Monedas y alias para <code>\spmoney</code>	7
1.3 Fuentes tipográficas	2	3.4 El comando <code>\spshort</code>	7
1.4 El paquete <code>hyperref</code>	2	3.4.1 Llaves para <code>\spshort</code>	9
2 El comando <code>\setspintent</code>	3	3.4.2 Argumentos para <code>\spshort</code>	9
3 The big four	3	4 Utilidades generales	11
3.1 El comando <code>\spnum</code>	3	5 Utilidades para símbolos	11
3.1.1 Llaves para <code>\spnum</code>	3	6 Utilidades para números	13
3.2 El comando <code>\spunit</code>	4	7 Utilidades para geometría	17
3.2.1 Llaves para <code>\spunit</code>	4	8 Referencias	18
3.2.2 Argumentos para <code>\spunit</code>	4	9 Change history	19
3.2.3 El comando <code>\spnewunit</code>	6	10 Índice de la Documentación	20
3.2.4 El comando <code>\spunitalias</code>	6	11 Implementation	22
		12 Índice de la Implementación	75

Motivación y agradecimientos

Desde que desarrollé `scontents`[7] en 2019 y, más tarde, `enumext`[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de $\text{\LaTeX}$ [19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como `veraPDF`[20] o inspeccionar su estructura con `ngpdf`[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza NVDA[11] junto con `MathCAT`[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de `The LaTeX Tagged PDF Project` y escucharlo con NVDA/`MathCAT` para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de **spintent** nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de $\text{\LaTeX}$, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar NVDA/`MathCAT`.

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de WTPDF[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de `MathML`. En ese contexto, Lua $\text{\LaTeX}$ ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

^{*}Este archivo describe la documentación para la versión vo.98 revisada por última vez el 2026-08-19.

[†]E-mail: «pablgonz@educarchile.cl».

Licencia y requerimientos

El paquete `spintent` carga y requiere el paquete `unicode-math`[1] y que se ejecute bajo Lua $\TeX$ por lo que es necesario contar con una distribución moderna de $\TeX$ como $\TeX$ Live 2026. Ha sido probado con las clases estándar proporcionadas por $\LaTeX$: `book`, `report`, `article` y `letter` en tamaños `10pt`, `11pt` y `12pt`.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de $\LaTeX$ Project Public License (LPPL), versión 1.3 o posterior (<https://www.latex-project.org/lppl.txt>). El paquete tiene la condición de mantenido al estilo «*A Work in Progress*»¹.

- El requerimiento mínimo es $\LaTeX$ release 2026-11-01.

Este paquete solo se puede utilizar solo con Lua $\TeX$ y *tagged* PDF activo. Está presente en $\TeX$ Live y MiK $\TeX$, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue `spintent.zip` y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

```
spintent.sty => TDS:tex/luatex/spintent/
spintent.lua => TDS:tex/luatex/spintent/
spintent.pdf => TDS:doc/luatex/spintent/
README.md   => TDS:doc/luatex/spintent/
spintent.dtx => TDS:source/luatex/spintent/
spintent.ins => TDS:source/luatex/spintent/
```

1 Descripción

El nombre `spintent` puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.

Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de NVDA/MathCAT teniendo esto presente.

- El paquete `spintent` usa y abusa del atributo `:intent` de MathML por medio del comando `\MathMLintent` definido en `latex-lab-mathintent`[30], un encapsulado de `\luamml_attribute:een` proveída por `luamml`[2]. ¡Gracias Marcel!

1.1 Carga del paquete

El paquete `spintent` trabaja solo con Lua $\TeX$ y *tagged* PDF activo usando `tagging=on` dentro del comando `\DocumentMetadata` al inicio del archivo. Las llaves (*keys*) aceptadas por `\DocumentMetadata` y en general para el código de *tagged* PDF están documentadas en los paquetes `tagpdf`[29] `documentmetadata-support`[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete `babel-spanish`[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveídas por `babel-spanish` hoy no son necesarias, más allá de `\sptext` para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete `spintent` provee el comando `\spshort` (§3.4) que cubre estos casos, de esta manera podemos cargar `babel`[9] con todas sus funcionalidades sin problemas.

¹A *Work in Progress* es un álbum en vivo de Neil Peart (* 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como `spintent` es un paquete Lua $\TeX$ debemos distinguir entre las fuentes tipográficas OpenType/TrueType versus las fuentes de 8bit estándar usadas por $\TeX$ para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* PDF preferiremos no combinar fuentes OpenType/TrueType con fuentes 8bit y trabajaremos solo con fuentes OpenType/TrueType que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para Lua $\TeX$ se maneja por medio del paquete `fontspec`[3], si no se especifica ninguna se usarán las fuentes Latin Modern para el *modo texto* y Latin Modern Math para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente Libertinus para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente Erewhon-Math para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente Source Code Pro para el texto tipo «máquina de escribir» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete hyperref

El paquete `hyperref`[23] es «casi obligatorio» cuando se crean documentos *tagged* PDF, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
{
  colorlinks      = true,
  pdfstartview    = FitH,
  pdfdisplaydoctitle = true,
  pdftitle        = {Test para el paquete spintent},
  pdfinfo         =
  {
    Title   = {Test},
    Subject = {spintent},
  },
}
```

2 El comando \setspintent

`\setspintent` `\setspintent{⟨comando⟩}{⟨key-uno = valor, key-dos = valor, key-tres = valor, ...⟩}`

El comando `\setspintent` establece las llaves (*⟨keys⟩*) de forma global para los comandos provistos por `spintent`. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee. Las llaves (*⟨keys⟩*) definidas en el *argumento opcional* [*⟨key = val⟩*] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de `\setspintent`.

Muchas de las utilidades que provee el paquete `spintent` implementan el sistema [*⟨key = val⟩*] y están pensadas para ser utilizadas en la generación de «*hojas de ejercicios*» o «*clases escritas*». Se recomienda usar `\setspintent` y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de NVDA/MathCAT a lo largo del mismo.

El sistema [*⟨key = val⟩*] utilizado por `spintent` se implementa usando el módulo `l3keys` de `expl3`[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno válido; las llaves documentadas como (*valor prohibido*) no reciben *ningun valor*. En caso de omitir un *valor* válido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo (*valor prohibido*) se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete `spintent` provee cuatro grandes comandos («*the big four*²»): `\spnum` para el manejo de números, `\spunit` para el manejo de unidades, `\spmoney` para el manejo de dinero y `\spshort` para el manejo de abreviaturas, acrónimos y números ordinales.

• También se incluyen las utilidades `\spnewunit` y `\spunitalias` asociadas a al comando `\spunit`.

3.1 El comando \spnum

<code>\spnum</code>	<code>\spnum{⟨número⟩}</code>	<code>\spnum[⟨key = val⟩]{⟨número,número;número⟩}</code>
	<code>\spnum[⟨key = val⟩]{⟨+número⟩}</code>	<code>\spnum[⟨key = val⟩]{⟨número.número;número⟩}</code>
	<code>\spnum[⟨key = val⟩]{⟨-número⟩}</code>	<code>\spnum[⟨key = val⟩]{⟨número;número⟩}</code>
	<code>\spnum[⟨key = val⟩]{⟨número,número⟩}</code>	<code>\spnum[⟨key = val⟩]{⟨número e exponente⟩}</code>
	<code>\spnum[⟨key = val⟩]{⟨número.número⟩}</code>	<code>\spnum[⟨key = val⟩]{⟨número E exponente⟩}</code>

El comando `\spnum` trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨número⟩}, donde la «*parte entera*» puede iniciar con los signos '+' o '-', la «*parte decimal finita*» **debe** iniciar con una *coma* ',' o con un *punto* '.', y la «*parte decimal infinita (periódica)*» **debe** iniciar con un *punto* y *coma* ';'.

Si no se proporciona el *argumento opcional* [*⟨key = val⟩*] y se ejecuta `\spnum{+23}` se imprimirá '+23' en la salida PDF y NVDA/MathCAT verbalizará «*más veintitrés*», si se ejecuta `\spnum{3,14}` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce*», si se ejecuta `\spnum{3,1;4}` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma uno con cuatro periódico*».

Además, el *argumento obligatorio* {⟨número⟩} admite al final de este un sufijo de «*notación científica*». Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '-' seguido por los dígitos del exponente. Por ejemplo `\spnum{3,14e3}` imprimirá '3,14 · 10³' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

• La *coma* ',' o el *punto* '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {⟨argumento⟩}. El {⟨argumento⟩} puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep = {⟨true | false⟩}`

default: *true*

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en *true* se siguen las normas de la RAE y se utiliza un *espacio fino* '\,' entre las cifras; si se establece en *false*, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `\spnum{100000}` imprimirá '100 000' en la salida PDF y `\spnum[cifras-sep=false]{10\,00\,00}` imprimirá '100000' en la salida PDF.

`read-sign = {⟨terse | clear⟩}`

default: *terse*

Establece la *forma* en la que NVDA/MathCAT verbalizará los signos '+' o '-'. Si se establece en *terse*, NVDA/MathCAT verbalizará «*más*» o «*menos*» **antes** de leer el número; si se establece en *clear*, NVDA/MathCAT verbalizará «*positivo*» o «*negativo*» **después** de leer el número. Por ejemplo `\spnum{-23}` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*menos veintitrés*» y `\spnum[read-sign=clear]{-23}` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés negativo*».

`read-period-sep = {⟨coma | punto⟩}`

default: *coma*

²Los «*cuatro grandes*» del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo `spintent`.

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación ‘,’ de la «*parte decimal infinita (periódica)*» cuando no se tiene la «*parte decimal finita*». Si se establece en `coma`, se imprimirá ‘,’ en la salida PDF y NVDA/MathCAT verbalizará «*coma*»; si se establece en `punto`, se imprimirá ‘.’ en la salida PDF y NVDA/MathCAT verbalizará «*punto*». Por ejemplo `\spnum{3;4}` imprimirá ‘3,4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma cuatro periódico*» y `\spnum[read-period-sep=punto]{3;4}` imprimirá ‘3.4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres punto cuatro periódico*»

- Esta llave es necesaria SOLO cuando se utilizan «*decimales periódicos puros*», es decir, cuando no se ha incluido en el `{\argumento}` una *coma* ‘,’ o un *punto* ‘.’ antes del signo de separación ‘,’ periódica.

`notation-sym = {\times | \cdot}`

default: `\cdot`

Establece el *símbolo* que se utilizará para imprimir la «*notación científica*» al usar ‘e’ o ‘E’. Si se establece en `times` imprimirá ‘x’ en la salida PDF y NVDA/MathCAT verbalizará «*por*»; si se establece en `cdot` imprimirá ‘.’ en la salida PDF y NVDA/MathCAT verbalizará «*por*». Por ejemplo `\spnum{3,14e3}` imprimirá ‘3,14 · 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*» y `\spnum[notation-sym=times]{3,14e3}` imprimirá ‘3,14 × 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- Decir «*notación científica*» aquí es un poco exagerado, en realidad es «*multiplicación por potencias de 10*», no se verifica si la entrada cumple o no con la condición de estar escrita en «*notación científica*».

3.2 El comando \spunit

<code>\spunit</code>	<code>\spunit{\langle unidad \rangle}</code>	<code>\spunit[\langle key = val \rangle]{\langle número unidad * unidad \rangle}</code>
	<code>\spunit[\langle key = val \rangle]{\langle unidad \rangle}</code>	<code>\spunit[\langle key = val \rangle]{\langle número unidad * unidad / unidad \rangle}</code>
	<code>\spunit[\langle key = val \rangle]{\langle número unidad \rangle}</code>	<code>\spunit[\langle key = val \rangle]{\langle número unidad * unidad / unidad * unidad \rangle}</code>

El comando `\spunit` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{\langle unidad \rangle}`, el cual puede contener una «*parte numérica*» opcional la cual se procesa internamente por el comando `\spnum` al inicio del `{\langle argumento \rangle}` seguido de una `{\langle unidad \rangle}` o un `{\langle alias \rangle}` valido descritos en §3.2.2 o una `{\langle unidad \rangle}` creada por el comando `\spnewunit` (§3.2.3) o un `{\langle alias \rangle}` creado por el comando `\spunitalias` (§3.2.4).

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]` y se ejecuta `\spunit{23 m}` se imprimirá ‘23m’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros*».

La multiplicación entre las `{\langle unidades \rangle}` se realiza mediante un *asterisco* ‘*’ y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* ‘^`{\langle exponente \rangle}`’. Por ejemplo si se ejecuta `\spunit{23 m * s^2}` se imprimirá ‘23ms²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros segundos al cuadrado*».

Se pueden usar *fracciones lineales* con las `{\langle unidades \rangle}` utilizando una «*única barra*» ‘/’ para la separación entre las `{\langle unidades \rangle}` del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo si se ejecuta `\spunit{23 m/s^2}` se imprimirá ‘23m/s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros por segundos al cuadrado*».

- Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El `{\langle argumento \rangle}` puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo, NO puede contener más de una «*única barra*» ‘/’ y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.2.1 Llaves para \spunit

El comando `\spunit` posee las *meta-keys* `cifras-sep`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1).

`print-cdot = {\true | false}`

default: `false`

Establece si se *imprime* `\cdot` o un *espacio fino* ‘\,’ para la multiplicación de `{\langle unidades \rangle}`. Si se establece en `true`, se imprimirá ‘.’ entre las `{\langle unidades \rangle}` en la salida PDF; si se establece en `false` se imprimirá un *espacio fino* ‘\,’ entre las `{\langle unidades \rangle}` en la salida PDF.

`read-cdot = {\true | false}`

default: `false`

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de `{\langle unidades \rangle}`. Si se establece en `true`, NVDA/MathCAT verbalizará «*por*» entre las `{\langle unidades \rangle}`; si se establece en `false`, NVDA/MathCAT no verbalizará nada entre las `{\langle unidades \rangle}`.

`read-slash = {\por | partido-por}`

default: `por`

Establece la *forma* en la que NVDA/MathCAT verbalizará la «*barra*» ‘/’ de separación entre el numerador y el denominador. Si se establece en `por`, NVDA/MathCAT verbalizará «*por*»; si se establece en `partido-por`, NVDA/MathCAT verbalizará «*partido por*».

3.2.2 Argumentos para \spunit

Resumen de `{\langle unidades \rangle}` y `{\langle alias \rangle}` soportados por `\spunit`.

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	«metro»
g	gramo, gramos	Gramo	g	«gramo»
s	segundo, segundos	Segundo (Tiempo)	s	«segundo (Tiempo)»
l	litro, l-litro	Litro (minúscula)	l	«litro (minúscula)»
h	hora, horas	Hora	h	«hora»
d	dia, dias	Día	d	«día»
A	amperio, ampere	Amperio	A	«amperio»
V	voltio, volt	Voltio	V	«voltio»
W	vatio, watt	Vatio	W	«vatio»
J	julio, joule	Julio	J	«julio»
N	newton	Newton	N	«newton»
Pa	pascal	Pascal	Pa	«pascal»
Hz	hercio, hertz	Hercio	Hz	«hercio»
Ω	ohmio, ohm, Omega	Ohmio	Ω	«Ohmio»
F	faradio	Faradio	F	«Faradio»
C	culombio	Culombio	C	«Culombio»
K	kelvin, Kelvin	Kelvin	K	«Kelvin»
°C	celsius, degC, °C	Grados Celsius	°C	«Grados Celsius»
°F	fahrenheit, degF, °F	Grados Fahrenheit	°F	«Grados Fahrenheit»
cal	caloria, calorías	Caloría	cal	«Caloría»
b	bit	Bit	b	«Bit»
B	byte, bytes	Byte	B	«Byte»
in	pulgada, pulgadas	Pulgada	in	«Pulgada»
ft	pie, pies	Pie	ft	«Pie»
yd	yarda, yardas	Yarda	yd	«Yarda»
mi	milla, millas	Milla	mi	«Milla»
lb	libra, libras	Libra	lb	«Libra»
oz	onza, onzas	Onza	oz	«Onza»
gal	galon, galones	Galón	gal	«Galón»
Å	angstrom	Angstrom	Å	«Angstrom»
μ	micro	Micro (Prefijo)	μ	«Micro (Prefijo)»
Ω	mho	Mho / Siemens	Ω	«Mho / Siemens»
'	minmin	Minuto (Arco)	'	«Minuto (Arco)»
"	segseg	Segundo (Arco)	"	«Segundo (Arco)»
u	u-masa	Unidad masa atómica	u	«Unidad masa atómica»
Unidades directas (sin alias):				
L	-	Litro (Mayúscula)	L	«Litro (Mayúscula)»
ℓ	-	Litro (Cursiva)	ℓ	«Litro (Cursiva)»
w	-	Vatio (Minúscula)	w	«Vatio (Minúscula)»
°K	-	Grado Kelvin	°K	«Grado Kelvin»
Bq	-	Becquerel	Bq	«Becquerel»
Da	-	Dalton	Da	«Dalton»
Gy	-	Gray	Gy	«Gray»
S	-	Siemens	S	«Siemens»
Sv	-	Sievert	Sv	«Sievert»
T	-	Tesla	T	«Tesla»
Wb	-	Weber	Wb	«Weber»
atm	-	Atmósfera	atm	«Atmósfera»
au	-	Unidad astronómica	au	«Unidad astronómica»
bar	-	Bar	bar	«Bar»
cd	-	Candela	cd	«Candela»
dB	-	Decibelio	dB	«Decibelio»
dyn	-	Dina	dyn	«Dina»
eV	-	Electronvoltio	eV	«Electronvoltio»
erg	-	Ergio	erg	«Ergio»
ha	-	Hectárea	ha	«Hectárea»
kat	-	Katal	kat	«Katal»
kg	-	Kilogramo	kg	«Kilogramo»
km	-	Kilómetro	km	«Kilómetro»
lm	-	Lumen	lm	«Lumen»

Continúa en la siguiente página...

Continuación de la tabla 1

Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	«Lux»
cm	-	Centímetro	cm	«Centímetro»
min	-	Minuto (Tiempo)	min	«Minuto (Tiempo)»
mm	-	Milímetro	mm	«Milímetro»
mol	-	Mol	mol	«Mol»
pc	-	Pársec	pc	«Pársec»
rad	-	Radián	rad	«Radián»
rpm	-	Rev. por minuto	rpm	«Rev. por minuto»
sr	-	Estereorradián	sr	«Estereorradián»
t	-	Tonelada	t	«Tonelada»
a	-	Año / Área (are)	a	«Año / Área (are)»
y	-	Año (Year)	y	«Año (Year)»
°	-	Grado (Arco)	°	«Grado (Arco)»

3.2.3 El comando \spnewunit

`\spnewunit` `\spnewunit{⟨nueva unidad⟩}{⟨verbalización NVDA⟩}`

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: `{⟨nueva unidad⟩}` que establece lo que se imprimirá en la salida PDF y `{⟨verbalización NVDA⟩}` que establece como NVDA/MathCAT verbalizará la `{⟨nueva unidad⟩}`.

La `{⟨nueva unidad⟩}` no debe estar registrada como `{⟨unidad⟩}` o `{⟨alias⟩}` para `\spunit` descritos en §3.2.2. Si la `{⟨nueva unidad⟩}` ya está registrada se devolverá un “error” y se detendrá la compilación del documento.

La declaración de la `{⟨nueva unidad⟩}` es «global» y se almacena en el módulo `spintent.lua` como una `{⟨unidad⟩}` válida para pasar como `{⟨argumento⟩}` a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la `{⟨nueva unidad⟩}` ‘px’ y cuando se utilice `\spunit{100 px}` se imprimirá ‘100px’ en la salida PDF y NVDA/MathCAT verbalizará «cien píxeles».

3.2.4 El comando \spunitalias

`\spunitalias` `\spunitalias{⟨nuevo alias⟩}{⟨expresión de unidades⟩}`

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: `{⟨nuevo alias⟩}` que establece es el nuevo `{⟨alias⟩}` de `{⟨argumento⟩}` para `\spunit` y `{⟨expresión de unidades⟩}` que puede contener multiplicación o división de `{⟨unidades⟩}` ya registradas que se imprimirán en la salida PDF.

El `{⟨nuevo alias⟩}` no debe estar registrado como `{⟨unidad⟩}` o `{⟨alias⟩}` para `\spunit` descritos en §3.2.2 o estar definido como `{⟨unidad⟩}` por medio del comando `\spnewunit`. Si el `{⟨nuevo alias⟩}` ya está registrado se devolverá un “error” y se detendrá la compilación del documento.

La declaración de un `{⟨nuevo alias⟩}` es «global» y se almacena en el módulo `spintent.lua` como una `{⟨alias⟩}` válido para pasar como `{⟨argumento⟩}` a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^3/s}` creará el `{⟨alias⟩}` ‘caudal’ y cuando se utilice `$\spunit{50 caudal}$` se imprimirá ‘50m³/s’ en la salida PDF y NVDA/MathCAT verbalizará «cincuenta metros al cubo por segundos».

3.3 El comando \spmoney

`\spmoney` `\spmoney{⟨cifra⟩}`
`\spmoney{⟨cifra moneda⟩}`
`\spmoney[⟨key = val⟩]{⟨cifra moneda⟩}`

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{⟨cifra⟩}` que contiene la «cifra numérica» positiva o negativa la se cual procesa internamente por el comando `\spnum` al inicio del `{⟨argumento⟩}` acompañada de un «símbolo de moneda» o «alias de moneda» opcional descritos en §3.3.2.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]` y se ejecuta `$\spmoney{500}$` se imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos».

El comportamiento de `\spmoney` depende de como se pase el *argumento obligatorio* que tiene prioridad por sobre los valores por defecto. Por ejemplo: `$\spmoney{500 dolares}$` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos dolares» aunque la moneda por defecto sean pesos.

Si se pasa un «alias de moneda» del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estandar ISO. Por ejemplo: `$\spmoney{500 CLP}$` imprimirá ‘CLP500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos chilenos».

Si se pasa un «alias de moneda» del tipo ISO (código de divisas) como clp o usd (minúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país, pero, NO se modifica la salida PDF imprimiendo el

«*símbolo de moneda*» correspondiente. Por ejemplo: `\$ \spmoney{500 clp}` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos chilenos»

Pasar de manera directa en el *argumento obligatorio* un «*símbolo de moneda*» como '\$', '€', '£' o un «*alias de moneda*» como peso, euro o libra produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT.

- La posición del «*símbolo de moneda*» a la izquierda o a la derecha de la «*cifra numérica*» y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El «*símbolo de moneda*» es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando, se debe tener presente en caso de querer usar un «*símbolo de moneda*» poco habitual que esté soportada por `spintent`. El `{\argumento}` puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{\#1}` o `\mathbf{\#1}` por ejemplo. La coma ',' o el punto '.' como *separador decimal* solo pueden aparecer una «única vez». Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.3.1 Llaves para \spmoney

El comando `\spmoney` posee la *meta-key* `cifras-sep` pasada al comando `\spnum` (§3.1.1).

`currency = {\moneda | alias}` default: peso

Establece el *símbolo de moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una «*cifra numérica*» como `{\argumento}` al comando `\spmoney`. El «*símbolo de moneda*» o el «*alias de moneda*» pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units = {\true | false}` default: false

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `\$ \spmoney[sub-units=true]{100,5 dolares}` imprimirá '\$100,5' en la salida PDF y NVDA/MathCAT verbalizará «*cient dólares con cincuenta centavos*».

`dolar-math = {\true | false}` default: true

Establece si el *símbolo* '\$' se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir '\$' en la salida PDF; si se establece en `false`, se usará la fuente tipográfica definida para el *modo texto* para imprimir '\$' en la salida PDF.

3.3.2 Monedas y alias para \spmoney

Resumen de los `{\símbolos de moneda}` y `{\alias de moneda}` soportados como `{\argumento}` para `\spmoney`.

3.4 El comando \spshort

`\spshort` `\spshort{\abreviaturas | acrónimos | números ordinales}`
`\spshort[key = val]{\abreviaturas | acrónimos | números ordinales}`
`\spshort[read-arg={\verbalización del argumento para NVDA}]{\comandos LATEX}`

El comando `\spshort` trabaja *solo en modo texto* y recibe un *argumento obligatorio* del tipo `{\abreviaturas}`, `{\acrónimos}` o `{\números ordinales}` descritos en §3.4.2.

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\spshort{dr. a}` se imprimirá 'Dr.^a' en la salida PDF y NVDA/MathCAT verbalizará «*doctora*»; si se ejecuta `\spshort{onu}` se imprimirá 'ONU' en la salida PDF y NVDA/MathCAT verbalizará «*organización de las naciones unidas*» y si se ejecuta `\spshort{mineduc}` se imprimirá 'Mineduc' en la salida PDF y NVDA/MathCAT verbalizará «*ministerio de educación*».

Para la escritura de `{\números ordinales}` se puede ejecutar `\spshort{1. a}` que imprimirá '1.^a' en la salida PDF y NVDA/MathCAT verbalizará «*primera*»; si se ejecuta `\spshort{1. o}` imprimirá '1.^o' en la salida PDF y NVDA/MathCAT verbalizará «*primero*».

Para crear una nueva `{\abreviatura}`, `{\acrónimo}` o `{\número ordinal}` que no se encuentre registrada por `spintent` dentro del módulo Lua `spintent.lua` descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del *argumento obligatorio* que puede contener `{\comandos LATEX}` en *modo texto* los cuales se imprimirán en salida PDF.

- Para la escritura de `{\números ordinales}` se debe tener cuidado y no confundir la «*letra o voladita*» 'º' con el símbolo de grado '°'. Si la fuente tipográfica del *modo texto* está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la «*letra voladita*», la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1. o}}`.
- El comando `\spshort` está pensado como un reemplazo para *shorthands* y el comando `\sptext` provistos por el paquete `babel` en español compatible con *tagged* PDF. Es de las pocas utilidades que provee el paquete `spintent` que trabaja *solo en modo texto*. Internamente utiliza la llave `E` (*expansion text*) proveída por el paquete `tagpdf`[29].

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
\$	peso, pesos	peso	\$	«peso / pesos»
€	euro, euros	euro	€	«euro / euros»
£	libra, libras	libra	£	«libra / libras»
¥	yen, yenes	yen	¥	«yen / yenes»
¥	yuan, yuanes	yuan	¥	«yuan / yuanes»
₩	won, wons	won	₩	«won / wons»
₪	shekel, shekels, sequel, sequeles	séquel	₪	«séquel / séqueles»
₹	rupia, rupias	rupia	₹	«rupia / rupias»
₽	rublo, rublos	rublo	₽	«rublo / rublos»
₺	lira, liras	lira	₺	«lira / liras»
₾	grivna, grivnas	grivna	₾	«grivna / grivnas»
¢	centavo, centavos	centavo	¢	«centavo / centavos»

Divisas en formato ISO (Mayúsculas)

CLP	—	peso chileno	CLP	«peso chileno / pesos chilenos»
MXN	—	peso mexicano	MXN	«peso mexicano / pesos mexicanos»
USD	—	dólar	USD	«dólar estadounidense / dólares estadounidenses»
EUR	—	euro	EUR	«euro / euros»
GBP	—	libra	GBP	«libra / libras»
JPY	—	yen	JPY	«yen / yenes»
CNY	—	yuan	CNY	«yuan / yuanes»
KRW	—	won	KRW	«won / wons»
ILS	—	séquel	ILS	«séquel / séqueles»
INR	—	rupia	INR	«rupia / rupias»
RUB	—	rublo	RUB	«rublo / rublos»
TRY	—	lira	TRY	«lira / liras»
UAH	—	grivna	UAH	«grivna / grivnas»

Códigos ISO en minúsculas

clp	—	peso chileno	\$	«peso chileno / pesos chilenos»
mxn	—	peso mexicano	\$	«peso mexicano / pesos mexicanos»
usd	dolar, dolares	dólar	\$	«dólar estadounidense / dólares estadounidenses»
eur	—	euro	€	«euro / euros»
gbp	—	libra	£	«libra / libras»
jpy	—	yen	¥	«yen / yenes»
cny	—	yuan	¥	«yuan / yuanes»
krw	—	won	₩	«won / wons»
ils	—	séquel	₪	«séquel / séqueles»
inr	—	rupia	₹	«rupia / rupias»
rub	—	rublo	₽	«rublo / rublos»
try	—	lira	₺	«lira / liras»
uah	—	grivna	₾	«grivna / grivnas»

3.4.1 Llaves para \spshort

`read-arg = { <verbalización NVDA> }` default: no usado

Establece la *verbalización del argumento* { <comandos $\LaTeX$ > } de la nueva { <abreviatura> }, { <acrónimo> } o { <número ordinal> } para NVDA. El valor de esta llave *siempre* debe pasarse entre ‘{ }’. Por ejemplo: `\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}}` creará una nueva { <abreviatura> } que imprimirá ‘8.^{avo}’ en la salida PDF y NVDA verbalizará «octavo».

`small-caps = { <true | false> }` default: false

Establece si se usarán *versalitas* para imprimir { <acrónimos> } y para la «letra volada» en { <abreviaturas> } y { <números ordinales> } respetando las reglas de la RAE para el uso de estas. Si se establece en `true`, las abreviaturas con mayúscula como EE. UU. y los acrónimos de cinco o más letras como Unicef no se verán afectados; si se establece en `false`, no se aplicarán *versalitas*. Por ejemplo: `\spshort[small-caps=true]{dr.a}` imprimirá ‘Dr.^a’ en la salida PDF y NVDA verbalizará «doctora».

3.4.2 Argumentos para \spshort

El cuadro 2 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 2: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
dr.a	dr. ^a	Dr. ^a	«doctora»
dr.as	—	Dr. ^{as}	«doctoras»
prof.a	—	Prof. ^a	«profesora»
d.a	d. ^a	D. ^a	«doña»
m.a	—	M. ^a	«maría»
n.o	n. ^o	N. ^o	«número»
n.os	—	N. ^{os}	«números»
c.ia	—	C. ^{ia}	«compañía»
Abreviaturas comunes y expresiones latinas			
pág.	pag.	pág.	«página»
vol.	—	vol.	«volumen»
etc.	—	etc.	«etcétera»
et al.	et. al.	et al.	«y otros»
ibíd.	ibid.	ibíd.	«ibídem»
op. cit.	op.cit.	op. cit.	«obra citada»
loc. cit.	loc.cit.	loc. cit.	«lugar citado»
v. gr.	v.gr.	v. gr.	«verbigracia»
i. e.	i.e.	i. e.	«esto es»
e. g.	e.g.	e. g.	«por ejemplo»
p. ej.	p.ej.	p. ej.	«por ejemplo»
Títulos, profesiones y cargos			
sr.	—	Sr.	«señor»
sra.	—	Sra.	«señora»
srta.	—	Srta.	«señorita»
dr.	—	Dr.	«doctor»
dra.	—	Dra.	«doctora»
dres.	—	Dres.	«doctores»
dras.	—	Dras.	«doctoras»
prof.	—	Prof.	«profesor»
profa.	—	Profa.	«profesora»
profs.	—	Profs.	«profesores»
ing.	—	Ing.	«ingeniero»
ings.	—	Ings.	«ingenieros»
lic.	—	Lic.	«licenciado»
v. b.	v.b.	V. B.	«visto bueno»
Abreviaturas compuestas e institucionales			
s. a.	s.a.	S. A.	«sociedad anónima»
ee. uu.	ee.uu.	EE. UU.	«estados unidos»
dd. hh.	dd.hh.	DD. HH.	«derechos humanos»
jj. oo.	jj.oo.	JJ. OO.	«juegos olímpicos»

Continúa en la siguiente página...

Cuadro 2: (Continuación - Abreviaturas soportadas)

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
ff. aa.	ff.aa.	FF. AA.	«fuerzas armadas»
rr. ee.	rr.ee.	RR. EE.	«relaciones exteriores»
cc. aa.	cc.aa.	CC. AA.	«comunidades autónomas»
p. d.	p.d.	P. D.	«posdata»
a. c.	a.c.	a. C.	«antes de Cristo»
d. c.	d.c.	d. C.	«después de Cristo»
d. n. i.	d.n.i.	D. N. I.	«documento nacional de identidad»
r. u. t.	r.u.t.	R. U. T.	«rol único tributario»
r. u. n.	r.u.n.	R. U. N.	«rol único nacional»

El cuadro 3 detalla las siglas y acrónimos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
dni	—	DNI	«documento nacional de identidad»
rut	—	RUT	«rol único tributario»
run	—	RUN	«rol único nacional»
onu	—	ONU	«organización de las naciones unidas»
rae	—	RAE	«real academia española»
ong	ongs	ONG	«organización no gubernamental»
urss	—	URSS	«unión de repúblicas socialistas soviéticas»
oea	—	OEA	«organización de los estados americanos»
oms	—	OMS	«organización mundial de la salud»
fmi	—	FMI	«fondo monetario internacional»
bid	—	BID	«banco interamericano de desarrollo»
otan	—	OTAN	«organización del tratado del atlántico norte»
ue	—	UE	«unión europea»
ru	—	RU	«reino unido»
Acrónimos silábicos			
unicef	—	Unicef	«fondo de las naciones unidas para la infancia»
unesco	—	Unesco	«organización de las naciones unidas para la educación, la ciencia y la cultura»
mercosur	—	Mercosur	«mercado común del sur»
cepal	—	Cepal	«comisión económica para américa latina»
mineduc	—	Mineduc	«ministerio de educación»
conicet	—	Conicet	«consejo nacional de investigaciones»

El cuadro 4 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`.

Cuadro 4: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
1.a	1. ^a	1. ^a	«primera»
2.o	2. ^o	2. ^o	«segundo»
3.er	—	3. ^{er}	«tercer»
4.os	—	4. ^{os}	«cuartos»
5.as	—	5. ^{as}	«quintas»

4 Utilidades generales

Además de los «cuatro grandes» y sus asociados, el paquete **spintent** provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

```
\spsiglo \spsiglo{<siglo>}
\spsiglo[<key = val>]{<siglo>}
```

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el *argumento obligatorio* `{<siglo>}`, el cual puede ser un *número arábigo* ‘21’ o un *número romano* ‘XXI’ mayor que ‘0’ y menor que ‘4000’.

Si no se proporciona el *argumento opcional* `[<key = val>]` y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

- El *argumento obligatorio* `{<siglo>}` «siempre» es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete **spintent** que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete **tagpdf** en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

```
print-era = {<ac | dc | none>} default: none
```

Establece si se *imprime* o no la abreviatura después de imprimir el argumento `{<siglo>}`. Si se establece en `ac`, `\spsiglo[print-era=ac]{XXI}` imprimirá ‘xi a. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno antes de cristo»; si se establece en `dc`, `\spsiglo[print-era=dc]{XXI}` imprimirá ‘xi d. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno después de cristo»; si se establece en `none`, `\spsiglo[print-era=none]{XXI}` imprimirá ‘xi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

El comando `\sptime`

```
\sptime \sptime{<horas : minutos>}
\sptime{<horas : minutos : segundos>}
```

El comando `\sptime` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<horas : minutos>}` o `{<horas : minutos : segundos>}`. Por ejemplo: `\sptime{23:45}` imprimirá ‘23:45’ en *modo texto* la salida PDF y NVDA/MathCAT verbalizará «las veintitrés y cuarenta y cinco minutos».

El comando `\spdate`

```
\spdate \spdate{<DD/ MM/YYYY>} \spdate{<DD- MM-YYYY>}
\spdate{<YYYY/ MM/DD>} \spdate{<YYYY- MM-DD>}
```

El comando `\spdate` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<DD/MM/YYYY>}`, `{<DD-MM-YYYY>}` o `{<YYYY-MM-DD>}`. Por ejemplo: `\spdate{1981-01-02}` y `\spdate{02/01/1981}` imprimirán en *modo texto* la salida PDF ‘1981-01-02’ y ‘02/01/1981’ respectivamente y en ambos casos, NVDA/MathCAT verbalizará «dos de enero de mil novecientos ochenta y uno».

- Si se pasa el argumento `{<YYYY/MM/DD>}`, internamente se convertirá en `{<DD/MM/YYYY>}` y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete **spintent** provee tres comandos `\spread`, `\spdsym` y `\spmsym` para trabajar con «símbolos matemáticos» descritos en esta sección. Están pensando desde el punto de vista docente, para situaciones donde la lectura de un «símbolo» (acorde al contexto) es relevante.

- A diferencia de las otras utilidades implementados por **spintent** (que analizan y dan formato a la salida), estos no procesan los «símbolos matemáticos», solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spread`

```
\spread \spread{<verbalización NVDA>}{<símbolo matemático>}
```

El comando `\spread` trabaja en *modo matemático* y recibe dos *argumentos obligatorios*: El primer *argumento* `{<verbalización NVDA>}` establece la «forma» en la cual NVDA/MathCAT verbalizará al segundo *argumento* `{<símbolo matemático>}`. Por ejemplo: `\spread{es-paralela}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralela»; `\spread{es-paralelo}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralelo».

El comando `\spusep`

```
\spusep <separador>
\spusep[<key = val>]{<separador>}
```

El comando `\spusep` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<separador>}`, el cual debe ser uno de los «delimitadores» soportados: ‘(’, ‘)’, ‘[’, ‘]’, ‘{’, ‘}’ o ‘\’. Imprime el *delimitador* indicado con el tamaño establecido por la llave `size` y controla si NVDA/MathCAT lo verbaliza o no mediante la llave `silent`.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\$<separador>$` imprimirá ‘(’ en la salida PDF y NVDA/MathCAT verbalizará «paréntesis izquierdo».

- El motivo de existencia de `\spusep` es que los comandos `\left`, `\right`, `\Uleft` y `\Uright` provistos por LuaTeX crean un *grupo* implícito que dificulta el código para *tagged* PDF cuando se necesita interrumpir algo entre ellos, por ejemplo al construir expresiones complejas con `\spnZ`, `\spmQ` o `\spdiv`. El comando `\spusep` imprime el *delimitador* sin crear ningún grupo.

Llaves para `\spusep`

```
size = {<none> | big | Big | bigg | Bigg} default: none
```

Establece el *tamaño* del `{<delimitador>}` que se imprimirá en la salida PDF. Los valores corresponden a los cuatro tamaños fijos provistos por L^AT_EX: si se establece en `none`, el `{<delimitador>}` se imprime en el tamaño por defecto del modo matemático en curso; si se establece en `big`, se usa `\bigl/\bigr`; si se establece en `Big`, se usa `\Bigl/\Bigr`; si se establece en `bigg`, se usa `\biggl/\biggr`; y si se establece en `Bigg`, se usa `\Biggl/\Biggr`. El tamaño se aplica automáticamente en el lado correcto (izquierdo o derecho) según el `{<delimitador>}` pasado.

```
silent = {<true> | <false>} default: false
```

Establece si NVDA/MathCAT verbalizará el `{<delimitador>}` impreso por `\spusep`. Si se establece en `true`, el `{<delimitador>}` se marcará con el *intent* `:silent` y NVDA/MathCAT no lo verbalizará; si se establece en `false`, NVDA/MathCAT verbalizará el `{<delimitador>}` de forma normal.

El comando `\spdsym`

```
\spdsym
\spdsym[<key = val>]
```

El comando `\spdsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de división* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\$<spdsym>$` imprimirá ‘÷’ en la salida PDF y NVDA/MathCAT verbalizará «dividido».

Llaves para `\spdsym`

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el `{<valor>}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
suffix = {<sufijo>} default: no usado
```

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el `{<valor>}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
set-sym = {<old-style> | <two-dots> | <slash>} default: two-dots
```

Establece el *símbolo de división* que se imprimirá en la salida PDF. Si se establece en `old-style`, imprimirá ‘÷’; si se establece en `two-dots`, imprimirá ‘:’ y si se establece `slash`, imprimirá ‘/’.

```
read-sym = {<dividido> | <dividido-por> | <dividido-entre>} default: dividido
```

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de división*. Si se establece en `dividido`, NVDA/MathCAT verbalizará «dividido»; si se establece en `dividido-por`, NVDA/MathCAT verbalizará «dividido por» y si se establece en `dividido-entre`, NVDA/MathCAT verbalizará «dividido entre».

El comando `\spmsym`

```
\spmsym
\spmsym[<key = val>]
```

El comando `\spmsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de multiplicación* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\$<spmsym>$` imprimirá ‘·’ en la salida PDF y NVDA/MathCAT verbalizará «por».

Llaves para `\spmsym`

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`suffix = {⟨sufijo⟩}` default: no usado

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`set-sym = {⟨cross | dot | asterisk⟩}` default: dot

Establece el *símbolo de multiplicación* que se imprimirá en la salida PDF. Si se establece en `cross`, imprimirá '×'; si se establece en `dot`, imprimirá '·' y si se establece en `asterisk`, imprimirá '*'.

`read-sym = {⟨por | multiplicado-por | veces⟩}` default: por

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de multiplicación*. Si se establece en `por`, NVDA/MathCAT verbalizará «por»; si se establece en `multiplicado-por`, NVDA/MathCAT verbalizará «multiplicado por» y si se establece en `veces`, NVDA/MathCAT verbalizará «veces».

`not-print = {⟨true | false⟩}` default: false

Deshabilita la *impresión* del *símbolo de multiplicación*. Cuando se establece esta llave, NVDA/MathCAT verbalizará el *símbolo de multiplicación* acorde al valor establecido por la llave `read-sym`, pero, no se imprimirá este en la salida PDF.

6 Utilidades para números

El paquete `spintent` provee utilidades para trabajar con números descritas en esta sección.

El comando `\spdiv`

```
\spdiv {⟨dividendo⟩}{⟨divisor⟩}
\spdiv [⟨key = val⟩]{⟨dividendo⟩}{⟨divisor⟩}
```

El comando `\spdiv` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* {⟨dividendo⟩} y {⟨divisor⟩}.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `$\spdiv{20}{26}$` imprimirá '20 : 26' en la salida PDF y NVDA/MathCAT verbalizará «veinte dividido veinte y seis»

Llaves para `\spdiv`

El comando `\spdiv` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 12).

`wrap-parens = {⟨true | false⟩}` default: false

Establece si se *imprimen* paréntesis redondos '(...)' alrededor de los *argumentos* {⟨dividendo⟩} y {⟨divisor⟩} pasados a `\spdiv`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}` default: true

Establece si NVDA/MathCAT verbalizará los paréntesis redondos '(...)' alrededor de los *argumentos* {⟨dividendo⟩} y {⟨divisor⟩} pasados a `\spdiv`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spmul`

```
\spmul {⟨factor⟩}{⟨factor⟩}
\spmul [⟨key = val⟩]{⟨factor⟩}{⟨factor⟩}
```

El comando `\spmul` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* {⟨factor⟩} y {⟨factor⟩}.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `$\spmul{20}{26}$` imprimirá '20 · 26' en la salida PDF y NVDA/MathCAT verbalizará «veinte por veinte y seis».

Llaves para `\spmul`

El comando `\spmul` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 13).

`wrap-parens = {⟨true | false⟩}` default: false

Establece si se *imprimen* paréntesis redondos '(...)' alrededor de los *argumentos* {⟨factor⟩} y {⟨factor⟩} pasados a `\spmul`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}` default: true

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(…)’ alrededor de los *argumentos* $\{ \langle \text{factor} \rangle \}$ y $\{ \langle \text{factor} \rangle \}$ pasados a `\spmul`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spnD`

```
\spnD \spnD
\spnD(\<número natural>)
\spnD[key = val](\<número natural>)
```

El comando `\spnD` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* $(\langle \text{número natural} \rangle)$, por ejemplo: ‘(6)’. Si se usa sin $(\langle \text{argumento} \rangle)$, `\spnD` imprimirá ‘D(n)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de n».

Si se usa con $(\langle \text{argumento} \rangle)$, por ejemplo, `\spnD(6)`, imprimirá ‘D(6)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis».

Llaves para `\spnD`

`prefix = { \<prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnD(\<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { \<true | false> }` default: false

Establece si se *imprime* el resultado de calcular los *divisores* del $(\langle \text{número natural} \rangle)$ pasado a `\spnD`. Si se establece en `true`, `\spnD[result=true](6)` imprimirá ‘D(6) = {1,2,3,6}’ en la salida PDF y NVDA/MathCAT verbalizará «los divisores de seis son uno, dos, tres y seis»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *divisores*.

`result-num = { \<número natural> }` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo `5` valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = { \<true | false> }` default: false

Desactiva la *validación* interna del $(\langle \text{argumento} \rangle)$ pasado a `\spnD`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnD[sample-arg=true](m)` imprimirá ‘D(m)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de m»; si se establece en `false`, el $(\langle \text{argumento} \rangle)$ debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = { \<true | false> }` default: false

Establece si se *silencia* la verbalización para NVDA/MathCAT de `\spnD(\<argumento>)` cuando se establece `sample-arg=true` o cuando `\spnD` se usa sin $(\langle \text{argumento} \rangle)$. Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnM`

```
\spnM \spnM
\spnM(\<número natural>)
\spnM[key = val](\<número natural>)
```

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* $(\langle \text{número natural} \rangle)$, por ejemplo: ‘(6)’. Si se usa sin $(\langle \text{argumento} \rangle)$, `\spnM` imprimirá ‘M(n)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de n».

Si se usa con $(\langle \text{argumento} \rangle)$, por ejemplo, `\spnM(6)`, imprimirá ‘M(6)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis».

Llaves para `\spnM`

`prefix = { \<prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnM(\<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { \<true | false> }` default: false

Establece si se *imprime* el resultado de calcular los *múltiplos* del $(\langle \text{número natural} \rangle)$ pasado a `\spnM`. Si se establece en `true`, `\spnM[result=true](6)` imprimirá ‘M(6) = {6,12,18,24,30,...}’ en la salida PDF y NVDA/MathCAT verbalizará «los múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspensivos»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *múltiplos*.

`result-num = { \<número natural> }` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF seguidos de ‘...’; si no se establece, se imprimirán por defecto los `5` primeros valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = { \<true | false> }` default: false

Desactiva la *validación* interna del (*argumento*) pasado a `\spnM`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnM[sample-arg=true](m)` imprimirá ‘M(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de *m*»; si se establece en `false`, el (*argumento*) debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = { (true | false) }` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spnM`(*argumento*) cuando se establece `sample-arg=true` o cuando `\spnM` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false` NVDA/MathCAT lo verbalizará.

El comando `\spmc`

```
\spmc \spmc
\spmc( (números separados por coma) )
\spmc[ (key = val) ] ( (números separados por coma) )
```

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (*números separados por coma*), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (*argumento*) `\spmc` imprimirá ‘mcd’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor» o «*m c d*», acorde al {*valor*} establecido por la llave `read-cmd`.

Si se usa con (*argumento*) `\spmc(2, 3, 6)` imprimirá ‘mcd(2,3,6)’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis» o «*m c d* entre dos, tres y seis» acorde al {*valor*} establecido por la llave `read-cmd`.

Llaves para `\spmc`

`upper-case = { (true | false) }` default: false

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando `\spmc`. Si se establece en `true`, se imprimirá ‘MCD’ en la salida PDF; si se establece en `false`, se imprimirá ‘mcd’ en la salida PDF.

`read-cmd = { (terse | clear) }` default: clear

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando `\spmc`. Si se establece en `terse`, NVDA/MathCAT verbalizará «*m c d*»; si se establece en `clear`, NVDA/MathCAT verbalizará «máximo común divisor».

`prefix = { (prefijo) }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el valor establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { (true | false) }` default: false

Establece si se *imprime* el resultado de calcular el «máximo común divisor» del (*argumento*) pasado a `\spmc`. Si se establece en `true`: `\spmc[result=true](2, 3, 6)` imprimirá ‘mcd(2,3,6) = 1’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis es igual a uno»; si se establece en `false` no se imprimirá en la salida PDF el resultado de calcular el «máximo común divisor».

`sample-arg = { (true | false) }` default: false

Desactiva la *validación* interna del (*argumento*) pasado a `\spmc`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`: `\spmc[sample-arg=true](m, n)`, imprimirá ‘mcd(*m*,*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre *m* y *n*»; si se establece en `false` el (*argumento*) debe ser una lista de *números naturales*, en caso contrario se devolverá un “error”.

`silent-cmd = { (true | false) }` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc`(*argumento*) cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spmc`

```
\spmc \spmc
\spmc( (números separados por coma) )
\spmc[ (key = val) ] ( (números separados por coma) )
```

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (*números separados por coma*), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (*argumento*) `\spmc` imprimirá ‘mcm’ en la salida PDF, y NVDA/MathCAT verbalizará «mínimo común múltiplo» o «*m c m*», acorde al {*valor*} establecido por la llave `read-cmd`.

Si se usa con (*argumento*) `\spmc(2, 3, 6)` imprimirá ‘mcm(2,3,6)’ en la salida PDF, y NVDA/MathCAT verbalizará «mínimo común múltiplo entre dos, tres y seis» o «*m c m* entre dos, tres y seis» acorde al {*valor*} establecido por la llave `read-cmd`.

Llaves para `\spmc`

`upper-case` = { `<true>` | `<false>` } default: `false`

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando `\spmc`. Si se establece en `true` se imprimirá ‘MCM’ en la salida PDF; si se establece en `false` se imprimirá ‘mcm’ en la salida PDF.

`read-cmd` = { `<terse>` | `<clear>` } default: `clear`

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando `\spmc`. Si se establece en `terse`, NVDA/MathCAT verbalizará «*m c m*»; si se establece en `clear`, NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «*antes*» de verbalizar el valor establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result` = { `<true>` | `<false>` } default: `false`

Establece si se *imprime* el resultado de calcular el «*mínimo común múltiplo*» del (`<argumento>`) pasado a `\spmc`. Si se establece en `true`: `\spmc[result=true](2, 3, 6)` imprimirá ‘mcm(2,3,6) = 6’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis es igual a seis*»; si se establece en `false` no se imprimirá en la salida PDF el resultado de calcular el «*mínimo común múltiplo*».

`sample-arg` = { `<true>` | `<false>` } default: `false`

Desactiva la *validación* interna del (`<argumento>`) pasado a `\spmc`, permitiendo usar ejemplos «*no numéricos*» como entrada. Si se establece en `true`: `\spmc[sample-arg=true](m, n)`, imprimirá ‘mcm(*m*,*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre m y n*»; si se establece en `false` el (`<argumento>`) debe ser una lista de *números naturales*, en caso contrario se devolverá un “error”.

`silent-cmd` = { `<true>` | `<false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc(<argumento>)` cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (`<argumento>`). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false` NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnZ`

`\spnZ` `\spnZ{<número entero>}`
`\spnZ[<key = val>]{<número entero>}`

El comando `\spnZ` trabaja en *modo matemático* y recibe el *argumento* obligatorio {`<número entero>`} el cual se procesa internamente bajo el comando `\spnum`. Este comando está dedicado exclusivamente al manejo de «*números enteros*», y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para `\spnZ`

El comando `\spnZ` posee las *meta-keys* `cifras-sep` y `read-sign` pasadas al comando `\spnum` (§3.1.1).

`wrap-parens` = { `<true>` | `<false>` } default: `false`

Establece si se *imprimen* paréntesis redondos ‘()’ alrededor del argumento {`<número entero>`} pasado a `\spnZ`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor del {`<número entero>`}; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens` = { `<true>` | `<false>` } default: `false`

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘()’ alrededor del *argumento* {`<número entero>`} pasado a `\spnZ`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura del paréntesis ‘()’; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spmQ`

`\spmQ` `\spmQ{<número entero>}{<numerador>}{<denominador>}`
`\spmQ[<key = val>]{<número entero>}{<numerador>}{<denominador>}`

El comando `\spmQ` trabaja en *modo matemático* y recibe *tres argumentos obligatorios*: {`<número entero>`}, {`<numerador>`} y {`<denominador>`}, los procesa internamente y devuelve un «*número mixto*».

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spmQ{2}{5}{7}` imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y cinco séptimos*».

- En el caso de las fracciones, y en especial con los «*números mixtos*», no se pueden manejar muchas cosas desde el lado de **spintent**. Por ejemplo, la entrada `\frac{5}{7}` con *tagged* PDF se verbaliza por NVDA/MathCAT como «*tres más cinco séptimos*», lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para `\spmQ`

`join-word = {⟨entero | enteros | y⟩}`

default: `y`

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el *número mixto*. Si se establece en `entero`, al usar `$\spmQ[join-word=entero]{1}{5}{7}$` imprimirá ‘ $1\frac{5}{7}$ ’, en la salida PDF, y NVDA/MathCAT verbalizará «un entero cinco séptimos»; si se establece en `enteros`, al usar `$\spmQ[join-word=enteros]{2}{5}{7}$` imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF, y NVDA/MathCAT verbalizará «dos enteros cinco séptimos»; si se establece en `y`, al usar `$\spmQ[join-word=y]{2}{5}{7}$` imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF, y NVDA/MathCAT verbalizará «dos y cinco séptimos».

`use-spnZ = {⟨true | false⟩}`

default: `true`

Establece si el `{⟨numerador⟩}` y el `{⟨denominador⟩}` que forman la fracción del *número mixto* se procesan internamente bajo el comando `\spnZ`, añadiendo así *semántica de entero* en la estructura MathML de la salida PDF. Si se establece en `true`, `{⟨numerador⟩}` y `{⟨denominador⟩}` se pasarán a `\spnZ` y NVDA/MathCAT los verbalizará correctamente como *números ordinales*; si se establece en `false`, los argumentos se tipografían directamente sin procesamiento semántico adicional.

`print-dfrac = {⟨true | false⟩}`

default: `false`

Establece si la fracción del *número mixto* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en `true`, al usar `$\spmQ[print-dfrac=true]{2}{5}{7}$` imprimirá la fracción en tamaño *display* ‘ $2\frac{5}{7}$ ’, en la salida PDF; si se establece en `false`, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`wrap-parens = {⟨true | false⟩}`

default: `false`

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor del *número mixto* completo devuelto por `\spmQ`. Si se establece en `true`, se imprimirán paréntesis redondos cuyo tamaño se ajusta automáticamente al estilo matemático en curso; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}`

default: `true`

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(...)’ alrededor del *número mixto* cuando la llave `wrap-parens` está activa. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

7 Utilidades para geometría

El paquete `spintent` provee utilidades para trabajar con geometría (plana), que describiremos en esta sección.

El comando `\spang`

`\spang {⟨grados⟩}`
`\spang {⟨grados : minutos⟩}`
`\spang {⟨grados : minutos : segundos⟩}`

El comando `\spang` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{⟨grados⟩}`, `{⟨grados : minutos⟩}` o `{⟨grados : minutos : segundos⟩}`. Los procesa internamente bajo los comandos `\spnum` y `\spunit` para imprimir y verbalizar el «ángulo sexagesimal». Por ejemplo, `$\spang{45:30:15}$` imprimirá ‘45°30’15” en la salida PDF y NVDA/MathCAT verbalizará «cuarenta y cinco grados treinta minutos quince segundos».

8 Referencias

- [1] ROBERTSON, WILL . “unicode-math - Unicode mathematics support for X_YTeX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/unicode-math>, 2023.
- [2] KRUEGER, MARCEL. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, <https://ctan.org/pkg/luamml>, 2026.
- [3] The L^AT_EX Project. “fontspec - Advanced font selection for X_YL^AT_EX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/fontspec>, 2025.
- [4] VOß, HERBERT. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, <https://www.ctan.org/pkg/libertinus-otf>, 2025.
- [5] FLIPO, DANIEL. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, <https://ctan.org/pkg/erewhon-math>, 2026.
- [6] HOFSTRA, SILKE. “sourcecodepro - Use SourceCodePro with T_EX(-alike) systems”. Available from CTAN, <https://ctan.org/pkg/sourcecodepro>, 2025.
- [7] GONZÁLEZ, PABLO. “scontents - Stores L^AT_EX contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2026.
- [8] GONZÁLEZ, PABLO. “enumext - Enumerate exercise sheets”. Available from CTAN, <https://www.ctan.org/pkg/enumext>, 2026.
- [9] BRAAMS, JOHANNES AND BEZOS, JAVIER. “babel - Multilingual support for L^AT_EX, LuaL^AT_EX, X_YL^AT_EX, and Plain T_EX”. Available from CTAN, <https://www.ctan.org/pkg/babel>, 2026.
- [10] BEZOS, JAVIER. “babel-spanish - Babel support for Spanish”. Available from CTAN, <https://www.ctan.org/pkg/babel-spanish>, 2026.
- [11] NV ACCESS. “NVDA - NonVisual Desktop Access screen reader”. Available from NV ACCESS, <https://www.nvaccess.org/>, 2026.
- [12] SOIFFER, NEIL. “MathCAT - Math to speech and braille translation”. Available from GITHUB, <https://nsoiffer.github.io/MathCAT/>, 2026.
- [13] ADOBE INC. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from ADOBE, <https://get.adobe.com/reader/>, 2026.
- [14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF ASSOCIATION, <https://pdfa.org/resource/iso-32000-2/>, 2020.
- [15] PDF ASSOCIATION. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, <https://pdfa.org/wtpdf/>, 2024.
- [16] PDF ASSOCIATION. “PDF 2.0 Errata Collection 3”. Available from PDFA, <https://pdfa.org/PDF-2-0-errata-collection-3-now-available/>, 2026.
- [17] L^AT_EX PROJECT LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, <https://pdfa.org/resource/best-practice-guide-math-in-PDF/>, 2026.
- [18] JOHNSON, DUFF. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, <https://pdfa.org/PDF-2-0-new-features-real-world-impact/>, 2026.
- [19] The L^AT_EX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from L^AT_EX PROJECT, <https://latex3.github.io/tagging-project/>, 2026.
- [20] VERAPDF CONSORTIUM. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from VERAPDF, <https://verapdf.org/>, 2026.
- [21] DUAL LAB. “ngPDF - Next Generation PDF demonstrator and validator”. Available from NGPDF, <https://ngpdf.com/>, 2026.
- [22] BERRY, KARL. “L^AT_EX 2_ε: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2026.
- [23] The L^AT_EX Project. “Extensive support for hypertext in L^AT_EX”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2026.
- [24] The L^AT_EX Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.

- [25] The $\text{\LaTeX}$ Project. “The $\text{\LaTeX}$ 3 Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.
- [26] The $\text{\LaTeX}$ Project. “The $\text{\LaTeX}$ 2 ϵ sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2026.
- [27] The $\text{\LaTeX}$ Project. “ $\text{\LaTeX}$ News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [28] The $\text{\LaTeX}$ Project. “ $\text{\LaTeX}$ for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2026.
- [29] FISCHER, ULRIKE. “tagpdf – $\text{\LaTeX}$ kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2026.
- [30] The $\text{\LaTeX}$ Project. “latex-lab – $\text{\LaTeX}$ laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2026.

9 Change history

v0.98 (ctan), 2026-08-19 – First public release.

10 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

C

Document class:

article1

book1

letter1

report1

Commands provide by **spintent**:

\setspintent3

\spang17

\spdate11

\spdiv13

\spdsym12

\spmQ16

\spmcd15

\spmcm15

\spmoney6

\spmsym12

\spmul13

\spnD14

\spnM14

\spnZ16

\spnewunit6

\spnum3

\spread11

\spshort7

\psiglo11

\sptime11

\spunitalias6

\spunit4

\spusep12

K

Keys for \spdiv provide by **spintent**:

read-parens13

wrap-parens13

Keys for \spdsym provide by **spintent**:

prefix12

read-sym12

set-sym12

suffix12

Keys for \spmcd provide by **spintent**:

prefix15

read-cmd15

result15

sample-arg15

silent-cmd15

upper-case15

Keys for \spmcm provide by **spintent**:

prefix16

read-cmd16

result16

sample-arg16

silent-cmd16

upper-case16

Keys for \spmoney provide by **spintent**:

currency7

dolar-math7

sub-units7

Keys for \spmQ provide by **spintent**:

join-word17

print-dfrac17

read-parens17

use-spnZ17

wrap-parens17

Keys for \spmsym provide by **spintent**:

not-print13

prefix13

read-sym13

set-sym13

suffix13

Keys for \spmul provide by **spintent**:

read-parens13

wrap-parens13

Keys for \spnD provide by **spintent**:

prefix14

result-num14

result14

sample-arg14

silent-cmd14

Keys for \spnM provide by **spintent**:

prefix14

result-num14

result14

sample-arg14

silent-cmd15

Keys for \spnum provide by **spintent**:

cifras-sep3

notation-sym4

read-period-sep3

read-sign3

Keys for \spnZ provide by **spintent**:

read-parens16

wrap-parens16

Keys for \spshort provide by **spintent**:

read-arg9

small-caps9

Keys for \psiglo provide by **spintent**:

print-era11

Keys for \spunit provide by **spintent**:

print-cdot4

read-cdot4

read-slash4

Keys for \spusep provide by **spintent**:

silent12

size12

Keys for \spdiv provide by **spintent**:

cifras-sep13

notation-sym13

read-period-sep13

read-sign13

read-sym13

set-sym13

Keys for \spdsym provide by **spintent**:

read-sym12

Keys for \spmcd provide by **spintent**:

read-cmd15

Keys for \spmcm provide by **spintent**:

read-cmd16

Keys for \spmoney provide by **spintent**:

cifras-sep7

Keys for \spmQ provide by **spintent**:

wrap-parens17

Keys for \spmsym provide by **spintent**:

read-sym

13

Keys for \spm\ provide by **spintent**:

cifras-sep

13

notation-sym

13

read-period-sep

13

read-sign

13

read-sym

13

set-sym

13

Keys for \spnZ provide by **spintent**:

cifras-sep

16

read-sign

16

Keys for \spunit provide by **spintent**:

cifras-sep

4

notation-sym

4

read-period-sep

4

Keys for \spusep provide by **spintent**:

silent

12

size

12

M

Lua module provide by **spintent**:

spintent.lua

6, 7, 9, 10

P

Packages :

babel-spanish

2

babel

2, 7

documentmetadata-support

1

enumext

1

expl3

3

fontspec

2

fourier-otf

2

hyperref

2

l3keys

3

latex-lab-mathintent

1

libertinus-otf

2

luamml

1

scontents

1

sourcecodepro

2

tagpdf

1, 7, 11

unicode-math

1

11 Implementation

The most recent publicly released version of `spintent` is available at CTAN: <https://www.ctan.org/pkg/spintent>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/spintent/issues>.

All variables and functions defined in this package are “private” and are NOT intended to work or be used by another package or module.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TeXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

11.1 Initial set up

Start the DocStrip guards.

```
1 (*package)
```

Identify the internal prefix (LaTeX DocStrip convention) for `l3doc` class.

```
2 (@@=spintent)
```

- Functions of the form `\__spintent_luafun_some;nn` or variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

11.2 Declaration of the package

First we will make sure we have a minimum (super updated) version of LaTeX to work correctly.

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
```

Now declare the `spintent` package.

```
4 \ProvidesExplPackage {spintent} {2026-08-19} {0.98} {Spanish parse intents}
```

We check if the requirements for running the `spintent` package are met; that is: `\DocumentMetadata` must be active, LuaTeX must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
5 \IfDocumentMetadataTF
6 {
7   \sys_if_engine luatex:TF
8   {
9     \msg_new:nnn { spintent } { package-not-load }
10    {
11      The~'#1'~package~will~be~loaded~as~a~dependency.
12    }
13    \IfPackageLoadedF { unicode-math }
14    {
15      \IfPackageLoadedF { lua-unicode-math }
16      {
17        \msg_info:nnn { spintent } { package-not-load } { unicode-math }
18        \RequirePackage{unicode-math}
19      }
20    }
21    \msg_new:nnn { spintent } { env-success }
22    { Everything~looks~correct,~spintent~will~make~the~intent! }
23    \msg_info:nn { spintent } { env-success }
24  }
25  {
26    \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
27    { The~spintent~package~requires~LuaLaTeX. }
28    { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
29    \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
30  }
31 }
32 {
33   \msg_new:nnnn { spintent } { fatal-metadata-missing }
34   { \token_to_str:N \DocumentMetadata \c_space_tl is~missing. }
35   {
36     The~spintent~package~requires~tagged~PDF~active.\\
37     You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
38   }
39   \msg_fatal:nn { spintent } { fatal-metadata-missing }
40 }
```

11.3 Some kernel variants

Non-standard kernel variants.

```
\str_if_eq:nVTF
\str_if_eq:nVF
\str_uppercase:V
\str_lowercase:V
\seq_set_from_clist:Ne
41 \cs_generate_variant:Nn \str_if_eq:nnTF { V }
42 \cs_generate_variant:Nn \str_if_eq:nnF { V }
43 \cs_generate_variant:Nn \str_uppercase:n { V }
44 \cs_generate_variant:Nn \str_lowercase:n { V }
45 \cs_generate_variant:Nn \seq_set_from_clist:Nn { Ne }
```

(End of definition for `\str_if_eq:nVTF` and others.)

11.4 Loading the Lua module and functions

Now we load the Lua module `spintent.lua` and generate the variants for the functions `\__spintent_luafun_spunit_lookup_alias:V` used by `\spunit` (§11.12), `\__spintent_luafun_spmoney_lookup_data:V` used by `\spmoney` (§11.14), `\__spintent_luafun_spmQ_scale_int:Vn` used by `\spmQ` (§11.27) and `\__spintent_luafun_clean_split_and_set:V` used by `\spnum` (§11.11) defined in it.

```
46 \lua_load_module:n { spintent.lua }
47 \cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
48 \cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
49 \cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { VV, Vn }
50 \cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }
```

(End of definition for `\__spintent_luafun_spunit_lookup_alias:V` and others.)

11.5 Internal copy of the unicode invisible separator

The function `\__spintent_invisible_sep:` is an internal copy of the Unicode invisible separator U-2063 that uses `\luamml_annotate:en` from the package `luamml`[2] and let `<mo>...</mo>` in the `MathML` structure embedded in the PDF output.

```
51 \cs_new_protected:Nn \__spintent_invisible_sep:
52 {
53   \luamml_annotate:en
54   {
55     core = {[0] = 'mo', intent=':silent', "^^^2063"}
56   }
57   {
58     \latelua{}
59   }
60 }
```

The token list constant `\c__spintent_invisible_sep_tl` is an internal copy of the Unicode invisible separator U-2063 using the primitive `\Umathchardef` from `LuaTeX` which leaves `<mi>...</mi>` in the structure `MathML` embedded in the output PDF.

```
61 \hook_gput_code:nnn { begindocument } { spintent }
62 {
63   \Umathchardef \c__spintent_invisible_sep_tl = "0 "0 "2063
64 }
```

- Both copies are used for different purposes, the function `\__spintent_invisible_sep:` will be used to force the start of a `<mrow>` in the structure `MathML`, the token list constant `\c__spintent_invisible_sep_tl` will be used to inject text into the structure `MathML` when we use `\MathMLintent{⟨text⟩}`.

(End of definition for `\__spintent_invisible_sep:` and `\c__spintent_invisible_sep_tl`.)

11.6 Normalize argument and affixes

We normalize the `{⟨argument⟩}` to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

```
\__spintent_collapse_hyphens:N
65 \cs_new_protected:Nn \__spintent_collapse_hyphens:N
66 {
67   \tl_if_in:NnT #1 { -- }
68   {
69     \tl_replace_all:Nnn #1 { -- } { - }
70     \__spintent_collapse_hyphens:N #1
71   }
72 }
```

(End of definition for `\__spintent_collapse_hyphens:N`.)


```

\__spintent_sanitize_affix:Nn
\__spintent_sanitize_affix:cn

```

The function `\__spintent_sanitize_affix:Nn` checks if the `{⟨argument⟩}` passed in ‘#2’ is *blank*. If so, it clears the target token list variable passed in ‘#1’. Otherwise, trimming bounds, converting spaces ‘`\space`’ to hyphens ‘`-`’, collapsing multiple hyphens, and safely stripping leading/trailing hyphens. For example, if you pass a target variable and raw text:

```

\__spintent_sanitize_affix:Nn \l__spintent_spsomesym_prefix_tl{ foo bar baz }

```

The function will “store” the normalized string `foo-bar-baz` inside `\l__spintent_spsomesym_prefix_tl`, ready to be conditionally appended.

```

73 \cs_new_protected:Nn \__spintent_sanitize_affix:Nn
74 {
75   \tl_if_blank:nTF {#2}
76   { \tl_clear:N #1 }
77   {
78     \tl_set:Nx #1 {#2}
79     \tl_trim_spaces:N #1
80     \tl_replace_all:Nnn #1 { ~ } { - }
81     \__spintent_collapse_hyphens:N #1
82     \tl_put_left:Nn #1 { \q_mark }
83     \tl_put_right:Nn #1 { \q_stop }
84     \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
85     \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
86     \tl_remove_all:Nn #1 { \q_mark }
87     \tl_remove_all:Nn #1 { \q_stop }
88   }
89 }
90 \cs_generate_variant:Nn \__spintent_sanitize_affix:Nn { cn }

```

(End of definition for `\__spintent_sanitize_affix:Nn`.)

11.7 The command `\setspintent`

The `\setspintent` command is used to configure `⟨keys⟩` “globally” for the utilities provided by `spintent`.

11.7.1 Internal variables and error messages for `\setspintent`

```

\l__spintent_setspintent_input_arg_cmd_tl
unknown-command-setup

```

The variable `\l__spintent_setspintent_input_arg_cmd_tl` will store the command or command name passed to the “first” `{⟨argument⟩}` ‘#1’ of the command `\setspintent`.

```

91 \tl_new:N \l__spintent_setspintent_input_arg_cmd_tl

```

Error message if the `{⟨argument⟩}` ‘#1’ passed to `\setspintent` is a “non-existent” command.

```

92 \msg_new:nnnn { spintent } { unknown-command-setup }
93 {
94   Cannot~configure~unknown~command~'\exp_not:c{#1}'.
95 }
96 {
97   You~attempted~to~use~\token_to_str:N \setspintent{#1}{...},~but~the~
98   command~\exp_not:c{#1}~is~not~defined.~Please~check~for~typos.
99 }

```

(End of definition for `\l__spintent_setspintent_input_arg_cmd_tl` and `unknown-command-setup`.)

```

\__spintent_setspintent_set_keys:nn
\__spintent_setspintent_set_keys:en

```

The function `\__spintent_setspintent_set_keys:nn` will check if the command passed as `{⟨argument⟩}` exists in ‘#1’ and set the `⟨keys⟩` passed as `{⟨argument⟩}` in ‘#2’ to the correct path; otherwise, it will return an error.

```

100 \cs_new_protected:Nn \__spintent_setspintent_set_keys:nn
101 {
102   \cs_if_exist:cTF { #1 }
103   {
104     \keys_set:nn { spintent / #1 } { #2 }
105   }
106   {
107     \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
108   }
109 }
110 \cs_generate_variant:Nn \__spintent_setspintent_set_keys:nn { en }

```

(End of definition for `\__spintent_setspintent_set_keys:nn`.)

```
\__spintent_setspintent_parse_args:nn
\__spintent_setspintent_parse_args:Vn
```

The function `\__spintent_setspintent_parse_args:nn` will check if the argument passed in `#1` begins with ‘\’ or is just a *command name*. If it begins with ‘\’, it will remove it using `\cs_to_str:N` and then execute `\__spintent_setspintent_set_keys:en`. Otherwise, i.e., if ‘\’ was not passed, it will directly execute `\__spintent_setspintent_set_keys:nn`.

```
111 \cs_new_protected:Nn \__spintent_setspintent_parse_args:nn
112 {
113   \tl_if_single:nTF { #1 }
114   {
115     \token_if_cs:NTF #1
116     {
117       \__spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }
118     }
119     {
120       \__spintent_setspintent_set_keys:nn { #1 } { #2 }
121     }
122   }
123   {
124     \__spintent_setspintent_set_keys:nn { #1 } { #2 }
125   }
126 }
127 \cs_generate_variant:Nn \__spintent_setspintent_parse_args:nn { Vn }
```

(End of definition for `\__spintent_setspintent_parse_args:nn`.)

`\setspintent`

Now we define the user command `\setspintent`.

```
128 \NewDocumentCommand \setspintent { m m }
129 {
130   \tl_if_blank:nTF { #1 }
131   {
132     \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
133   }
134   {
135     \tl_set:Nn \l__spintent_setspintent_input_arg_cmd_tl { #1 }
136     \tl_trim_spaces:N \l__spintent_setspintent_input_arg_cmd_tl
137     \__spintent_setspintent_parse_args:Vn \l__spintent_setspintent_input_arg_cmd_tl { #2 }
138   }
139 }
```

(End of definition for `\setspintent`. This function is documented on page 3.)

11.8 Handling unknown keys

```
\l__spintent_command_name_str
unknown-choice
cmd-key-unknown
cmd-key-value-unknown
```

We define the variable `\l__spintent_command_name_str` which will *store* the “name” of each command within the implementation along with the messages `unknown-choice` used by *(keys)* that use `.choice:n` in their implementation and the messages `cmd-key-unknown`, `cmd-key-value-unknown` used by the *(key)* `unknown` in commands that use `[<key = val>]`.

```
140 \str_new:N \l__spintent_command_name_str
141 \msg_new:nnn { spintent } { unknown-choice }
142 {
143   The~value~'#3'~for~'#1'~key~is~invalid~use~('#2').
144 }
145 \msg_new:nnnn { spintent } { cmd-key-unknown }
146 {
147   The~key~'#1'~is~unknown~by~command~
148   \c_backslash_str \l__spintent_command_name_str \c_space_tl
149   and~is~being~ignored~\msg_line_context:.
150 }
151 {
152   The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
153   does~not~have~a~key~called~'#1'.\\
154   Check~that~you~have~spelled~the~key~name~correctly.
155 }
156 \msg_new:nnnn { spintent } { cmd-key-value-unknown }
157 {
158   The~key~'#1=#2'~is~unknown~by~command~
159   \c_backslash_str \l__spintent_command_name_str \c_space_tl
160   and~is~being~ignored~\msg_line_context:.
161 }
162 {
163   The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
164   does~not~have~a~key~called~'#1'.\\
165   Check~that~you~have~spelled~the~key~name~correctly.
```

```
166 }
```

(End of definition for `\l__spintent_command_name_str` and others.)

```
\__spintent_unknown_keys:n
\__spintent_unknown_keys:nn
```

The internal functions `\__spintent_unknown_keys:n` and `\__spintent_unknown_keys:nn` used by the `<key>` unknown.

```
167 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
168 {
169   \exp_args:NV \__spintent_unknown_keys_cmd:nn \l_keys_key_str {#1}
170 }
171 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
172 {
173   \tl_if_blank:nTF {#2}
174     { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
175     { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }
176 }
```

(End of definition for `\__spintent_unknown_keys:n` and `\__spintent_unknown_keys:nn`.)

11.9 Definition of core base variables

```
\__spintent_luafun_clean_split_and_set:n
\__spintent_luafun_clean_split_and_set:v
```

The function `\__spintent_luafun_clean_split_and_set:n` defined in the Lua module `spintent.lua`, is common to several commands in this implementation. It analyzes, processes, cleans and splits the `{<argument>}` passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical “units”, assigning them directly to `expl3` variables.

This function assigns the states of the variables ‘_str’ or ‘_tl’ described below to `true`, `false`, `error` or `stores` information according to the role it fulfills within the implementation.

(End of definition for `\__spintent_luafun_clean_split_and_set:n`.)

```
\__spintent_luaset_sign_tl
\__spintent_luaset_part_int_tl
\__spintent_luaset_dec_sep_tl
\__spintent_luaset_part_dec_tl
\__spintent_luaset_part_period_tl
\__spintent_luaset_has_integer_str
\__spintent_luaset_has_natural_str
```

The variable `\__spintent_luaset_sign_tl` stores the explicit ‘+’ or ‘−’ sign if present. The variable `\__spintent_luaset_part_int_tl` stores the raw digit sequence of the “integer part”. The variable `\__spintent_luaset_dec_sep_tl` stores the matched ‘,’ or ‘.’ used as the decimal separator. The variable `\__spintent_luaset_part_dec_tl` stores the raw digits of the “finite decimal part” that follow the separator, and the variable `\__spintent_luaset_part_period_tl` stores the digits of the “infinite periodic decimal part” that follow a semicolon ‘;’.

The variable `\__spintent_luaset_has_integer_str` store the state `true` when it is *only* an “integer”. The variable `\__spintent_luaset_has_natural_str` store the state `true` when it is *only* an “natural number”.

```
177 \tl_new:N \__spintent_luaset_sign_tl
178 \tl_new:N \__spintent_luaset_part_int_tl
179 \tl_new:N \__spintent_luaset_dec_sep_tl
180 \tl_new:N \__spintent_luaset_part_dec_tl
181 \tl_new:N \__spintent_luaset_part_period_tl
182 \str_new:N \__spintent_luaset_has_integer_str
183 \str_new:N \__spintent_luaset_has_natural_str
```

(End of definition for `\__spintent_luaset_sign_tl` and others.)

```
\__spintent_luaset_only_part_int_str
\__spintent_luaset_only_part_dec_str
\__spintent_luaset_only_part_period_str
\__spintent_luaset_format_part_int_str
\__spintent_luaset_format_part_dec_str
```

The literal entries of the numbers that we will pass to the *intent function* :number of `\MathMLintent` are stored in `\__spintent_luaset_only_part_int_str` for the “integer part”, in `\__spintent_luaset_only_part_dec_str` for the “finite decimal part”, and in `\__spintent_luaset_only_part_period_str` for the “infinite periodic decimal part”.

Formatted versions (`cifras-sep=true`) are stored in `\__spintent_luaset_format_part_int_str` for the “integer part” and `\__spintent_luaset_format_part_dec_str` for the “finite decimal part”.

```
184 \str_new:N \__spintent_luaset_only_part_int_str
185 \str_new:N \__spintent_luaset_only_part_dec_str
186 \str_new:N \__spintent_luaset_only_part_period_str
187 \str_new:N \__spintent_luaset_format_part_int_str
188 \str_new:N \__spintent_luaset_format_part_dec_str
```

(End of definition for `\__spintent_luaset_only_part_int_str` and others.)

```
\__spintent_luaset_millions_str
\__spintent_luaset_decimal_period_str
\__spintent_luaset_not_decimal_period_str
\__spintent_luaset_not_decimal_not_period_str
\__spintent_luaset_dec_is_all_zeros_str
\__spintent_luaset_has_exponent_str
\__spintent_luaset_exponent_tl
```

The flag variable `\__spintent_luaset_millions_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in MathML, we use the flag variables `\__spintent_luaset_decimal_period_str`, `\__spintent_luaset_not_decimal_period_str`, and `\__spintent_luaset_not_decimal_not_period_str`. The flag variable `\__spintent_luaset_dec_is_all_zeros_str` monitors whether the “finite decimal part” is completely zero (useful for “pure periodic decimals”).

The flag variable `\l__spintent_luaset_has_exponent_str` indicates whether active “scientific notation” was detected at the end of the entered number, passed with ‘e’ or ‘E’. If true, it will store the digits of the exponent in `\l__spintent_luaset_exponent_tl`.

```

189 \str_new:N \l__spintent_luaset_millions_str
190 \str_new:N \l__spintent_luaset_decimal_period_str
191 \str_new:N \l__spintent_luaset_not_decimal_period_str
192 \str_new:N \l__spintent_luaset_not_decimal_not_period_str
193 \str_new:N \l__spintent_luaset_dec_is_all_zeros_str
194 \str_new:N \l__spintent_luaset_has_exponent_str
195 \tl_new:N \l__spintent_luaset_exponent_tl

```

(End of definition for `\l__spintent_luaset_millions_str` and others.)

```

\l__spintent_luaset_has_units_str
\l__spintent_luaset_status_str
\l__spintent_luaset_arg_above_tl
\l__spintent_luaset_arg_below_tl
\l__spintent_luaset_denom_has_number_str

```

Since the function `\__spintent_luafun_clean_split_and_set:n` is common to `\spnum` (§11.11) and `\spunit` (§11.12) (as well as others), it determines the following unit-related variables after processing the `{⟨arguments⟩}` via the Lua module `spintent.lua`.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical “units”. It sets the flag variable `\l__spintent_luaset_has_units_str` accordingly. Following this, it verifies if this text contains more than one “/”. It then sets the variable `\l__spintent_luaset_status_str` to `valid` or `multislash`. If the structure is `valid`, it splits the unit into `\l__spintent_luaset_arg_above_tl` and `\l__spintent_luaset_arg_below_tl`. Finally, it analyzes `\l__spintent_luaset_arg_below_tl`; if the denominator starts with a number, it sets `\l__spintent_luaset_denom_has_number_str` to `true`, otherwise to `false`.

```

196 \str_new:N \l__spintent_luaset_has_units_str
197 \str_new:N \l__spintent_luaset_status_str
198 \tl_new:N \l__spintent_luaset_arg_above_tl
199 \tl_new:N \l__spintent_luaset_arg_below_tl
200 \str_new:N \l__spintent_luaset_denom_has_number_str

```

(End of definition for `\l__spintent_luaset_has_units_str` and others.)

11.10 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```

201 \msg_new:nnnn { spintent } { need-math-mode }
202 {
203   Math~mode~required~for~'#1'~\msg_line_context:.
204 }
205 {
206   Use~inside~$...$~or~in~math~environment.
207 }
208 \msg_new:nnnn { spintent } { need-text-mode }
209 {
210   Text~mode~required~for~'#1'~\msg_line_context:.
211 }
212 {
213   Use~outside~of~$...$~or~math~environment.
214 }

```

11.11 The command `\spnum`

The user command `\spnum` is *first* of “the big four” provided by the `spintent` package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module `spintent.lua`, which is responsible for separating and assigning variables defined in `expl3`.

From the `expl3` side, we handle the printing logic and the `intent` functions passed to `\MathMLintent` to achieve a valid MathML structure that is accepted by MathCAT.

11.11.1 Definition of keys and variables for `\spnum`

```

\l__spintent_spnum_read_terse_bool
\l__spintent_spnum_read_period_sep_str
\l__spintent_spnum_notation_sym_tl
\l__spintent_spnum_intent_read_sign_str
\l__spintent_spnum_intent_read_dec_sep_str

```

The variable `\l__spintent_spnum_read_terse_bool` handled by the key `read-sign`, controls the `intent` passed to `\MathMLintent` to spoken the ‘+’ or ‘−’. If set to `clear`, it will say “positivo” or “negativo” *after* reading the number; if set to `terse`, it will say “más” or “menos” *before* reading the number.

The variable `\l__spintent_spnum_read_period_sep_str` handled by the key `read-period-sep`, controls how the decimal separator ‘;’ (periodic part) is printed and spoken when the `{⟨argument⟩}` is a *pure decimal periodic* and is passed the `intent` to `\MathMLintent`. If set to `coma`, it will say “coma” and print ‘,’; if set to `punto` it will say “punto” and print ‘.’.

The variable `\l__spintent_spnum__notation_sym_tl`, handled by the key `notation-sym`, determines whether ‘×’ will be printed when set to `times` or ‘·’ when set to `cdot`.

The variable `\l__spintent_spnum_intent_read_sign_str` will *store* the spoken translation used in the `intent` function passed to `\MathMLintent` for the ‘+’ or ‘-’ according to the value passed to the key `read-sign`.

The variable `\l__spintent_spnum_intent_read_dec_sep_str` *stores* the spoken translation that will be used in the `intent` function passed to `\MathMLintent` for reading the decimal separation as a “coma” or a “punto” according to the value found in the variable `\l__spintent_luaset_dec_sep_tl`.

```

215 \bool_new:N \l__spintent_spnum_read_terse_bool
216 \str_new:N \l__spintent_spnum_read_period_sep_str
217 \tl_new:N \l__spintent_spnum_notation_sym_tl
218 \str_new:N \l__spintent_spnum_intent_read_sign_str
219 \str_new:N \l__spintent_spnum_intent_read_dec_sep_str

```

(End of definition for `\l__spintent_spnum_read_terse_bool` and others.)

```

cifras-sep
read-sign
read-period-sep
notation-sym

```

Now we add the keys `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym`.

```

220 \keys_define:nn { spintent/spnum }
221 {
222   cifras-sep .bool_set:N = \l__spintent_spnum_cifras_sep_bool,
223   cifras-sep .initial:n = true,
224   cifras-sep .value_required:n = true,
225   read-sign .choices:nn =
226     { terse , clear }
227     {
228       \int_case:nn { \l_keys_choice_int }
229       {
230         {1} { \bool_set_true:N \l__spintent_spnum_read_terse_bool }
231         {2} { \bool_set_false:N \l__spintent_spnum_read_terse_bool }
232       }
233     },
234   read-sign .initial:n = terse,
235   read-sign .value_required:n = true,
236   read-sign / unknown .code:n =
237     \msg_error:nneee { spintent } { unknown-choice }
238     { read-sign } { terse , ~clear } { \exp_not:n {#1} },
239   read-period-sep .choices:nn =
240     { coma , punto }
241     {
242       \int_case:nn { \l_keys_choice_int }
243       {
244         {1} { \str_set:Nn \l__spintent_spnum_read_period_sep_str { coma } }
245         {2} { \str_set:Nn \l__spintent_spnum_read_period_sep_str { punto } }
246       }
247     },
248   read-period-sep .initial:n = coma,
249   read-period-sep .value_required:n = true,
250   read-period-sep / unknown .code:n =
251     \msg_error:nneee { spintent } { unknown-choice }
252     { read-period-sep } { coma , ~punto } { \exp_not:n {#1} },
253   notation-sym .choices:nn =
254     { times , cdot }
255     {
256       \int_case:nn { \l_keys_choice_int }
257       {
258         {1} { \tl_set:Nn \l__spintent_spnum_notation_sym_tl { \times } }
259         {2} { \tl_set:Nn \l__spintent_spnum_notation_sym_tl { \cdot } }
260       }
261     },
262   notation-sym .initial:n = cdot,
263   notation-sym .value_required:n = true,
264   notation-sym / unknown .code:n =
265     \msg_error:nneee { spintent } { unknown-choice }
266     { notation-sym } { times , ~cdot } { \exp_not:n {#1} },
267   unknown .code:n =
268     {
269       \str_set:Nn \l__spintent_command_name_str { spnum }
270       \__spintent_unknown_keys_cmd:n {#1}
271     },
272 }

```

(End of definition for `cifras-sep` and others.)

11.11.2 Internal functions for \spnum

```

__spintent_spnum_render_part:n
__spintent_spnum_render_int:
__spintent_spnum_render_dec:

```

The function `\__spintent_spnum_render_part:n` processes the $\langle argument \rangle$ passed in ‘#1’ used by the functions `\__spintent_spnum_render_int:` and `\__spintent_spnum_render_dec:` to define the *intent function* :number for the integer and decimal part passed to `\MathMLintent`. It evaluates whether the key `cifras-sep` is active or not.

```

273 \cs_new_protected:Nn \__spintent_spnum_render_part:n
274 {
275   \MathMLintent { \str_use:c { l__spintent_luaset_only_part_#1_str } :number }
276   {
277     \bool_if:NTF \__spintent_spnum_cifras_sep_bool
278     { \str_use:c { l__spintent_luaset_format_part_#1_str } }
279     { \tl_use:c { l__spintent_luaset_part_#1_tl } }
280   }
281 }
282 \cs_new_protected:Nn \__spintent_spnum_render_int:
283 {
284   \__spintent_spnum_render_part:n { int }
285 }
286 \cs_new_protected:Nn \__spintent_spnum_render_dec:
287 {
288   \__spintent_spnum_render_part:n { dec }
289 }

```

(End of definition for `\__spintent_spnum_render_part:n`, `\__spintent_spnum_render_int:`, and `\__spintent_spnum_render_dec:`.)

```

__spintent_spnum_render_only_period:
__spintent_spnum_print_period_bar:
__spintent_spnum_render_period_sep:

```

The function `\__spintent_spnum_render_only_period:` isolates the “*infinite periodic decimal part*” from the “*finite decimal part*”, assigning the *intent function* :number passed to `\MathMLintent` which will be used by `\MathMLarg` in the function `\__spintent_spnum_print_period_bar:`.

```

290 \cs_new_protected:Nn \__spintent_spnum_render_only_period:
291 {
292   \MathMLintent { \l__spintent_luaset_only_part_period_str :number }
293   {
294     \l__spintent_luaset_part_period_tl
295   }
296 }

```

The function `\__spintent_spnum_print_period_bar:` inserts the function `\__spintent_spnum_render_only_p` as an argument within the *intent function* `_periódico:postfix($dp)` passed to `\MathMLintent`, generating the correct spoken for the “*infinite periodic decimal part*”.

```

297 \cs_new_protected:Nn \__spintent_spnum_print_period_bar:
298 {
299   \MathMLintent { _periódico:postfix($dp) }
300   {
301     {
302       \overline
303       {
304         \MathMLarg { dp } { \__spintent_spnum_render_only_period: }
305       }
306     }
307   }
308 }

```

The function `\__spintent_spnum_render_period_sep:` sets how the decimal separator handled by the key `read-period-sep` will be printed and spoken when a “*pure periodic decimal*” is passed.

```

309 \cs_new_protected:Nn \__spintent_spnum_render_period_sep:
310 {
311   \str_if_eq:VnTF \__spintent_spnum_read_period_sep_str { coma }
312   {
313     \MathMLintent { _coma:prefix($pp) }
314     {
315       \mathord{,}
316       \MathMLarg { pp } { \__spintent_spnum_print_period_bar: }
317     }
318   }
319   {
320     \MathMLintent { _punto:prefix($pp) }
321     {
322       \mathord{.}
323       \MathMLarg { pp } { \__spintent_spnum_print_period_bar: }
324     }
325   }

```



```

325     }
326 }

```

(End of definition for `\__spintent_snum_render_only_period:`, `\__spintent_snum_print_period_bar:`, and `\__spintent_snum_render_period_sep:`.)

`\__spintent_snum_render_decimal_sep:`

The function `\__spintent_snum_render_decimal_sep:` parses the value of the variable `\l__spintent_luaset` and sets the variable `\l__spintent_snum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a “coma” or `_punto:prefix($dp)` for reading a “punto” to `\MathMLintent`. We print the decimal separator stored in `\l__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_snum_render_dec:` to print the finite decimal part.

```

327 \cs_new_protected:Nn \__spintent_snum_render_decimal_sep:
328 {
329   \str_if_eq:NnTF \l__spintent_luaset_dec_sep_tl { , }
330   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _coma } }
331   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _punto } }
332   \MathMLintent { \l__spintent_snum_intent_read_dec_sep_str :prefix($dp) }
333   {
334     \mathord { \l__spintent_luaset_dec_sep_tl }
335     \MathMLarg { dp }
336     {
337       \__spintent_invisible_sep:
338       \__spintent_snum_render_dec:
339     }
340   }
341   % period part
342   \str_if_eq:NnTF \l__spintent_luaset_decimal_period_str { true }
343   {
344     \str_if_eq:NnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
345     {
346       \__spintent_invisible_sep:
347       \__spintent_snum_print_period_bar:
348     }
349     {
350       \MathMLintent { _con:prefix($pnum) }
351       {
352         \__spintent_invisible_sep:
353         \MathMLarg { pnum }
354         {
355           \__spintent_snum_print_period_bar:
356         }
357       }
358     }
359   }
360 }

```

(End of definition for `\__spintent_snum_render_decimal_sep:`.)

`\__spintent_snum_print_part_dec:`

`\__spintent_snum_print_part_period:`

The function `\__spintent_snum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (“coma” or “punto”). It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

The function `\__spintent_snum_print_part_period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```

361 \cs_new_protected:Nn \__spintent_snum_print_part_dec:
362 {
363   \tl_if_empty:NF \l__spintent_luaset_part_dec_tl
364   {
365     \__spintent_snum_render_decimal_sep:
366   }
367 }
368 \cs_new_protected:Nn \__spintent_snum_print_part_period:
369 {
370   \tl_if_empty:NF \l__spintent_luaset_part_period_tl
371   {
372     \str_if_eq:NnTF \l__spintent_luaset_not_decimal_period_str { true }
373     { \__spintent_snum_render_period_sep: }
374     {

```

```

375         \tl_if_empty:NT \l__spintent_luaset_part_dec_tl
376         {
377             \__spintent_snum_render_period_sep:
378         }
379     }
380 }
381 }

```

(End of definition for `\__spintent_snum_print_part_dec:` and `\__spintent_snum_print_part_period:`.)

```

\__spintent_snum_print_integer:
\__spintent_snum_print_num_start:
\__spintent_snum_print_number_core:

```

The function `\__spintent_snum_print_integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a `MathML` intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spintent_snum_print_num_start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (“*más*” or “*menos*”) or as a postfix (“*positivo*” or “*negativo*”), and wraps the intent around the rest of the number.

The function `\__spintent_snum_print_number_core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

📌 Note for `\__spintent_invisible_sep:`.

```

382 \cs_new_protected:Nn \__spintent_snum_print_integer:
383 {
384     \str_if_eq:VnTF \l__spintent_luaset_not_decimal_not_period_str { true }
385     { \__spintent_snum_render_int: }
386     {
387         \MathMLintent { \l__spintent_luaset_only_part_int_str :number:prefix($next) }
388         {
389             \bool_if:NTF \l__spintent_snum_cifras_sep_bool
390             { \l__spintent_luaset_format_part_int_str }
391             { \l__spintent_luaset_part_int_tl }
392             \MathMLarg { next }
393             {
394                 \tl_if_empty:NTF \l__spintent_luaset_part_dec_tl
395                 { \__spintent_snum_print_part_period: }
396                 { \__spintent_snum_print_part_dec: }
397             }
398         }
399     }
400 }
401 \cs_new_protected:Nn \__spintent_snum_print_num_start:
402 {
403     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
404     {
405         \__spintent_invisible_sep: % need
406         \__spintent_snum_print_integer:
407     }
408     {
409         \bool_if:NTF \l__spintent_snum_read_terse_bool
410         {
411             \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
412             { \str_set:Nn \l__spintent_snum_intent_read_sign_str { _más:prefix($n) } }
413             { \str_set:Nn \l__spintent_snum_intent_read_sign_str { _menos:prefix($n) } }
414         }
415         {
416             \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
417             { \str_set:Nn \l__spintent_snum_intent_read_sign_str { _positivo:postfix($n) } }
418             { \str_set:Nn \l__spintent_snum_intent_read_sign_str { _negativo:postfix($n) } }
419         }
420         \__spintent_invisible_sep: % need
421         \MathMLintent { \l__spintent_snum_intent_read_sign_str }
422         {
423             \l__spintent_luaset_sign_tl
424             \MathMLarg { n } { \__spintent_snum_print_integer: }
425         }
426     }
427 }
428 \cs_new_protected:Nn \__spintent_snum_print_number_core:
429 {
430     \__spintent_snum_print_num_start:
431     \str_if_eq:VnT \l__spintent_luaset_has_exponent_str { true }

```

```

432     {
433       \MathMLintent { _por } { \l__spintent_spnum_notation_sym_tl }
434       10^{\l__spintent_luaset_exponent_tl }
435     }
436   }

```

(End of definition for `\__spintent_spnum_print_integer:`, `\__spintent_spnum_print_num_start:`, and `\__spintent_spnum_print_number_core:`.)

11.11.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first “error” triggers if the user includes “units” in the $\langle argument \rangle$ passed in ‘#1’, advising them to use the `\spunit` command instead. The second “error” is raised if the provided $\langle argument \rangle$ contains no numerical digits.

```

437 \msg_new:nnnn { spintent } { spnum-has-units }
438 {
439   The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~('#1').
440 }
441 {
442   Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
443 }
444 \msg_new:nnnn { spintent } { spnum-no-digits }
445 {
446   The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value.
447 }
448 {
449   You~provided~'#1',~which~contains~no~digits.
450 }

```

(End of definition for `spnum-has-units` and `spnum-no-digits`.)

11.11.4 Implementation of the `\spnum` command

`\spnum`

The public user command `\spnum` accepts an optional $[\langle key = val \rangle]$ argument in ‘#1’ and a mandatory $\langle rational\ number \rangle$ in ‘#2’. It first checks if it is executed inside math mode, raising an error if it is not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `\__spintent_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `\__spintent_spnum_print_number_core:` to render the final number, and finally closes the group.

```

451 \NewDocumentCommand \spnum { O{} m }
452 {
453   \mode_if_math:TF
454   {
455     \group_begin:
456     \keys_set:nn { spintent/spnum } { #1 }
457     \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
458     \str_if_eq:VnTF \l__spintent_luaset_has_units_str { true }
459     { \msg_error:nnn { spintent } { spnum-has-units } { #2 } }
460     {
461       \tl_if_empty:NTF \l__spintent_luaset_part_int_tl
462       { \msg_error:nnn { spintent } { spnum-no-digits } { #2 } }
463       { \__spintent_spnum_print_number_core: }
464     }
465     \group_end:
466   }
467   { \msg_error:nnn { spintent } { need-math-mode } { \spnum } }
468 }

```

(End of definition for `\spnum`. This function is documented on page 3.)

11.12 The command `\spunit`

The user command `\spunit` is the *second* of “the big four” provided by the `spintent` package and is used to parse and print physical units and angular systems. It sends the $\langle argument \rangle$ to Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n` to separate and clean the $\langle argument \rangle$.

This ensures that all “units” (such as SI fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

11.12.1 Definition of variables

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical “*unit symbol*”, the dictionary lookup status, the correct speech text for NVDA/MathCAT, and specific flags that indicate if the “*unit*” is compact or represents a sexagesimal angle.

```

469 \str_new:N \__spintent_spunit_luaset_canonical_str
470 \str_new:N \__spintent_spunit_luaset_lookup_status_str
471 \str_new:N \__spintent_spunit_luaset_read_str
472 \str_new:N \__spintent_spunit_luaset_is_compact_str
473 \str_new:N \__spintent_spunit_luaset_compact_exp_str
474 \str_new:N \__spintent_spunit_luaset_is_sexagesimal_str
475 \str_new:N \__spintent_spunit_luaset_status_str

```

(End of definition for `\__spintent_spunit_luaset_canonical_str` and others.)

These variables are used to process and format the “*unit*”. The sequences `\__spintent_spunit_mult_seq` and `\__spintent_spunit_exp_seq` split compound “*units*” at multiplication ‘`*`’ or exponent ‘`^`’ boundaries.

The boolean variables control specific rendering options, such as whether the multiplication dot ‘`\cdot`’ is explicitly read aloud or silently skipped.

```

476 \tl_new:N \__spintent_spunit_exponent_tl
477 \tl_new:N \__spintent_spunit_unit_name_tl
478 \str_new:N \__spintent_spunit_read_slash_str
479 \seq_new:N \__spintent_spunit_exp_seq
480 \seq_new:N \__spintent_spunit_mult_seq
481 \bool_new:N \__spintent_spunit_is_mult_bool
482 \bool_new:N \__spintent_spunit_read_cdod_bool

```

(End of definition for `\__spintent_spunit_exponent_tl` and others.)

11.12.2 Definition of error messages

We define three error messages for the `\spunit` command. The first “error” triggers if the “*unit expression*” provided in ‘`#1`’ is unknown or malformed. The second “error” occurs if no “*units*” are found in the $\langle \textit{argument} \rangle$, advising the user to use `\spnum` for pure numbers. The third “error” is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```

483 \msg_new:nnnn { spintent } { spunit-invalid-unit }
484 {
485   The~expression~'#1'~contains~an~unknown~or~malformed~unit~structure.
486 }
487 {
488   Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
489 }
490 \msg_new:nnnn { spintent } { spunit-no-units }
491 {
492   The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1').
493 }
494 {
495   Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
496 }
497 \msg_new:nnnn { spintent } { inline-numeric-denominator }
498 {
499   Illegal~numeric~coefficient~in~inline~denominator~'#1'.
500 }
501 {
502   The~inline~mode~of~\token_to_str:N \spunit \c_space_tl
503   does~not~allow~numbers~after~the~slash~(/).
504 }

```

(End of definition for `spunit-invalid-unit`, `spunit-no-units`, and `inline-numeric-denominator`.)

11.12.3 Definition of keys for `\spunit`

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```

505 \keys_define:nn { spintent/spunit }
506 {
507   print-cdot .bool_set:N = \__spintent_spunit_print_cdod_bool,
508   print-cdot .initial:n = false,
509   print-cdot .value_required:n = true,
510   read-cdot .bool_set:N = \__spintent_spunit_read_cdod_bool,
511   read-cdot .initial:n = false,
512   read-cdot .value_required:n = true,
513   read-slash .choices:nn =

```

```

514     { por , partido-por }
515     {
516         \int_case:nn { \l_keys_choice_int }
517         {
518             {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { por } }
519             {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { partido-por } }
520         }
521     },
522     read-slash .initial:n = por,
523     read-slash .value_required:n = true,
524     read-slash / unknown .code:n =
525     \msg_error:nneee { spintent } { unknown-choice }
526     { read-slash } { por, ~partido-por } { \exp_not:n {#1} },
527     cifras-sep .meta:nn = { spintent/spnum } { cifras-sep = {#1} },
528     cifras-sep .value_required:n = true,
529     read-period-sep .meta:nn = { spintent/spnum } { read-period-sep = {#1} },
530     read-period-sep .value_required:n = true,
531     notation-sym .meta:nn = { spintent/spnum } { notation-sym = {#1} },
532     notation-sym .value_required:n = true,
533     unknown .code:n =
534     {
535         \str_set:Nn \l__spintent_command_name_str { spunit }
536         \__spintent_unknown_keys_cmd:n {#1}
537     },
538 }

```

(End of definition for `read-cdot` and others.)

11.12.4 Internal functions for `\spunit`

`\__spintent_spunit_format_canonical:`
`\__spintent_spunit_process_base_exp:n`

The function `\__spintent_spunit_format_canonical:` handles the visual formatting of the “unit symbol”. It specifically checks for the *liter symbol* ‘ ℓ ’ to print it as `\ell`, and formats all other valid “units” in standard upright text using `\mathrm`.

The function `\__spintent_spunit_process_base_exp:n` evaluates a “single unit” along with its exponent. It first looks up the unit ‘`#1`’ in the dictionary. If the “unit” is not found, it flags it as invalid. If found, it handles the spacing and multiplication dot ‘`\cdot`’ intents, and then renders the base unit and its exponent (either using a pre-defined compact exponent from Lua or by manually splitting the string at the ‘`^`’ symbol).

```

539 \cs_new_protected:Nn \__spintent_spunit_format_canonical:
540 {
541     \str_case:VnF \l__spintent_spunit_luaset_canonical_str
542     {
543         { \ell } { \ell }
544     }
545     { \mathrm { \l__spintent_spunit_luaset_canonical_str } }
546 }
547 \cs_new_protected:Nn \__spintent_spunit_process_base_exp:n
548 {
549     \tl_clear:N \l__spintent_spunit_exponent_tl
550     \__spintent_luafun_spunit_lookup_alias:n { #1 }
551     \str_if_eq:VnTF \l__spintent_spunit_luaset_lookup_status_str { notfound }
552     {
553         \str_set:Nn \l__spintent_luaset_status_str { invalid }
554     }
555     {
556         \bool_if:NF \l__spintent_spunit_is_mult_bool
557         {
558             \bool_if:NTF \l__spintent_spunit_print_cdot_bool
559             {
560                 \bool_if:NTF \l__spintent_spunit_read_cdot_bool
561                 { \MathMLintent { por } { \cdot } }
562                 { \MathMLintent { :silent } { \cdot } }
563             }
564             {
565                 \bool_if:NTF \l__spintent_spunit_read_cdot_bool
566                 { \MathMLintent { por } { \, } }
567                 { \, }
568             }
569         }
570         \str_if_eq:VnTF \l__spintent_spunit_luaset_is_compact_str { true }
571         {
572             {
573                 \MathMLintent { \l__spintent_spunit_luaset_read_str }

```

```

574         {
575             \__spintent_spunit_format_canonical:
576         }
577     }
578     ^ { \l__spintent_spunit_luaset_compact_exp_str }
579 }
580 {
581     \seq_set_split:Nnn \l__spintent_spunit_exp_seq { ^ } { #1 }
582     \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_unit_name_tl
583     \seq_if_empty:NF \l__spintent_spunit_exp_seq
584     {
585         \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_exponent_tl
586     }
587     \MathMLintent { \l__spintent_spunit_luaset_read_str }
588     {
589         \__spintent_spunit_format_canonical:
590     }
591     \tl_if_empty:NF \l__spintent_spunit_exponent_tl
592     {
593         ^ { \l__spintent_spunit_exponent_tl }
594     }
595 }
596 \bool_set_false:N \l__spintent_spunit_is_mult_bool
597 }
598 }

```

(End of definition for `\__spintent_spunit_format_canonical:` and `\__spintent_spunit_process_base_exp:n`.)

`\__spintent_spunit_process_mult:n`
`\__spintent_spunit_process_mult:V`

The function `\__spintent_spunit_process_mult:n` handles compound “units” joined by an explicit multiplication star ‘*’. It splits the $\{\langle argument \rangle\}$ at each “star symbol” and applies `\__spintent_spunit_process_base_exp:n` to evaluate and render each resulting unit individually.

```

599 \cs_new_protected:Nn \__spintent_spunit_process_mult:n
600 {
601     \bool_set_true:N \l__spintent_spunit_is_mult_bool
602     \seq_set_split:Nnn \l__spintent_spunit_mult_seq { * } { #1 }
603     \seq_map_function:NN \l__spintent_spunit_mult_seq \__spintent_spunit_process_base_exp:n
604 }
605 \cs_generate_variant:Nn \__spintent_spunit_process_mult:n { V }

```

(End of definition for `\__spintent_spunit_process_mult:n`.)

`\__spintent_print_spunit_hierarchy:`

The function `\__spintent_print_spunit_hierarchy:` handles the layout of inline “unit” fractions. It first processes the “units” before the slash (the numerator). If a denominator exists, it prints the slash / ‘/’ with its corresponding speech intent, and then processes the remaining “units”.

```

606 \cs_new_protected:Nn \__spintent_print_spunit_hierarchy:
607 {
608     \tl_if_empty:NF \l__spintent_luaset_arg_above_tl
609     {
610         \__spintent_spunit_process_mult:V \l__spintent_luaset_arg_above_tl
611         \tl_if_empty:NF \l__spintent_luaset_arg_below_tl
612         {
613             \MathMLintent { \l__spintent_spunit_read_slash_str } { / }
614             \__spintent_spunit_process_mult:V \l__spintent_luaset_arg_below_tl
615         }
616     }
617 }

```

(End of definition for `\__spintent_print_spunit_hierarchy:`.)

`\__spintent_spunit_safe_exec:n`

The function `\__spintent_spunit_safe_exec:n` performs the main validation and layout for “units”. It first checks if the $\{\langle argument \rangle\}$ has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate “errors” if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal “units”, it prints the number and the symbol “without any space in between”, applying a slight negative space ‘\!’ for arcseconds. For standard units, it prints the number followed by a thin space ‘\,’ and then processes the unit sequence.

```

618 \cs_new_protected:Nn \__spintent_spunit_safe_exec:n
619 {
620     \str_if_eq:VnTF \l__spintent_luaset_has_units_str { false }
621     { \msg_error:nnn { spintent } { spunit-no-units } { #1 } }
622     {

```



```

623 \str_if_eq:VnTF \l__spinttent_luaaset_status_str { valid }
624 {
625   \str_if_eq:VnTF
626     \l__spinttent_luaaset_denom_has_number_str { true }
627     {
628       \msg_error:nnn { spinttent }
629       { inline-numeric-denominator } { #1 }
630     }
631   {
632     \l__spinttent_luafun_spunit_lookup_alias:V \l__spinttent_luaaset_arg_above_tl
633     \str_if_eq:VnTF \l__spinttent_spunit_luaaset_is_sexagesimal_str { true }
634     {
635       \tl_if_empty:NF \l__spinttent_luaaset_part_int_tl
636       {
637         \__spinttent_spnum_print_number_core:
638       }
639       \MathMLintent { \l__spinttent_spunit_luaaset_read_str }
640       {
641         \mathord
642         {
643           \str_case:Vn \l__spinttent_spunit_luaaset_canonical_str
644             {
645               { " } { \prime \! \prime }
646               { ' } { \prime }
647               { ° } { ° }
648             }
649           }
650         }
651       }
652     {
653       \tl_if_empty:NF \l__spinttent_luaaset_part_int_tl
654       {
655         \__spinttent_spnum_print_number_core: \,
656       }
657       \__spinttent_print_spunit_hierarchy:
658     }
659   }
660 }
661 { \msg_error:nnn { spinttent } { spunit-invalid-unit } { #1 } }
662 }
663 }

```

(End of definition for `\__spinttent_spunit_safe_exec:n`.)

11.12.5 Implementation of the `\spunit` command

`\spunit` The public user command `\spunit` accepts an optional [*key* = *val*] argument in ‘#1’ and a mandatory {*unit expression*} in ‘#2’. It first checks if it is executed inside *math mode*, raising an “error” if it is not. If correct, it opens a local group to apply the options and passes the {*argument*} to Lua module for parsing via `\__spinttent_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spinttent_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```

664 \NewDocumentCommand \spunit { O{} m }
665 {
666   \mode_if_math:TF
667   {
668     \group_begin:
669     \keys_set:nn { spinttent/spunit } { #1 }
670     \__spinttent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
671     \__spinttent_spunit_safe_exec:n { #2 }
672     \group_end:
673   }
674   { \msg_error:nnn { spinttent } { need-math-mode } { \spunit } }
675 }

```

(End of definition for `\spunit`. This function is documented on page 4.)

11.13 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom “units” or “aliases” to the package’s dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spinttent.lua` via `\__spinttent_luafun_define_custom_unit:nn` and `\__spinttent_luafun_define_spunit`

During the definition, the Lua module checks if the new “*unit*” or “*alias*” is valid and ensures it does not conflict with existing “*units*”. It returns the result of this operation to T_EX using the `\l__spintent_spunit_luaset_status_str` variable. If a collision is detected or the $\langle argument \rangle$ is invalid, the package throws the corresponding “error” message.

11.13.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of “*units*” and “*aliases*”. The first two “errors” trigger if the user attempts to define a “*new unit*” or alias passed in ‘*#1*’ that already exists in the dictionary. The third “error” is raised by `\spunitalias` if the target expression provided in ‘*#2*’ is invalid (for example, if it contains unknown “*units*” or incorrect syntax).

```

676 \msg_new:nnnn { spintent } { spnewunit-duplicate }
677 {
678   The~unit~symbol~or~alias~'~#1'~is~already~defined!
679 }
680 {
681   You~cannot~redefine~with~\token_to_str:N \spnewunit.
682 }
683 \msg_new:nnnn { spintent } { spunitalias-duplicate }
684 {
685   The~alias~'~#1'~is~already~defined~or~reserved!
686 }
687 {
688   You~cannot~redefine~an~existing~unit~or~another~alias.
689 }
690 \msg_new:nnnn { spintent } { spunitalias-invalid-expr }
691 {
692   The~expression~'~#2'~for~alias~'~#1'~is~invalid.
693 }
694 {
695   It~must~contain~only~defined~units,~exponents~and~one~'/'~.
696 }

```

(End of definition for `spnewunit-duplicate`, `spunitalias-duplicate`, and `spunitalias-invalid-expr`.)

`\spnewunit`

The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol ‘*#1*’ and its speech text ‘*#2*’ to Lua via `\__spintent_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```

697 \NewDocumentCommand \spnewunit { m m }
698 {
699   \mode_if_math:TF
700   {
701     \msg_error:nnn { spintent } { need-text-mode } { \spnewunit }
702   }
703   {
704     \__spintent_luafun_define_custom_unit:nn { #1 } { #2 }
705     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
706     {
707       \msg_error:nnn { spintent } { spnewunit-duplicate } { #1 }
708     }
709   }
710 }

```

(End of definition for `\spnewunit`. This function is documented on page 6.)

`\spunitalias`

The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name ‘*#1*’ and the target expression ‘*#2*’ to Lua via `\__spintent_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```

711 \NewDocumentCommand \spunitalias { m m }
712 {
713   \mode_if_math:TF
714   {
715     \msg_error:nnn { spintent } { need-text-mode } { \spunitalias }
716   }
717   {
718     \__spintent_luafun_define_spunit_alias:nn { #1 } { #2 }
719     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
720     {
721       \msg_error:nnn { spintent } { spunitalias-duplicate } { #1 }
722     }
723   }

```

```

723     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { invalid-expr }
724     {
725         \msg_error:nnnn { spintent } { spunitalias-invalid-expr } { #1 } { #2 }
726     }
727 }
728 }

```

(End of definition for `\spunitalias`. This function is documented on page 6.)

11.14 The command `\spmoney`

The user command `\spmoney` is the *third* of “the big four” provided by the `spintent` package, designed to format monetary values and currencies. It delegates $\{\langle argument \rangle\}$ parsing to the Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n`. The module then retrieves the correct “currency symbol”, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default “currency symbol” or enable “subunit” rendering. Finally, it constructs the MathML intent for MathCAT.

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the “currency symbol”, its layout position (pre or post), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. T_EX then uses this information to render the visual layout and construct the correct MathML speech intents.

```

\l__spintent_spmoney_luaset_symbol_str
\l__spintent_spmoney_luaset_position_str
\l__spintent_spmoney_luaset_sing_str
\l__spintent_spmoney_luaset_plur_str
\l__spintent_spmoney_luaset_conde_str
\l__spintent_spmoney_luaset_subsing_str
\l__spintent_spmoney_luaset_subplur_str
\l__spintent_spmoney_luaset_print_iso_str
\l__spintent_spmoney_luaset_currency_arg_str
\l__spintent_spmoney_luaset_status_str

```

```

729 \str_new:N \l__spintent_spmoney_luaset_symbol_str
730 \str_new:N \l__spintent_spmoney_luaset_position_str
731 \str_new:N \l__spintent_spmoney_luaset_sing_str
732 \str_new:N \l__spintent_spmoney_luaset_plur_str
733 \str_new:N \l__spintent_spmoney_luaset_conde_str
734 \str_new:N \l__spintent_spmoney_luaset_subsing_str
735 \str_new:N \l__spintent_spmoney_luaset_subplur_str
736 \str_new:N \l__spintent_spmoney_luaset_print_iso_str
737 \str_new:N \l__spintent_spmoney_luaset_currency_arg_str
738 \str_new:N \l__spintent_spmoney_luaset_status_str

```

(End of definition for `\l__spintent_spmoney_luaset_symbol_str` and others.)

```

\l__spintent_spmoney_sub_units_bool
\l__spintent_spmoney_intent_money_str
\l__spintent_spmoney_intent_sign_str
\l__spintent_spmoney_intent_sub_unit_str
\l__spintent_spmoney_res_main_voice_str
\l__spintent_spmoney_subnum_int
\l__spintent_spmoney_extracted_curr_str
\l__spintent_spmoney_extracted_num_tl

```

The boolean variable tracks whether subunit rendering is enabled. The string variables assemble the specific speech text (intents) for NVDA/MathCAT, including the sign, the main currency, and the subunits. Finally, the remaining variables temporarily store the numeric value and currency code extracted from the user’s input.

```

739 \bool_new:N \l__spintent_spmoney_sub_units_bool
740 \str_new:N \l__spintent_spmoney_intent_money_str
741 \str_new:N \l__spintent_spmoney_intent_sign_str
742 \str_new:N \l__spintent_spmoney_intent_sub_unit_str
743 \str_new:N \l__spintent_spmoney_res_main_voice_str
744 \int_new:N \l__spintent_spmoney_subnum_int
745 \str_new:N \l__spintent_spmoney_extracted_curr_str
746 \tl_new:N \l__spintent_spmoney_extracted_num_tl

```

(End of definition for `\l__spintent_spmoney_sub_units_bool` and others.)

11.14.1 Definition of error messages

```

currency-no-subunits
invalid-currency

```

We define two messages to handle invalid currency configurations. The first “error” warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second “error” is triggered if the provided currency identifier is not registered in the Lua database.

```

747 \msg_new:nnnn { spintent } { currency-no-subunits }
748 {
749     The~currency~'#1'~does~not~support~subunits.~
750     The~option~'sub-units=true'~is~ignored.
751 }
752 {
753     Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
754     do~not~have~fractional~subunits~registered~in~Lua~module.~
755 }
756 \msg_new:nnnn { spintent } { invalid-currency }
757 {
758     Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney .
759 }
760 {
761     The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
762     the~Lua~module.
763 }

```

(End of definition for `currency-no-subunits` and `invalid-currency`.)

11.14.2 Auxiliary functions for `\spmoney`

```
__spintent_spmoney_set_currency:n
__spintent_spmoney_normalize_key:n
__spintent_spmoney_normalize_key:V
```

The function `__spintent_spmoney_set_currency:n` validates the currency identifier provided in ‘`#1`’ against the database. It delegates the lookup to the helper function `__spintent_spmoney_normalize_key:n`, which interfaces with the Lua module `spintent.lua`. If the module returns a `notfound` status, an “error” is triggered. Otherwise, it stores the valid currency identifier in `\__spintent_spmoney_luaset_currency_arg_str` for subsequent processing.

```
764 \cs_new_protected:Nn __spintent_spmoney_set_currency:n
765 {
766   __spintent_spmoney_normalize_key:n {#1}
767   \str_if_eq:VnTF \__spintent_spmoney_luaset_status_str { notfound }
768   {
769     \msg_error:nnn { spintent } { invalid-currency } {#1}
770   }
771   { \str_set:Nn \__spintent_spmoney_luaset_currency_arg_str {#1} }
772 }
773 \cs_new_protected:Nn __spintent_spmoney_normalize_key:n
774 {
775   __spintent_luafun_spmoney_normalize_key:n { #1 }
776 }
777 \cs_generate_variant:Nn __spintent_spmoney_normalize_key:n { V }
```

(End of definition for `__spintent_spmoney_set_currency:n` and `__spintent_spmoney_normalize_key:n`.)

11.14.3 Definition of keys for `\spmoney`

```
currency
sub-units
dolar-math
cifras-sep
```

Now we add the keys `currency`, `sub-units`, `dolar-math` and `cifras-sep`.

```
778 \keys_define:nn { spintent / spmoney }
779 {
780   currency      .code:n      = { __spintent_spmoney_set_currency:n {#1} },
781   currency      .initial:n    = { pesos },
782   currency      .value_required:n = true,
783   sub-units     .bool_set:N   = \__spintent_spmoney_sub_units_bool,
784   sub-units     .value_required:n = true,
785   dolar-math    .bool_set:N   = \__spintent_spmoney_dolar_math_bool,
786   dolar-math    .initial:n    = { true },
787   dolar-math    .value_required:n = true,
788   cifras-sep    .meta:nn      = { spintent/spnum } { cifras-sep = {#1} },
789   cifras-sep    .value_required:n = true,
790   unknown       .code:n      =
791   {
792     \str_set:Nn \__spintent_command_name_str { spmoney }
793     \__spintent_unknown_keys_cmd:n {#1}
794   },
795 }
```

(End of definition for `currency` and others.)

11.14.4 Internal functions for `\spmoney`

```
__spintent_spmoney_validate_money:
```

The function `__spintent_spmoney_validate_money:` verifies if the parsed input contains an embedded currency identifier by checking `\__spintent_luaset_has_units_str`. If a currency is detected, it extracts and normalizes the key from `\__spintent_luaset_arg_above_tl`. It throws an “error” if the currency is not found in the database. If valid, it updates `\__spintent_spmoney_luaset_currency_arg_str` and retrieves the full currency metadata from the `spintent.lua` via `__spintent_luafun_spmoney_lookup_data:V`.

```
796 \cs_new_protected:Nn __spintent_spmoney_validate_money:
797 {
798   \str_if_eq:VnT \__spintent_luaset_has_units_str { true }
799   {
800     __spintent_spmoney_normalize_key:V \__spintent_luaset_arg_above_tl
801     \str_if_eq:VnTF \__spintent_spmoney_luaset_status_str { notfound }
802     { \msg_error:nne { spintent } { invalid-currency } { \__spintent_luaset_arg_above_tl }
803     {
804       \str_set:NV
805         \__spintent_spmoney_luaset_currency_arg_str
806         \__spintent_luaset_arg_above_tl
807     }
808   }
809   \str_set:Ne \__spintent_spmoney_luaset_status_str { found }
810   __spintent_luafun_spmoney_lookup_data:V \__spintent_spmoney_luaset_currency_arg_str
811 }
```

(End of definition for `\__spintent_spmoney_validate_money:`)

`\__spintent_spmoney_print_symbol:`

The function `\__spintent_spmoney_print_symbol:` renders the “*currency symbol*”. If `\l__spintent_spmoney_luase` is `true`, it outputs the ISO code using `\mathrm`. Otherwise, it prints the “*standard symbol*”. For dollar signs ‘\$’, it checks `\l__spintent_spmoney_dolar_math_bool` to determine whether to render it directly as a math character ‘\$’ or wrap it inside `\text`.

```

812 \cs_new_protected:Nn \__spintent_spmoney_print_symbol:
813 {
814   \str_if_eq:VnTF \l__spintent_spmoney_luaset_print_iso_str { true }
815   { \mathrm { \l__spintent_spmoney_luaset_currency_arg_str } }
816   {
817     \str_if_eq:VnTF \l__spintent_spmoney_luaset_symbol_str { $ }
818     {
819       \bool_if:NTF \l__spintent_spmoney_dolar_math_bool
820       { \$ } { \text { \$ } }
821     }
822     { \l__spintent_spmoney_luaset_symbol_str }
823   }
824 }
```

(End of definition for `\__spintent_spmoney_print_symbol:`)

`\__spintent_spmoney_render_layout:`

The function `\__spintent_spmoney_render_layout:` determines the layout order of the “*currency symbol*” relative to the numeric value. It evaluates `\l__spintent_spmoney_luaset_position_str`; if set to `pre`, it renders the “*currency symbol*” *before* the number node. Otherwise, it places the number first, followed by the “*currency symbol*”.

```

825 \cs_new_protected:Nn \__spintent_spmoney_render_layout:
826 {
827   \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
828   {
829     \__spintent_spmoney_print_symbol:
830     \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
831   }
832   {
833     \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
834     \__spintent_spmoney_print_symbol:
835   }
836 }
```

(End of definition for `\__spintent_spmoney_render_layout:`)

`\__spintent_spmoney_render_number_node:`

`\__spintent_spmoney_render_only_integer_node:`

The function `\__spintent_spmoney_render_number_node:` renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintent_spmoney_render_only_integer_node:` outputs exclusively the integer portion, omitting any decimal data.

```

837 \cs_new_protected:Nn \__spintent_spmoney_render_number_node:
838 {
839   \__spintent_spmoney_render_int:
840   \__spintent_spmoney_print_part_dec:
841 }
842 \cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
843 {
844   \__spintent_spmoney_render_int:
845 }
```

(End of definition for `\__spintent_spmoney_render_number_node:` and `\__spintent_spmoney_render_only_integer_node:`)

`\__spintent_spmoney_print_int:`

`\__spintent_spmoney_intent_int:`

The function `\__spintent_spmoney_print_int:` evaluates the presence of an explicit sign in `\l__spintent_luase`. If a sign is detected, it assigns a prefix speech intent ‘más’ or ‘menos’ and embeds the sign and subsequent layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

The function `\__spintent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional ‘de’ variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix `MathML` intent over the layout rendered by `\__spintent_spmoney_render_layout:`.

```

846 \cs_new_protected:Nn \__spintent_spmoney_print_int:
847 {
848   \tl_if_empty:NTF \l__spintent_luaset_sign_tl
849   {
850     \bool_if:NTF \l__spintent_spmoney_sub_units_bool
851     { \__spintent_spmoney_set_and_print_sub_units: }
```

```

852         { \__spintent_spmoney_intent_int: }
853     }
854     {
855         \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
856         {
857             \str_set:Nn \l__spintent_spmoney_intent_sign_str { _más:prefix ($n) }
858         }
859         {
860             \str_set:Nn \l__spintent_spmoney_intent_sign_str { _menos:prefix ($n) }
861         }
862         \__spintent_invisible_sep:
863         \MathMLintent { \l__spintent_spmoney_intent_sign_str }
864         {
865             \l__spintent_luaset_sign_tl
866             \MathMLarg { n }
867             {
868                 \bool_if:NTF \l__spintent_spmoney_sub_units_bool
869                 { \__spintent_spmoney_set_and_print_sub_units: }
870                 { \__spintent_spmoney_intent_int: }
871             }
872         }
873     }
874 }
875 \cs_new_protected:Nn \__spintent_spmoney_intent_int:
876 {
877     \tl_if_empty:NTF \l__spintent_luaset_dec_sep_tl
878     {
879         \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
880         {
881             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_sing_str
882         }
883         {
884             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
885         }
886         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
887         {
888             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde_s
889         }
890     }
891     {
892         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
893         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
894         {
895             \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
896             {
897                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde
898             }
899             {
900                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur
901             }
902         }
903     }
904     \__spintent_invisible_sep: % need for MathCAT
905     \MathMLintent { \l__spintent_spmoney_intent_money_str :postfix($n) }
906     {
907         \__spintent_spmoney_render_layout:
908     }
909 }

```

(End of definition for `\__spintent_spmoney_print_int:` and `\__spintent_spmoney_intent_int:`)

`\__spintent_spmoney_check_sub_units:`
`\__spintent_spmoney_set_and_print_sub_units:`

The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports “*subunits*”. If the database returns empty subunit data, it triggers a warning and automatically disables the “*subunit*” rendering flag.

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when “*subunits*” are active. It normalizes the decimal component (for instance, padding a single-digit decimal like ‘5’ to ‘50’), evaluates the correct singular or plural inflections for both the main “*currency*” and the “*subunits*”, and delegates the final MathML tree assembly to `\__spintent_spmoney_build_tree:`.

```

910 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
911 {
912     \str_if_empty:NT \l__spintent_spmoney_luaset_subsing_str

```



```

913     {
914         \msg_warning:nne { spintent } { currency-no-subunits }
915         { \l__spintent_spmoney_luaset_currency_arg_str }
916         \bool_set_false:N \l__spintent_spmoney_sub_units_bool
917     }
918 }
919 \cs_new_protected:Nn \l__spintent_spmoney_set_and_print_sub_units:
920 {
921     \int_compare:nTF { \str_count:N \l__spintent_luaset_only_part_dec_str == 1 }
922     {
923         \int_set:Nn \l__spintent_spmoney_subnum_int
924         { \l__spintent_luaset_only_part_dec_str * 10 }
925         \str_set:Ne \l__spintent_luaset_only_part_dec_str
926         { \l__spintent_luaset_only_part_dec_str 0 }
927     }
928     {
929         \int_set:Nn \l__spintent_spmoney_subnum_int
930         { \l__spintent_luaset_only_part_dec_str }
931     }
932     \int_compare:nTF { \l__spintent_spmoney_subnum_int == 1 }
933     {
934         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
935         { \l__spintent_spmoney_luaset_subsing_str :postfix($sub) }
936     }
937     {
938         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
939         { \l__spintent_spmoney_luaset_subplur_str :postfix($sub) }
940     }
941     \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
942     {
943         \str_set:NV \l__spintent_spmoney_res_main_voice_str
944         \l__spintent_spmoney_luaset_sing_str
945     }
946     {
947         \str_set:NV \l__spintent_spmoney_res_main_voice_str
948         \l__spintent_spmoney_luaset_plur_str
949     }
950     \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
951     {
952         \str_set:NV \l__spintent_spmoney_res_main_voice_str
953         \l__spintent_spmoney_luaset_conde_str
954     }
955     \l__spintent_spmoney_build_tree:
956 }

```

(End of definition for `\l__spintent_spmoney_check_sub_units:` and `\l__spintent_spmoney_set_and_print_sub_units:`)

`\l__spintent_spmoney_build_tree:` The function `\l__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional “currencies”. It arranges the integer component, the decimal separator, and the “*subunit*” component based on the layout position (pre or post) of the “*currency symbol*”.

`\l__spintent_spmoney_print_integer_continue:` The function `\l__spintent_spmoney_print_integer_continue:` isolates the rendering of the integer node with its associated main “*currency*” intent. Furthermore, `\l__spintent_spmoney_print_sep_con:` applies the specific speech intent ‘con’ to the decimal separator, while `\l__spintent_spmoney_print_dec_sub_unit:` binds the computed “*subunit*” intent to the decimal digits, ensuring accurate accessibility output.

```

957 \cs_new_protected:Nn \l__spintent_spmoney_build_tree:
958 {
959     \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
960     {
961         \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
962         {
963             \l__spintent_spmoney_print_symbol:
964             \MathMLarg { n } { \l__spintent_spmoney_render_only_integer_node: }
965         }
966         \l__spintent_spmoney_print_sep_con:
967         \l__spintent_spmoney_print_dec_sub_unit:
968     }
969     {
970         \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
971         {
972             \l__spintent_invisible_sep:
973             \MathMLarg { n } { \l__spintent_spmoney_render_only_integer_node: }

```

```

974     }
975     \__spintent_spmoney_print_sep_con:
976     \__spintent_spmoney_print_dec_sub_unit:
977     \MathMLint { :silent } { \__spintent_spmoney_print_symbol: }
978   }
979 }
980 \cs_new_protected:Nn \__spintent_spmoney_print_integer_continue:
981 {
982   \MathMLint { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
983   {
984     \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
985   }
986 }
987 \cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
988 {
989   \MathMLint { con }
990   {
991     \mathord { \l__spintent_luaset_dec_sep_tl }
992   }
993 }
994 \cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
995 {
996   \MathMLint { \l__spintent_spmoney_intent_sub_unit_str }
997   {
998     \__spintent_invisible_sep:
999     \MathMLarg { sub }
1000     {
1001       \__spintent_spmoney_render_dec:
1002     }
1003   }
1004 }

```

(End of definition for `\__spintent_spmoney_build_tree:` and others.)

`\__spintent_spmoney_parse_and_route:n`

The internal function `\__spintent_spmoney_parse_and_route:n` processes the $\{(\textit{argument})\}$ passed in ‘#1’ using Lua module to extract the numeric value and any “currency” identifier. If a “currency” is found (`\l__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the “currency”. If “subunit” rendering is enabled (`\l__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:`. Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```

1005 \cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
1006 {
1007   \__spintent_luafun_spmoney_prepare_input:n { #1 }
1008   \__spintent_luafun_clean_split_and_set:V \l__spintent_spmoney_extracted_num_tl
1009   \str_if_empty:NF \l__spintent_spmoney_extracted_curr_str
1010   {
1011     \str_set:Ne \l__spintent_luaset_has_units_str { true }
1012     \str_set:NV \l__spintent_luaset_arg_above_tl \l__spintent_spmoney_extracted_curr_str
1013   }
1014   \__spintent_spmoney_validate_money:
1015   \bool_if:NT \l__spintent_spmoney_sub_units_bool
1016   { \__spintent_spmoney_check_sub_units: }
1017   \__spintent_spmoney_print_int:
1018 }

```

(End of definition for `\__spintent_spmoney_parse_and_route:n`.)

`\spmoney`

The public document command `\spmoney` formats monetary values with accessible speech intents. It accepts an optional argument ‘#1’ for local key-value configurations and a mandatory argument ‘#2’ containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local TeX group to safely isolate the key assignments, delegates the raw input string to `\__spintent_spmoney_parse_and_route:n` for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```

1019 \NewDocumentCommand \spmoney { O{} m }
1020 {
1021   \mode_if_math:TF
1022   {
1023     \group_begin:
1024     \keys_set:nn { spintent / spmoney } { #1 }
1025     \__spintent_spmoney_parse_and_route:n { #2 }
1026     \group_end:

```

```

1027     }
1028     { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
1029 }

```

(End of definition for `\spmoney`. This function is documented on page 6.)

11.15 The command `\spshort`

The user command `\spshort` is the *fourth* and *final* of “the big four”. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on MathML attributes, this command uses the PDF *tagging tools* provided by the `tagpdf`[29] package. Specifically, it uses the `E` (expanded) key to ensure that NVDA interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

These internal string variables serve as the primary data interface with the Lua module `spintent.lua`. During processing, the Lua module “stores” within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the `E` key intended for NVDA, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```

1030 \str_new:N \l__spintent_spshort_luaset_status_str
1031 \str_new:N \l__spintent_spshort_luaset_layout_str
1032 \str_new:N \l__spintent_spshort_luaset_expanded_str
1033 \str_new:N \l__spintent_spshort_luaset_output_str
1034 \str_new:N \l__spintent_spshort_luaset_base_str
1035 \str_new:N \l__spintent_spshort_luaset_suffix_str

```

(End of definition for `\l__spintent_spshort_luaset_status_str` and others.)

11.15.1 Definition of error messages for `\spshort`

unknown-abbreviation

The internal “error” message `unknown-abbreviation` is triggered when the command `\spshort` encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided `{⟨argument⟩}` cannot be mapped to any registered database entry in `spintent.lua` nor successfully resolved through the grammatical ordinal expansion rules.

```

1036 \msg_new:nnnn { spintent } { unknown-abbreviation }
1037 { The~abbreviation~or~ordinal~'#1'~is~invalid~or~not~recognized. }
1038 { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }

```

(End of definition for `unknown-abbreviation`.)

11.15.2 Accessibility interface

`\__spintent_spshort_tag_span:nn`

`\__spintent_spshort_tag_span:en`

The internal function `\__spintent_spshort_tag_span:nn` processes the `{⟨argument⟩}` passed in ‘#2’ using the tools provided by `tagpdf`. By utilizing the `tag` and `E` keys, it creates a native PDF /E (#1) structure and expands the `{⟨argument⟩}` passed in ‘#1’ for proper reading with NVDA.

```

1039 \cs_new_protected:Nn \__spintent_spshort_tag_span:nn
1040 {
1041     \tag_mc_end_push:
1042     \tag_struct_begin:n { tag=Span, E={ #1 } }
1043     \tag_mc_begin:n { tag=Span }
1044     \tag_suspend:n { \spshort }
1045     #2
1046     \tag_resume:n { \spshort }
1047     \tag_mc_end:
1048     \tag_struct_end:
1049     \tag_mc_begin_pop:n {}
1050 }
1051 \cs_generate_variant:Nn \__spintent_spshort_tag_span:nn { en }

```

(End of definition for `\__spintent_spshort_tag_span:nn`.)

11.15.3 Definition of keys for `\spshort`

small-caps

read-arg

Now we add the keys `small-caps` and `read-arg`.

```

1052 \keys_define:nn { spintent / spshort }
1053 {
1054     small-caps .bool_set:N = \l__spintent_spshort_smallcaps_bool,
1055     small-caps .initial:n = false,
1056     small-caps .value_required:n = true,
1057     read-arg .str_set:N = \l__spintent_spshort_read_arg_str,
1058     read-arg .initial:n = {},
1059     read-arg .value_required:n = true,
1060     unknown .code:n =
1061     {
1062         \str_set:Nn \l__spintent_command_name_str { spshort }
1063         \__spintent_unknown_keys_cmd:n {#1}

```

```

1064     },
1065 }

```

(End of definition for *small-caps* and *read-arg*.)

11.15.4 Internal functions for \spshort

The internal function `\__spintent_spshort_print:n` acts as the primary formatter for text abbreviations. It clears the string variable `\__spintent_spshort_read_arg_str` and evaluates the optional `<keys>` passed in ‘#1’. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `\__spintent_spshort_process_and_render:n` to process the `<argument>` passed in ‘#2’. Otherwise, if an override is detected, it directly delegates the execution to `\__spintent_spshort_render_bypass:n`.

```

1066 \cs_new_protected:Nn \__spintent_spshort_print:n
1067 {
1068   \str_clear:N \__spintent_spshort_read_arg_str
1069   \keys_set:nn { spintent / spshort } {#1}
1070   \str_if_empty:NTF \__spintent_spshort_read_arg_str
1071   {
1072     \__spintent_spshort_process_and_render:n {#2}
1073   }
1074   {
1075     \__spintent_spshort_render_bypass:n {#2}
1076   }
1077 }

```

(End of definition for `\__spintent_spshort_print:n`.)

The internal function `\__spintent_spshort_process_and_render:n` interfaces with the Lua module `spintent.lua` function `\__spintent_luafun_spshort_process:n` to resolve the abbreviation or ordinal `<argument>` passed in ‘#1’.

It subsequently evaluates the execution state stored within the string variable `\__spintent_spshort_luaset_status` and dynamically routes execution to the corresponding rendering subroutine success, fallback or `error`. Any unrecognized status automatically defaults to the “error” handler.

```

1078 \cs_new_protected:Nn \__spintent_spshort_process_and_render:n
1079 {
1080   \__spintent_luafun_spshort_process:n {#1}
1081   \str_case:VnF \__spintent_spshort_luaset_status_str
1082   {
1083     { success } { \__spintent_spshort_render_success: }
1084     { fallback } { \__spintent_spshort_render_fallback: }
1085     { error } { \__spintent_spshort_render_error:n {#1} }
1086   }
1087   { \__spintent_spshort_render_error:n {#1} }
1088 }

```

(End of definition for `\__spintent_spshort_process_and_render:n`.)

The function `\__spintent_spshort_render_success:` injects the key `E` and executes the targeted typographical layout. The `\__spintent_spshort_render_fallback:` function handles default cases, applying optional formatting such as `SMALL CAPS` to unrecognized or partially resolved terms.

Conversely, the function `\__spintent_spshort_render_bypass:n` directly maps the user-defined “*speech*” to the `<argument>`, bypassing the automated lookup Lua module. Finally, the function `\__spintent_spshort_render_error:n` triggers a structural exception while yielding the unformatted `<argument>` to prevent fatal compilation.

```

1089 \cs_new_protected:Nn \__spintent_spshort_render_success:
1090 {
1091   \__spintent_spshort_tag_span:en
1092   { \__spintent_spshort_luaset_expanded_str }
1093   { \__spintent_spshort_execute_layout: }
1094 }
1095 \cs_new_protected:Nn \__spintent_spshort_render_fallback:
1096 {
1097   \__spintent_spshort_tag_span:en
1098   { \__spintent_spshort_luaset_expanded_str }
1099   {
1100     \bool_if:NTF \__spintent_spshort_smallcaps_bool
1101     { \textsc{ \__spintent_spshort_luaset_output_str } }
1102     { \__spintent_spshort_luaset_output_str }
1103   }
1104 }
1105 \cs_new_protected:Nn \__spintent_spshort_render_bypass:n

```

```

1106 {
1107   \__spintent_spshort_tag_span:en { \__spintent_spshort_read_arg_str } { #1 }
1108 }
1109 \cs_new_protected:Nn \__spintent_spshort_render_error:n
1110 {
1111   \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
1112 }

```

(End of definition for __spintent_spshort_render_success: and others.)

__spintent_spshort_execute_layout:

The internal function `\__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor “*stored*” within the string variable `\__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding $\TeX$ formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional SMALL CAPS based on the active boolean variables.

```

1113 \cs_new_protected:Nn \__spintent_spshort_execute_layout:
1114 {
1115   \str_case:VnF \__spintent_spshort_luaset_layout_str
1116   {
1117     { linear_regular }
1118     {
1119       \bool_if:NTF \__spintent_spshort_smallcaps_bool
1120       { \textsc{ \__spintent_spshort_luaset_output_str } }
1121       { \__spintent_spshort_luaset_output_str }
1122     }
1123     { linear_caps }
1124     { \__spintent_spshort_luaset_output_str }
1125     { linear_mixed }
1126     { \__spintent_spshort_luaset_output_str }
1127     { superscript }
1128     {
1129       \__spintent_spshort_luaset_base_str
1130       \bool_if:NTF \__spintent_spshort_smallcaps_bool
1131       {
1132         \textsuperscript{
1133           \textsc{
1134             \str_lowercase:V \__spintent_spshort_luaset_suffix_str
1135           }
1136         }
1137       }
1138       {
1139         \textsuperscript{ \__spintent_spshort_luaset_suffix_str }
1140       }
1141     }
1142     { small_caps_pure }
1143     {
1144       \bool_if:NTF \__spintent_spshort_smallcaps_bool
1145       {
1146         \textsc{
1147           \str_lowercase:V \__spintent_spshort_luaset_output_str
1148         }
1149       }
1150       { \__spintent_spshort_luaset_output_str }
1151     }
1152     { acronym_long }
1153     { \__spintent_spshort_luaset_output_str }
1154   }
1155   { \__spintent_spshort_luaset_output_str }
1156 }

```

(End of definition for __spintent_spshort_execute_layout:.)

\spshort

The public document command `\spshort` manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts `\mode_leave_vertical:` and establishes a local group to call the function `\__spintent_spshort_print:nn` for processing and printing.

```

1157 \NewDocumentCommand \spshort { 0{} m }
1158 {
1159   \mode_if_math:TF
1160   {
1161     \msg_error:nnn { spintent } { need-text-mode } { \spshort }

```

```

1162     }
1163     {
1164         \mode_leave_vertical:
1165         \group_begin:
1166             \__spintent_spsshort_print:nn {#1} {#2}
1167         \group_end:
1168     }
1169 }

```

(End of definition for \spsshort. This function is documented on page 7.)

11.16 The command \spsiglo

The public document command `\spsiglo` provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as $\langle \textit{argument} \rangle$, interfacing with the Lua module `spintent.lua` to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible MathML intent expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native PDF tagging structure (such as `/Alt (#1)`) containing an `alt` key.

```

\__spintent_spsiglo_luaset_error_str
\__spintent_spsiglo_luaset_arabic_str
\__spintent_spsiglo_luaset_roman_min_str
\__spintent_spsiglo_print_era_str

```

These internal string variables handle data transfer and synchronization with the Lua module `spintent.lua`. They isolate the exception evaluation states, the Arabic numeral required for speech synthesis, the lowercase Roman numeral designated for small caps typesetting via `\textsc`, and the localized chronological era modifier.

```

1170 \str_new:N \__spintent_spsiglo_luaset_error_str
1171 \str_new:N \__spintent_spsiglo_luaset_arabic_str
1172 \str_new:N \__spintent_spsiglo_luaset_roman_min_str
1173 \str_new:N \__spintent_spsiglo_print_era_str

```

(End of definition for __spintent_spsiglo_luaset_error_str and others.)

11.16.1 Definition of error messages for \spsiglo

invalid-century-format

This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module `spintent.lua` cannot map the $\langle \textit{argument} \rangle$ to an integer within the valid range of 1 to 4000, preventing incorrect speech synthesis.

```

1174 \msg_new:nnnn { spintent } { invalid-century-format }
1175 {
1176     Invalid~century~format~in~'#1'.
1177 }
1178 {
1179     The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
1180     Ensure~the~value~is~between~1~and~4000.
1181 }

```

(End of definition for invalid-century-format.)

11.16.2 Definition of keys for \spsiglo

print-era

Now we add the key `print-era`. User configuration determining if an era designation is appended, available values are `none`, `ac` (antes de cristo) and `dc` (después de cristo).

```

1182 \keys_define:nn { spintent / spsiglo }
1183 {
1184     print-era .choices:nn =
1185         { none , ac , dc }
1186     {
1187         \int_case:nn { \l_keys_choice_int }
1188         {
1189             {1} { \str_set:Nn \__spintent_spsiglo_print_era_str { none } }
1190             {2} { \str_set:Nn \__spintent_spsiglo_print_era_str { ac } }
1191             {3} { \str_set:Nn \__spintent_spsiglo_print_era_str { dc } }
1192         }
1193     },
1194     print-era .initial:n = none,
1195     print-era .value_required:n = true,
1196     print-era / unknown .code:n =
1197         \msg_error:nneee { spintent } { unknown-choice }
1198         { print-era } { none, ~ac, ~dc } { \exp_not:n {#1} },
1199     unknown .code:n =
1200     {
1201         \str_set:Nn \__spintent_command_name_str { spsiglo }
1202         \__spintent_unknown_keys_cmd:n {#1}
1203     },
1204 }

```


(End of definition for `print-era`.)

11.16.3 Internal functions for `\spsiglo`

The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate MathML structures when century designations are evaluated within *math mode*. They construct MathML intent structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers `ac` or `dc`. It adds invisible spacing and postfix intents to ensure correct NVDA/MathCAT pronunciation.

```

1205 \cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
1206 {
1207   \MathMLintent { #1:number } { \text { \textsc {#2} } } }
1208 }
1209 \cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
1210 \cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
1211 {
1212   \str_if_eq:nnTF { #1 } { ac }
1213   {
1214     \MathMLintent { antes-de-cristo:postfix($z) }
1215     {
1216       \__spintent_invisible_sep:
1217       \MathMLarg { z }
1218       {
1219         \MathMLintent { #2:number }
1220         {
1221           \text { \textsc {#3}~a.~C. }
1222         }
1223       }
1224     }
1225   }
1226   {
1227     \MathMLintent { después-de-cristo:postfix($z) }
1228     {
1229       \__spintent_invisible_sep:
1230       \MathMLarg { z }
1231       {
1232         \MathMLintent { #2:number }
1233         {
1234           \text { \textsc {#3}~d.~C. }
1235         }
1236       }
1237     }
1238   }
1239 }
1240 \cs_generate_variant:Nn \__spintent_spsiglo_render_with_era:nnn { VVV }

```

(End of definition for `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn`.)

The function `\__spintent_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the `tagpdf` package to enclose the content inside a PDF /Alt (`#1`) structure. By passing the `{\argument}` to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct NVDA/MathCAT pronunciation without relying on MathML intents.

```

1241 \cs_new_protected:Nn \__spintent_spsiglo_tag_span:nn
1242 {
1243   \tag_mc_end_push:
1244   \tag_struct_begin:n { tag=Span, alt={ #1 } }
1245   \tag_mc_begin:n { tag=Span }
1246   \tag_suspend:n { \spsiglo }
1247   #2
1248   \tag_resume:n { \spsiglo }
1249   \tag_mc_end:
1250   \tag_struct_end:
1251   \tag_mc_begin_pop:n {}
1252 }
1253 \cs_generate_variant:Nn \__spintent_spsiglo_tag_span:nn { en }

```

(End of definition for `\__spintent_spsiglo_tag_span:nn`.)

The functions `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n` format century designations based on the active mode. Both first check the “error” status in the variable `\__spintent_spsiglo_lua_set_error_str` returned by the Lua module `spintent.lua` and issue an “error” if validation fails.

If the $\langle argument \rangle$ is valid, the function `\__spintent_spsiglo_print_math:n` operates in *math mode*, applying MathML structures depending on whether an era modifier is present. Conversely, the function `\__spintent_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., “antes de cristo”) for NVDA/MathCAT.

```

1254 \cs_new_protected:Nn \__spintent_spsiglo_print_math:n
1255 {
1256   \str_if_eq:VnTF \__spintent_spsiglo_luaset_error_str { true }
1257   {
1258     \msg_error:nnnn { spintent } { invalid-century-format }
1259     { \spsiglo } { #1 }
1260   }
1261   {
1262     \str_if_eq:VnTF \__spintent_spsiglo_print_era_str { none }
1263     {
1264       \__spintent_spsiglo_render_simple:VV
1265       \__spintent_spsiglo_luaset_arabic_str
1266       \__spintent_spsiglo_luaset_roman_min_str
1267     }
1268     {
1269       \__spintent_spsiglo_render_with_era:VVV
1270       \__spintent_spsiglo_print_era_str
1271       \__spintent_spsiglo_luaset_arabic_str
1272       \__spintent_spsiglo_luaset_roman_min_str
1273     }
1274   }
1275 }
1276 \cs_new_protected:Nn \__spintent_spsiglo_print_text:n
1277 {
1278   \str_if_eq:VnTF \__spintent_spsiglo_luaset_error_str { true }
1279   {
1280     \msg_error:nnnn { spintent } { invalid-century-format }
1281     { \spsiglo } { #1 }
1282   }
1283   {
1284     \str_case:Vn \__spintent_spsiglo_print_era_str
1285     {
1286       { none }
1287       {
1288         \__spintent_spsiglo_tag_span:en
1289         { \__spintent_spsiglo_luaset_arabic_str }
1290         { \textsc { \__spintent_spsiglo_luaset_roman_min_str } }
1291       }
1292       { ac }
1293       {
1294         \__spintent_spsiglo_tag_span:en
1295         { \__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
1296         { \textsc { \__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
1297       }
1298       { dc }
1299       {
1300         \__spintent_spsiglo_tag_span:en
1301         { \__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
1302         { \textsc { \__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
1303       }
1304     }
1305   }
1306 }

```

(End of definition for `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n`)

`\spsiglo` The public command `\spsiglo` is the main interface for century typesetting. It opens a local group and evaluates any optional $\langle keys \rangle$ passed in ‘#1’, then executes the function `\__spintent_luafun_spsiglo_parse:n` defined in the Lua module `spintent.lua` passed in ‘#2’ and formats the output for *math mode* or *text mode*.

```

1307 \NewDocumentCommand \spsiglo { 0{} m }
1308 {
1309   \group_begin:
1310   \keys_set:nn { spintent / spsiglo } { #1 }
1311   \__spintent_luafun_spsiglo_parse:n { #2 }
1312   \mode_if_math:TF
1313   { \__spintent_spsiglo_print_math:n { #2 } }
1314   {

```

```

1315         \mode_leave_vertical:
1316         \__spintent_spsiglo_print_text:n { #2 }
1317     }
1318     \group_end:
1319 }

```

(End of definition for \spsiglo. This function is documented on page 11.)

11.17 The command \sptime

The command `\sptime` is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as HH:MM or HH:MM:SS) using the Lua function `\__spintent_luafun_sptime_parse:n`. This separates the hours, minutes, seconds, and fractions to construct a MathML *intent* tree for NVDA/MathCAT.

```

\__spintent_sptime_luaset_seconds_str
\__spintent_sptime_luaset_has_seconds_str
\__spintent_sptime_luaset_error_str
\__spintent_sptime_luaset_is_fraction_str
\__spintent_sptime_luaset_base_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```

1320 \str_new:N \__spintent_sptime_luaset_seconds_str
1321 \str_new:N \__spintent_sptime_luaset_has_seconds_str
1322 \str_new:N \__spintent_sptime_luaset_error_str
1323 \str_new:N \__spintent_sptime_luaset_is_fraction_str
1324 \str_new:N \__spintent_sptime_luaset_base_str

```

(End of definition for __spintent_sptime_luaset_seconds_str and others.)

11.17.1 Definition of error messages

```
invalid-time-format
```

The error message `invalid-time-format` defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```

1325 \msg_new:nnnn { spintent } { invalid-time-format }
1326 {
1327     Invalid~time~format~in~#1. \\\
1328     The~argument~'#2'~does~not~match~any~supported~time~format.
1329 }
1330 {
1331     Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss. \\\
1332     Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
1333 }

```

(End of definition for `invalid-time-format`.)

11.17.2 Internal functions for \sptime

```
\__spintent_sptime_build_seconds:nn
```

The internal function `\__spintent_sptime_build_seconds:nn` builds the MathML structure for the seconds component of a time expression. It wraps the numeric value ‘#2’ in speech intent tags, applying a connector prefix (passed in ‘#1’) and the “segundos” postfix so it is read correctly by NVDA/MathCAT.

```

1334 \cs_new_protected:Nn \__spintent_sptime_build_seconds:nn
1335 {
1336     \MathMLintent { #1:prefix($z) }
1337     {
1338         \text { : }
1339         \MathMLarg { z }
1340         {
1341             \MathMLintent { segundos:postfix($m) }
1342             {
1343                 \__spintent_invisible_sep:
1344                 \MathMLarg { m }
1345                 {
1346                     \text { #2 }
1347                 }
1348             }
1349         }
1350     }
1351 }

```

(End of definition for `\__spintent_sptime_build_seconds:nn`.)

```
\__spintent_sptime_build_tree:
```

The internal function `\__spintent_sptime_build_tree:` builds the complete MathML structure for the time expression. It checks the string variables `\__spintent_sptime_luaset_has_seconds_str` and `\__spintent_sptime_luaset_is_fraction_str` to determine if the time includes seconds or decimal fractions. Based on these values, it inserts the base time string and either calls `\__spintent_sptime_build_seconds:`

with the appropriate prefix (`'con'` or `'minutos-con'`), or simply appends the `'minutos'` intent tag if no seconds are present.

```

1352 \cs_new_protected:Nn \__spintent_sptime_build_tree:
1353 {
1354   \str_if_eq:VnTF \l__spintent_sptime_luaset_has_seconds_str { true }
1355   {
1356     \MathMLintent { :time }
1357     {
1358       \text { \l__spintent_sptime_luaset_base_str }
1359     }
1360   \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { true }
1361   {
1362     \__spintent_sptime_build_seconds:nn { con }
1363     { \l__spintent_sptime_luaset_seconds_str }
1364   }
1365   {
1366     \__spintent_sptime_build_seconds:nn { minutos-con }
1367     { \l__spintent_sptime_luaset_seconds_str }
1368   }
1369 }
1370 {
1371   \MathMLintent { :time }
1372   {
1373     \text { \l__spintent_sptime_luaset_base_str }
1374   }
1375 \str_if_eq:VnT \l__spintent_sptime_luaset_is_fraction_str { false }
1376 {
1377   \MathMLintent { minutos }
1378   {
1379     \__spintent_invisible_sep:
1380   }
1381 }
1382 }
1383 }

```

(End of definition for `\__spintent_sptime_build_tree:`.)

\sptime The command `\sptime` is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using `\__spintent_luafun_sptime_parse:n` and checks the error status in `\l__spintent_sptime_luaset_error_str`. If an invalid format is detected, it issues the `invalid-time-format` error; otherwise, it calls `\__spintent_sptime_build_tree:` to assemble the MathML output.

```

1384 \NewDocumentCommand \sptime { m }
1385 {
1386   \mode_if_math:TF
1387   {
1388     \group_begin:
1389     \__spintent_luafun_sptime_parse:n { #1 }
1390     \str_if_eq:VnTF \l__spintent_sptime_luaset_error_str { true }
1391     {
1392       \msg_error:nnnn { spintent } { invalid-time-format }
1393       { \sptime } { #1 }
1394     }
1395     {
1396       \__spintent_sptime_build_tree:
1397     }
1398   \group_end:
1399 }
1400 { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
1401 }

```

(End of definition for `\sptime`. This function is documented on page 11.)

11.18 The command `\spdate`

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as `DD/MM/YYYY` or `YYYY-MM-DD`) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a MathML `intent` tree for NVDA/MathCAT.

`\l__spintent_spdate_luaset_output_str`
`\l__spintent_spdate_luaset_error_str`

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```

1402 \str_new:N \l__spintent_spdate_luaset_output_str

```

```
1403 \str_new:N \__spintent_spdate_luaset_error_str
```

(End of definition for `\__spintent_spdate_luaset_output_str` and `\__spintent_spdate_luaset_error_str`.)

11.18.1 Definition of error messages

invalid-date-format

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```
1404 \msg_new:nnnn { spintent } { invalid-date-format }
1405 {
1406   Invalid~date~or~unsupported~format~in~#1. \\\
1407   The~argument~'#2'~does~not~match~a~supported~format.
1408 }
1409 {
1410   Allowed~formats:~DD\slash MM\slash YYYY,~or~DD-MM-YYYY,~or~YYYY-MM-DD. \\\
1411   Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
1412 }
```

(End of definition for `invalid-date-format`.)

11.18.2 Internal functions for `\spdate`

`\__spintent_spdate_process:n`

The internal function `\__spintent_spdate_process:n` parses the input using `\__spintent_luafun_spdate_parse:n` and checks the error status in `\__spintent_spdate_luaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the `MathML intent` structure using the formatted date stored in `\__spintent_spdate_luaset_output_str`.

```
1413 \cs_new_protected:Nn \__spintent_spdate_process:n
1414 {
1415   \__spintent_luafun_spdate_parse:n { #1 }
1416   \str_if_eq:VnTF \__spintent_spdate_luaset_error_str { true }
1417   {
1418     \msg_error:nnnn { spintent } { invalid-date-format }
1419     { \spdate } { #1 }
1420   }
1421   {
1422     \MathMLintent { :date }
1423     {
1424       \text { \__spintent_spdate_luaset_output_str }
1425     }
1426   }
1427 }
```

(End of definition for `\__spintent_spdate_process:n`.)

`\spdate`

The command `\spdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spdate_process:n` to process the input.

```
1428 \NewDocumentCommand \spdate { m }
1429 {
1430   \mode_if_math:TF
1431   {
1432     \group_begin:
1433     \__spintent_spdate_process:n { #1 }
1434     \group_end:
1435   }
1436   { \msg_error:nnn { spintent } { need-math-mode } { \spdate } }
1437 }
```

(End of definition for `\spdate`. This function is documented on page 11.)

11.19 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional `MathCAT` intent. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

11.20 The command `\spusep`

For the use of parentheses, we will provide the command `\spusep` with only two *⟨keys⟩* `size` and `silent`. Most of the time we can use `\left` and `\right` or `\Uleft` and `\Uright`, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

`spusep-empty-arg`
`spusep-invalid-arg`

The first “error” message `spusep-empty-arg` is raised when `\spusep` receives an empty or blank *⟨argument⟩*, the second this is when an input is invalid.

```
1438 \msg_new:nnnn { spintent } { spusep-empty-arg }
1439 { Empty~argument~in~\token_to_str:N \spusep. }
1440 { \token_to_str:N \spusep~requires~a~non-empty~argument. }
1441 \msg_new:nnn { spintent } { spusep-invalid-arg }
1442 {
1443   Invalid~argument~'#1'~for~\token_to_str:N \spusep.
1444 }
```

(End of definition for `spusep-empty-arg` and `spusep-invalid-arg`.)

The variable `\l__spinttent_spusep_arg_tl` stores the *⟨argument⟩*, the variable `\l__spinttent_spusep_left_tl` and `\l__spinttent_spusep_right_tl` stores the parentheses.

```
1445 \tl_new:N \l__spinttent_spusep_arg_tl
1446 \tl_new:N \l__spinttent_spusep_left_tl
1447 \tl_new:N \l__spinttent_spusep_right_tl
```

(End of definition for `\l__spinttent_spusep_arg_tl`, `\l__spinttent_spusep_left_tl`, and `\l__spinttent_spusep_right_tl`.)

`size`
`silent`
`unknown`

We define the *⟨keys⟩* `size` and `silent`.

```
1448 \keys_define:nn { spintent / spusep }
1449 {
1450   size .choices:nn =
1451     { none , big , Big, bigg, Bigg }
1452     {
1453       \int_case:nn { \l_keys_choice_int }
1454       {
1455         { 1 }
1456         {
1457           \tl_clear:N \l__spinttent_spusep_left_tl
1458           \tl_clear:N \l__spinttent_spusep_right_tl
1459         }
1460         { 2 }
1461         {
1462           \tl_set:Nn \l__spinttent_spusep_left_tl { \bigl }
1463           \tl_set:Nn \l__spinttent_spusep_right_tl { \bigr }
1464         }
1465         { 3 }
1466         {
1467           \tl_set:Nn \l__spinttent_spusep_left_tl { \Bigl }
1468           \tl_set:Nn \l__spinttent_spusep_right_tl { \Bigr }
1469         }
1470         { 4 }
1471         {
1472           \tl_set:Nn \l__spinttent_spusep_left_tl { \biggl }
1473           \tl_set:Nn \l__spinttent_spusep_right_tl { \biggr }
1474         }
1475         { 5 }
1476         {
1477           \tl_set:Nn \l__spinttent_spusep_left_tl { \Biggl }
1478           \tl_set:Nn \l__spinttent_spusep_right_tl { \Biggr }
1479         }
1480       }
1481     },
1482   size .initial:n = none,
1483   size .value_required:n = true,
1484   size / unknown .code:n =
1485     \msg_error:nneee { spintent } { unknown-choice }
1486     { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
1487   silent .bool_set:N = \l__spinttent_spusep_silent_bool,
1488   silent .initial:n = false,
1489   silent .value_required:n = true,
1490   unknown .code:n =
1491     {
1492       \str_set:Nn \l__spinttent_command_name_str { spusep }
```

```

1493     \__spintrent_unknown_keys_cmd:n {#1}
1494 },
1495 }

```

(End of definition for `size`, `silent`, and `unknown`.)

__spintrent_spusep_set_parens:nn

The function `\__spintrent_spusep_set_parens:nn` will place the argument passed in ‘#2’ to the right of the variables `\__spintrent_spusep_left_tl` and `\__spintrent_spusep_right_tl` and then check the state of the variable `\__spintrent_spusep_silent_bool` handled by the key `silent` to print.

```

1496 \cs_new_protected:Nn \__spintrent_spusep_set_parens:nn
1497 {
1498   \tl_put_right:cn { \__spintrent_spusep_ #1 _tl } { #2 }
1499   \bool_if:NTF \__spintrent_spusep_silent_bool
1500   {
1501     \MathMLint { :silent } { \tl_use:c { \__spintrent_spusep_ #1 _tl } }
1502   }
1503   {
1504     \tl_use:c { \__spintrent_spusep_ #1 _tl }
1505   }
1506 }

```

(End of definition for `\__spintrent_spusep_set_parens:nn`.)

__spintrent_spusep_convert:n

The function `\__spintrent_spusep_convert:n` captures and stores the $\langle argument \rangle$ passed in ‘#1’ in the variable `\__spintrent_spusep_arg_tl`, validates the cases by converting them to the macros provided by `unicode-math`, and passes them to the function `\__spintrent_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an “error”.

```

1507 \cs_new_protected:Nn \__spintrent_spusep_convert:n
1508 {
1509   \tl_set:Nc \__spintrent_spusep_arg_tl { \tl_trim_spaces:n {#1} }
1510   \tl_if_empty:NT \__spintrent_spusep_arg_tl
1511   {
1512     \msg_error:nnn { spintrent } { spusep-invalid-arg } {#1}
1513   }
1514   \str_case:VnF \__spintrent_spusep_arg_tl
1515   {
1516     { ( }
1517     {
1518       \__spintrent_spusep_set_parens:nn { left } { \lparen }
1519     }
1520     { ) }
1521     {
1522       \__spintrent_spusep_set_parens:nn { right } { \rparen }
1523     }
1524     { [ ]
1525     {
1526       \__spintrent_spusep_set_parens:nn { left } { \lbrack }
1527     }
1528     { ] }
1529     {
1530       \__spintrent_spusep_set_parens:nn { right } { \rbrack }
1531     }
1532     { \{ }
1533     {
1534       \__spintrent_spusep_set_parens:nn { left } { \lbrace }
1535     }
1536     { \} }
1537     {
1538       \__spintrent_spusep_set_parens:nn { right } { \rbrace }
1539     }
1540   }
1541   {
1542     \msg_error:nnn { spintrent } { spusep-invalid-arg } {#1}
1543   }
1544 }

```

(End of definition for `\__spintrent_spusep_convert:n`.)

\spusep

The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in ‘#1’ and processes the $\langle argument \rangle$ passed in ‘#2’ by means of the function `\__spintrent_spusep_convert:n`.

```

1545 \NewDocumentCommand \spusep { O{} m }

```



```

1546 {
1547   \mode_if_math:TF
1548   {
1549     \group_begin:
1550     \keys_set:nn { spintent / spusep } {#1}
1551     \__spintent_spusep_convert:n {#2}
1552     \group_end:
1553   }
1554   { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
1555 }

```

(End of definition for `\spusep`. This function is documented on page 12.)

11.20.1 The command `\spread`

The command `\spread` provides a semantic read for MathCAT of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

`spread-empty-args` The error message `spread-empty-args` is raised when `\spread` receives an empty or blank $\langle argument \rangle$.

```

1556 \msg_new:nnnn { spintent } { spread-empty-args }
1557 { Empty~argument~in~\token_to_str:N \spread. }
1558 { \token_to_str:N \spread~requires~two~non-empty~arguments. }

```

(End of definition for `spread-empty-args`.)

`\__spintent_spread_intent_tl` The variable `\__spintent_spread_intent_tl` stores the sanitized semantic reading $\langle argument \rangle$ for symbol to be injected into the `\MathMLintent`.

```

1559 \tl_new:N \__spintent_spread_intent_tl

```

(End of definition for `\__spintent_spread_intent_tl`.)

`\__spintent_spread_exec:nn` The function `\__spintent_spread_exec:nn` implements the core logic for `\spread`. It verifies that neither $\langle argument \rangle$ is empty or blank. If valid, it evaluates the *reading* $\langle argument \rangle$ ‘#1’ using `\__spintent_sanitize_affix:Nn` and embeds it via `\MathMLintent` into the *math symbol* ‘#2’.

```

1560 \cs_new_protected:Nn \__spintent_spread_exec:nn
1561 {
1562   \bool_lazy_or:nnTF { \tl_if_blank_p:n { #1 } } { \tl_if_blank_p:n { #2 } }
1563   { \msg_error:nn { spintent } { spread-empty-args } }
1564   {
1565     \__spintent_sanitize_affix:Nn \__spintent_spread_intent_tl { #1 }
1566     \MathMLintent { \__spintent_spread_intent_tl } { #2 }
1567   }
1568 }

```

(End of definition for `\__spintent_spread_exec:nn`.)

`\spread` The command `\spread` assigns a custom semantic *reading* ‘#1’ to any mathematical *symbol* ‘#2’. It enforces math mode and passes execution to `\__spintent_spread_exec:nn`.

```

1569 \NewDocumentCommand \spread { m m }
1570 {
1571   \mode_if_math:TF
1572   {
1573     \group_begin:
1574     \__spintent_spread_exec:nn { #1 } { #2 }
1575     \group_end:
1576   }
1577   { \msg_error:nnn { spintent } { need-math-mode } { \spread } }
1578 }

```

(End of definition for `\spread`. This function is documented on page 11.)

11.20.2 The command `\spdsym`

`\__spintent_spdsym_read_sym_tl` Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spdsym`.

```

\__spintent_spdsym_symbol_tl
\__spintent_spdsym_prefix_tl
\__spintent_spdsym_suffix_tl
1579 \tl_new:N \__spintent_spdsym_read_sym_tl
1580 \tl_new:N \__spintent_spdsym_symbol_tl
1581 \tl_new:N \__spintent_spdsym_prefix_tl
1582 \tl_new:N \__spintent_spdsym_suffix_tl

```

(End of definition for `\__spintent_spdsym_read_sym_tl` and others.)

```

prefix We define the (keys) prefix, suffix, set-sym and read-sym for \spdsym.
suffix
set-sym
read-sym
unknown
1583 \keys_define:nn { spintent / spdsym }
1584 {
1585   prefix .code:n =
1586     { \__spintent_sanitizet_affix:Nn \__spintent_spdsym_prefix_tl {#1} },
1587   prefix .initial:n = ,
1588   prefix .value_required:n = true,
1589   suffix .code:n =
1590     { \__spintent_sanitizet_affix:Nn \__spintent_spdsym_suffix_tl {#1} },
1591   suffix .initial:n = ,
1592   suffix .value_required:n = true,
1593   set-sym .choices:nn =
1594     { old-style , two-dots , slash }
1595     {
1596       \int_case:nn { \l_keys_choice_int }
1597       {
1598         {1} { \tl_set:Nn \__spintent_spdsym_symbol_tl { \div } }
1599         {2} { \tl_set:Nn \__spintent_spdsym_symbol_tl { : } }
1600         {3} { \tl_set:Nn \__spintent_spdsym_symbol_tl { / } }
1601       }
1602     },
1603   set-sym .initial:n = two-dots,
1604   set-sym .value_required:n = true,
1605   set-sym / unknown .code:n =
1606     \msg_error:nneee { spintent } { unknown-choice }
1607     { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
1608   read-sym .choices:nn =
1609     { dividido , dividido-por , dividido-entre }
1610     {
1611       \int_case:nn { \l_keys_choice_int }
1612       {
1613         {1} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido } }
1614         {2} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido-por } }
1615         {3} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido-entre } }
1616       }
1617     },
1618   read-sym .initial:n = dividido,
1619   read-sym .value_required:n = true,
1620   read-sym / unknown .code:n =
1621     \msg_error:nneee { spintent } { unknown-choice }
1622     { read-sym } { dividido, ~dividido-por, ~dividido-entre }
1623     { \exp_not:n {#1} },
1624   unknown .code:n =
1625     {
1626       \str_set:Nn \__spintent_command_name_str { spdsym }
1627       \__spintent_unknown_keys_cmd:n {#1}
1628     },
1629 }

```

(End of definition for prefix and others.)

\spdsym Defines the **\spdsym** operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```

1630 \NewDocumentCommand \spdsym { 0{} }
1631 {
1632   \mode_if_math:TF
1633   {
1634     \group_begin:
1635     \keys_set:nn { spintent / spdsym } { #1 }
1636     \MathMLintent
1637     {
1638       \tl_if_empty:NF \__spintent_spdsym_prefix_tl
1639       { \__spintent_spdsym_prefix_tl - }
1640       \__spintent_spdsym_read_sym_tl
1641       \tl_if_empty:NF \__spintent_spdsym_suffix_tl
1642       { - \__spintent_spdsym_suffix_tl }
1643     }
1644     { \__spintent_spdsym_symbol_tl }
1645     \group_end:
1646   }
1647   { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
1648 }

```

(End of definition for `\spdsym`. This function is documented on page 12.)

11.20.3 The command `\spmsym`

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spmsym`.

```
\l__spintenc_spmsym_read_sym_tl
\l__spintenc_spmsym_symbol_tl
\l__spintenc_spmsym_prefix_tl
\l__spintenc_spmsym_suffix_tl
1649 \tl_new:N \l__spintenc_spmsym_read_sym_tl
1650 \tl_new:N \l__spintenc_spmsym_symbol_tl
1651 \tl_new:N \l__spintenc_spmsym_prefix_tl
1652 \tl_new:N \l__spintenc_spmsym_suffix_tl
```

(End of definition for `\l__spintenc_spmsym_read_sym_tl` and others.)

```
prefix We define the (keys) prefix, suffix, set-sym, read-sym and not-print for \spmsym.
suffix
set-sym
read-sym
not-print
unknown
1653 \keys_define:nn { spintenc / spmsym }
1654 {
1655   prefix .code:n =
1656   { \l__spintenc_sanitizate_affix:Nn \l__spintenc_spmsym_prefix_tl {#1} },
1657   prefix .initial:n = ,
1658   prefix .value_required:n = true,
1659   suffix .code:n =
1660   { \l__spintenc_sanitizate_affix:Nn \l__spintenc_spmsym_suffix_tl {#1} },
1661   suffix .initial:n = ,
1662   suffix .value_required:n = true,
1663   set-sym .choices:nn =
1664   { cross , dot , asterisk }
1665   {
1666     \int_case:nn { \l_keys_choice_int }
1667     {
1668       {1} { \tl_set:Nn \l__spintenc_spmsym_symbol_tl { \times } }
1669       {2} { \tl_set:Nn \l__spintenc_spmsym_symbol_tl { \cdot } }
1670       {3} { \tl_set:Nn \l__spintenc_spmsym_symbol_tl { \ast } }
1671     }
1672   },
1673   set-sym .initial:n = dot,
1674   set-sym .value_required:n = true,
1675   set-sym / unknown .code:n =
1676   \msg_error:nneee { spintenc } { unknown-choice }
1677   { set-sym } { cross, ~dot, ~asterisk } { \exp_not:n {#1} },
1678   read-sym .choices:nn =
1679   { por , multiplicado-por , veces }
1680   {
1681     \int_case:nn { \l_keys_choice_int }
1682     {
1683       {1} { \tl_set:Nn \l__spintenc_spmsym_read_sym_tl { por } }
1684       {2} { \tl_set:Nn \l__spintenc_spmsym_read_sym_tl { multiplicado-por } }
1685       {3} { \tl_set:Nn \l__spintenc_spmsym_read_sym_tl { veces } }
1686     }
1687   },
1688   read-sym .initial:n = por,
1689   read-sym .value_required:n = true,
1690   read-sym / unknown .code:n =
1691   \msg_error:nneee { spintenc } { unknown-choice }
1692   { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} },
1693   not-print .bool_set:N = \l__spintenc_spmsym_not_print_bool,
1694   not-print .initial:n = false,
1695   not-print .value_required:n = true,
1696   unknown .code:n =
1697   {
1698     \str_set:Nn \l__spintenc_command_name_str { spmsym }
1699     \l__spintenc_unknown_keys_cmd:n {#1}
1700   },
1701 }
```

(End of definition for `prefix` and others.)

`\spmsym` Defines the `\spmsym` operator. Similar to `\spdsym`, it outputs the correct mathematical glyph based on the local *(keys)*, safely attaching prefixes and suffixes if present.

```
1702 \NewDocumentCommand \spmsym { O{} }
1703 {
1704   \mode_if_math:TF
1705   {
```

```

1706 \group_begin:
1707 \keys_set:nn { spintent / spmsym } { #1 }
1708 \MathMLintent
1709 {
1710 \tl_if_empty:NF \l__spintent_spmsym_prefix_tl
1711 { \l__spintent_spmsym_prefix_tl - }
1712 \l__spintent_spmsym_read_sym_tl
1713 \tl_if_empty:NF \l__spintent_spmsym_suffix_tl
1714 { - \l__spintent_spmsym_suffix_tl }
1715 }
1716 {
1717 \bool_if:NTF \l__spintent_spmsym_not_print_bool
1718 {
1719 \invisibletimes
1720 }
1721 { \l__spintent_spmsym_symbol_tl }
1722 }
1723 \group_end:
1724 }
1725 { \msg_error:nnn { spintent } { need-math-mode } { \spmsym } }
1726 }

```

(End of definition for `\spmsym`. This function is documented on page 12.)

11.21 The command `\spdiv`

The user-level command `\spdiv` formats division representations. It takes two mandatory arguments: the $\langle dividend \rangle$ and the $\langle divisor \rangle$. Both $\langle arguments \rangle$ are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the $\langle dividend \rangle$ nor the $\langle divisor \rangle$ is *blank*.

11.21.1 Definition of keys for `\spdiv`

We define the $\langle keys \rangle$ `wrap-parens` and `read-parens` for `\spdiv` command. Number-related $\langle keys \rangle$ `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator $\langle keys \rangle$ `set-sym` and `read-sym` are forwarded to the `\spdsym` command using `.meta:nn`.

```

1727 \keys_define:nn { spintent / spdiv }
1728 {
1729 cifras-sep .meta:nn = { spintent / spnum } { cifras-sep = {#1} },
1730 read-sign .meta:nn = { spintent / spnum } { read-sign = {#1} },
1731 read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1732 notation-sym .meta:nn = { spintent / spnum } { notation-sym = {#1} },
1733 set-sym .meta:nn = { spintent / spdsym } { set-sym = {#1} },
1734 read-sym .meta:nn = { spintent / spdsym } { read-sym = {#1} },
1735 wrap-parens .bool_set:N = \l__spintent_spdiv_wrap_parens_bool,
1736 wrap-parens .initial:n = false,
1737 wrap-parens .value_required:n = true,
1738 read-parens .bool_set:N = \l__spintent_spdiv_read_parens_bool,
1739 read-parens .initial:n = true,
1740 read-parens .value_required:n = true,
1741 unknown .code:n =
1742 {
1743 \str_set:Nn \l__spintent_command_name_str { spdiv }
1744 \l__spintent_unknown_keys_cmd:n {#1}
1745 },
1746 }

```

(End of definition for `cifras-sep` and others.)

11.21.2 Definition of error messages

`spdiv-empty-argument` Raised when either the $\langle dividend \rangle$ or the $\langle divisor \rangle$ is missing or *blank*.

```

1747 \msg_new:nnnn { spintent } { spdiv-empty-argument }
1748 { The~arguments~for~\token_to_str:N \spdiv~cannot~be~empty. }
1749 {
1750 You~must~provide~both~a~dividend~and~a~divisor.\\
1751 }

```

(End of definition for `spdiv-empty-argument`.)

11.21.3 Internal functions for `\spdiv`

`\__spinttent_spdiv_render_op:nn`

The function `\__spinttent_spdiv_render_op:nn` parses both $\{\langle arguments \rangle\}$ through the `\spnum` command and embeds the semantic operator from `\spdsym`.

```
1752 \cs_new_protected:Nn \__spinttent_spdiv_render_op:nn
1753 {
1754   \spnum {#1}
1755   \MathMLintent { \l__spinttent_spdsym_read_sym_tl }
1756   {
1757     \l__spinttent_spdsym_symbol_tl
1758   }
1759   \spnum {#2}
1760 }
```

(End of definition for `\__spinttent_spdiv_render_op:nn`.)

`\__spinttent_spdiv_render_parens:nn`

The function `\__spinttent_spdiv_render_parens:nn` handles the structural parentheses around the division.

```
1761 \cs_new_protected:Nn \__spinttent_spdiv_render_parens:nn
1762 {
1763   \bool_if:NTF \l__spinttent_spdiv_read_parens_bool
1764   {
1765     ( \__spinttent_spdiv_render_op:nn {#1} {#2} )
1766   }
1767   {
1768     \MathMLintent { :silent } { ( }
1769     \__spinttent_spdiv_render_op:nn {#1} {#2}
1770     \MathMLintent { :silent } { ) }
1771   }
1772 }
```

(End of definition for `\__spinttent_spdiv_render_parens:nn`.)

`\__spinttent_spdiv_process:nnn`

The function `\__spinttent_spdiv_process:nnn` sets $\langle keys \rangle$, validates $\{\langle arguments \rangle\}$, and decides on parenthesis wrapping.

```
1773 \cs_new_protected:Nn \__spinttent_spdiv_process:nnn
1774 {
1775   \keys_set:nn { spinttent / spdiv } { #1 }
1776   \tl_if_blank:NTF { #2 }
1777   { \msg_error:nnn { spinttent } { spdiv-empty-argument } }
1778   {
1779     \tl_if_blank:NTF { #3 }
1780     { \msg_error:nnn { spinttent } { spdiv-empty-argument } }
1781     {
1782       \bool_if:NTF \l__spinttent_spdiv_wrap_parens_bool
1783       { \__spinttent_spdiv_render_parens:nn {#2} {#3} }
1784       { \__spinttent_spdiv_render_op:nn {#2} {#3} }
1785     }
1786   }
1787 }
```

(End of definition for `\__spinttent_spdiv_process:nnn`.)

`\spdiv`

Defines the user command `\spdiv`. It strictly relies on *math mode*.

```
1788 \NewDocumentCommand \spdiv { O{} m m }
1789 {
1790   \mode_if_math:TF
1791   {
1792     \group_begin:
1793     \__spinttent_spdiv_process:nnn { #1 } { #2 } { #3 }
1794     \group_end:
1795   }
1796   { \msg_error:nnn { spinttent } { need-math-mode } { \spdiv } }
1797 }
```

(End of definition for `\spdiv`. This function is documented on page 13.)

11.22 The command `\spmul`

The user-level command `\spmul` formats multiplication representations. It takes two mandatory arguments: the $\{\langle first factor \rangle\}$ and the $\{\langle second factor \rangle\}$. Both $\{\langle arguments \rangle\}$ are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the $\{\langle first factor \rangle\}$ nor the $\{\langle second factor \rangle\}$ is *blank*.

11.22.1 Definition of keys for \spmul

We define the $\langle keys \rangle$ `wrap-parens` and `read-parens` for `\spmul` command. Number-related $\langle keys \rangle$ `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator $\langle keys \rangle$ `set-sym` and `read-sym` are forwarded to the `\spmsym` command using `.meta:nn`.

```

1798 \keys_define:nn { spintenc / spmul }
1799 {
1800   cifras-sep      .meta:nn = { spintenc / spnum } { cifras-sep      = {#1} },
1801   read-sign       .meta:nn = { spintenc / spnum } { read-sign       = {#1} },
1802   read-period-sep .meta:nn = { spintenc / spnum } { read-period-sep = {#1} },
1803   notation-sym    .meta:nn = { spintenc / spnum } { notation-sym    = {#1} },
1804   set-sym         .meta:nn = { spintenc / spmsym } { set-sym         = {#1} },
1805   read-sym        .meta:nn = { spintenc / spmsym } { read-sym        = {#1} },
1806   wrap-parens     .bool_set:N = \l__spintenc_spmul_wrap_parens_bool,
1807   wrap-parens     .initial:n  = false,
1808   wrap-parens     .value_required:n = true,
1809   read-parens     .bool_set:N = \l__spintenc_spmul_read_parens_bool,
1810   read-parens     .initial:n  = true,
1811   read-parens     .value_required:n = true,
1812   unknown         .code:n =
1813   {
1814     \str_set:Nn \l__spintenc_command_name_str { spmul }
1815     \__spintenc_unknown_keys_cmd:n {#1}
1816   },
1817 }

```

(End of definition for `cifras-sep` and others.)

11.22.2 Definition of error messages

`spmul-empty-argument` Raised when either the $\langle first factor \rangle$ or the $\langle second factor \rangle$ is missing or *blank*.

```

1818 \msg_new:nnnn { spintenc } { spmul-empty-argument }
1819 { The~arguments~for~\token_to_str:N \spmul~cannot~be~empty. }
1820 {
1821   You~must~provide~both~first~factor~and~second~factor.\\
1822 }

```

(End of definition for `spmul-empty-argument`.)

11.22.3 Internal functions for \spmul

`\__spintenc_spmul_render_op:nn` The function `\__spintenc_spmul_render_op:nn` parses both $\langle arguments \rangle$ through the `\spnum` command and embeds the semantic operator from `\spmsym`.

```

1823 \cs_new_protected:Nn \__spintenc_spmul_render_op:nn
1824 {
1825   \spnum {#1}
1826   \MathMLintenc { \l__spintenc_spmsym_read_sym_tl }
1827   {
1828     \l__spintenc_spmsym_symbol_tl
1829   }
1830   \spnum {#2}
1831 }

```

(End of definition for `\__spintenc_spmul_render_op:nn`.)

`\__spintenc_spmul_render_parens:nn` The function `\__spintenc_spmul_render_parens:nn` handles the structural parentheses around the multiplication.

```

1832 \cs_new_protected:Nn \__spintenc_spmul_render_parens:nn
1833 {
1834   \bool_if:NTF \l__spintenc_spmul_read_parens_bool
1835   {
1836     ( \__spintenc_spmul_render_op:nn {#1} {#2} )
1837   }
1838   {
1839     \MathMLintenc { :silent } { ( }
1840     \__spintenc_spmul_render_op:nn {#1} {#2}
1841     \MathMLintenc { :silent } { ) }
1842   }
1843 }

```

(End of definition for `\__spintenc_spmul_render_parens:nn`.)

`\__spintent_spmul_process:nnn`

The function `\__spintent_spmul_process:nnn` sets $\langle keys \rangle$, validates $\{ \langle arguments \rangle \}$, and decides on parenthesis wrapping.

```

1844 \cs_new_protected:Nn \__spintent_spmul_process:nnn
1845 {
1846   \keys_set:nn { spintent / spmul } { #1 }
1847   \tl_if_blank:nTF { #2 }
1848   { \msg_error:nnn { spintent } { spmul-empty-argument } }
1849   {
1850     \tl_if_blank:nTF { #3 }
1851     { \msg_error:nnn { spintent } { spmul-empty-argument } }
1852     {
1853       \bool_if:NTF \__spintent_spmul_wrap_parens_bool
1854       { \__spintent_spmul_render_parens:nn {#2} {#3} }
1855       { \__spintent_spmul_render_op:nn {#2} {#3} }
1856     }
1857   }
1858 }

```

(End of definition for `\__spintent_spmul_process:nnn`.)

`\spmul` Defines the user command `\spmul`. It strictly relies on *math mode*.

```

1859 \NewDocumentCommand \spmul { 0{} m m }
1860 {
1861   \mode_if_math:TF
1862   {
1863     \group_begin:
1864     \__spintent_spmul_process:nnn { #1 } { #2 } { #3 }
1865     \group_end:
1866   }
1867   { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
1868 }

```

(End of definition for `\spmul`. This function is documented on page 13.)

11.23 The commands `\spgcd` and `\spmcn`

The user-level commands `\spgcd` and `\spmcn` provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid *MathCAT* intent.

11.23.1 Variables for `\spgcd` and `\spmcn`

`\__spintent_spmcm_spmcd_luaset_error_str`

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

```

\__spintent_mc_args_seq
\__spintent_spmcd_print_result_bool
\__spintent_spmcm_print_result_bool
\__spintent_spmcd_read_terse_bool
\__spintent_spmcm_read_terse_bool
\__spintent_spmcd_print_upper_bool
\__spintent_spmcm_print_upper_bool
\__spintent_spmcd_prefix_tl
\__spintent_spmcm_prefix_tl
\__spintent_spmcd_suffix_tl
\__spintent_spmcm_suffix_tl
1869 \str_new:N \__spintent_spmcm_spmcd_luaset_error_str
1870 \seq_new:N \__spintent_mc_args_seq
1871 \clist_map_inline:nn { mcd , mcm }
1872 {
1873   \tl_new:c { \__spintent_sp #1 _luaset_ #1 _value_tl }
1874   \bool_new:c { \__spintent_sp #1 _print_result_bool }
1875   \bool_new:c { \__spintent_sp #1 _read_terse_bool }
1876   \bool_new:c { \__spintent_sp #1 _print_upper_bool }
1877   \tl_new:c { \__spintent_sp #1 _prefix_tl }
1878   \tl_new:c { \__spintent_sp #1 _suffix_tl }
1879 }

```

(End of definition for `\__spintent_spmcm_spmcd_luaset_error_str` and others.)

11.23.2 Definition of error messages

`mc-invalid-arguments`

Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```

1880 \msg_new:nnnn { spintent } { mc-invalid-arguments }
1881 {
1882   Invalid~arguments~provided~to~#1.
1883 }
1884 {
1885   The~arguments~must~be~a~comma-separated~list. \\\
1886 }

```

(End of definition for `mc-invalid-arguments`.)

11.23.3 Keys for \spmc and \spmcm

We define the shared *(keys)* result, prefix, read-cmd, upper-case, sample-arg and silent-cmd.

```

1887 \clist_map_inline:nn { mcd , mcm }
1888 {
1889   \keys_define:nn { spintent / sp #1 }
1890   {
1891     result      .bool_set:c = { \__spintent_sp #1 _print_result_bool },
1892     result      .initial:n = false,
1893     result      .value_required:n = true,
1894     prefix      .code:n =
1895       { \__spintent_sanitizize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
1896     prefix      .initial:n = ,
1897     prefix      .value_required:n = true,
1898     read-cmd     .choices:nn =
1899       { terse , clear }
1900       {
1901         \int_case:nn { \l_keys_choice_int }
1902         {
1903           {1} { \bool_set_true:c { \__spintent_sp #1 _read_terse_bool } }
1904           {2} { \bool_set_false:c { \__spintent_sp #1 _read_terse_bool } }
1905         }
1906       },
1907     read-cmd     .initial:n = clear,
1908     read-cmd     .value_required:n = true,
1909     read-cmd / unknown .code:n =
1910       \msg_error:nneee { spintent } { unknown-choice }
1911       { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
1912     upper-case   .bool_set:c = { \__spintent_sp #1 _print_upper_bool },
1913     upper-case   .initial:n = false,
1914     upper-case   .value_required:n = true,
1915     sample-arg   .bool_set:c = { \__spintent_sp #1 _sample_arg_bool },
1916     sample-arg   .initial:n = false,
1917     sample-arg   .value_required:n = true,
1918     silent-cmd   .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
1919     silent-cmd   .initial:n = false,
1920     silent-cmd   .value_required:n = true,
1921     unknown      .code:n =
1922       {
1923         \str_set:Nn \__spintent_command_name_str { sp #1 }
1924         \__spintent_unknown_keys_cmd:n {##1}
1925       },
1926   }
1927 }

```

(End of definition for result and others.)

11.23.4 Internal functions for \spmc and \spmcm

The function `\__spintent_mc_process_args:nn` parses the operation ‘#1’ and its arguments ‘#2’, injecting MathML intents.

```

1928 \cs_new_protected:Nn \__spintent_mc_process_args:nn
1929 {
1930   \seq_set_from_clist:Nn \__spintent_mc_args_seq {#2}
1931   \bool_if:cTF { \__spintent_sp #1 _silent_cmd_bool }
1932   {
1933     \seq_set_map:Nnn \__spintent_mc_args_seq \__spintent_mc_args_seq
1934     { \MathMLintent { :silent } { ##1 } }
1935     \MathMLintent { entre:silent } { ( }
1936     \seq_use:Nnnn \__spintent_mc_args_seq
1937     { \MathMLintent { _y:silent } { , } }
1938     { \MathMLintent { :silent } { , } }
1939     { \MathMLintent { _y:silent } { , } }
1940   }
1941   {
1942     \MathMLintent { entre } { ( }
1943     \seq_use:Nnnn \__spintent_mc_args_seq
1944     { \MathMLintent { _y } { , } }
1945     { \MathMLintent { :silent } { , } }
1946     { \MathMLintent { _y } { , } }
1947   }
1948   \MathMLintent { :silent } { ) }
1949 }

```

(End of definition for `\__spintent_mc_process_args:nn`.)

`\__spintent_spmc_process:nnnn`

The internal function `\__spintent_spmc_process:nnnn` serves as a shared central handler for both the `\spmc` and `\spmc` commands. It takes the command identifier ‘#1’, its full reading string ‘#2’, the local $\langle keys \rangle$ ‘#3’, and the optional values ‘#4’. It dynamically resolves variables using the identifier to build the MathML `intent` string (including any prefixes or suffixes) and prints the math operator. If values are provided in ‘#4’, it calculates the result using the Lua backend. If an error occurs, it throws the `mc-invalid-arguments` error; otherwise, it processes the arguments and conditionally prints the calculated result.

```

1950 \cs_new_protected:Nn \__spintent_spmc_process:nnnn
1951 {
1952   \keys_set:nn { spintent / sp #1 } { #3 }
1953   \MathMLintent
1954   {
1955     \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
1956     { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
1957     \bool_if:cTF { l__spintent_sp #1 _read_terse_bool }
1958     { #1 } { #2 }
1959     \bool_if:cT { l__spintent_sp #1 _silent_cmd_bool }
1960     {
1961       \tl_if_no_value:nTF {#4}
1962       { :silent }
1963       { \bool_if:cT { l__spintent_sp #1 _sample_arg_bool } { :silent } }
1964     }
1965   }
1966   {
1967     \operatorname
1968     {
1969       \bool_if:cTF { l__spintent_sp #1 _print_upper_bool }
1970       { \text_uppercase:n {#1} } { #1 }
1971     }
1972   }
1973   \tl_if_no_value:nF { #4 }
1974   {
1975     \bool_if:cTF { l__spintent_sp #1 _sample_arg_bool }
1976     {
1977       \__spintent_mc_process_args:nn {#1} {#4}
1978     }
1979     {
1980       \use:c { __spintent_luafun_calculate_ #1 :n } { #4 }
1981       \str_if_eq:VnTF \l__spintent_spmcm_spmcd_luaset_error_str { true }
1982       {
1983         \msg_error:nne { spintent } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
1984       }
1985       {
1986         \__spintent_mc_process_args:nn {#1} {#4}
1987         \bool_if:cT { l__spintent_sp #1 _print_result_bool }
1988         {
1989           \MathMLintent { es-igual-a } { = }
1990           \tl_use:c { l__spintent_sp #1 _luaset_ #1 _value_tl }
1991         }
1992       }
1993     }
1994   }
1995 }

```

(End of definition for `\__spintent_spmc_process:nnnn`.)

`\spmc`
`\spmc`

The commands `\spmc` and `\spmc` act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```

1996 \NewDocumentCommand \spmc { 0{ } d() }
1997 {
1998   \mode_if_math:TF
1999   {
2000     \group_begin:
2001     \__spintent_spmc_process:nnnn { mcd } { máximo-común-divisor } {#1} {#2}
2002     \group_end:
2003   }
2004   { \msg_error:nnn { spintent } { need-math-mode } { \spmc } }
2005 }

```

```

2006 \NewDocumentCommand \spmc { 0{ } d{ } }
2007 {
2008   \mode_if_math:TF
2009   {
2010     \group_begin:
2011     \__spintent_spmc_process:nnnn { mcm } { mínimo-común-múltiplo } {#1} {#2}
2012     \group_end:
2013   }
2014   { \msg_error:nnn { spintent } { need-math-mode } { \spmc } }
2015 }

```

(End of definition for `\spmc` and `\spmc`. These functions are documented on page 15.)

11.24 The commands `\spnM` and `\spnD`

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (“*múltiplos*” in Spanish) and the Divisors of a number (“*divisores*” in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.24.1 Variables for `\spnM` and `\spnD`

Internal variables generated dynamically for multiples and divisors.

```

\__spintent_md_out_seq
\__spintent_spnM_luaset_result_tl
\__spintent_spnD_luaset_result_tl
\__spintent_spnM_luaset_is_truncated_str
\__spintent_spnD_luaset_is_truncated_str
\__spintent_spnM_print_result_bool
\__spintent_spnD_print_result_bool
\__spintent_spnM_silent_cmd_bool
\__spintent_spnD_silent_cmd_bool
\__spintent_spnM_prefix_tl
\__spintent_spnD_prefix_tl
\__spintent_spnM_print_result_num_tl
\__spintent_spnD_print_result_num_tl

```

```

2016 \seq_new:N \__spintent_md_out_seq
2017 \clist_map_inline:nn { nM , nD }
2018 {
2019   \tl_new:c { \__spintent_sp #1 _luaset_result_tl }
2020   \str_new:c { \__spintent_sp #1 _luaset_is_truncated_str }
2021   \bool_new:c { \__spintent_sp #1 _print_result_bool }
2022   \bool_new:c { \__spintent_sp #1 _silent_cmd_bool }
2023   \tl_new:c { \__spintent_sp #1 _prefix_tl }
2024   \tl_new:c { \__spintent_sp #1 _print_result_num_tl }
2025 }

```

(End of definition for `\__spintent_md_out_seq` and others.)

11.24.2 Keys for `\spnM` and `\spnD`

We define the shared `<keys>` `result`, `silent-cmd`, `result-num` and `prefix`.

```

result
silent-cmd
result-num
prefix
sample-arg

```

```

2026 \clist_map_inline:nn { nM , nD }
2027 {
2028   \keys_define:nn { spintent / sp #1 }
2029   {
2030     result .bool_set:c = { \__spintent_sp #1 _print_result_bool },
2031     result .initial:n = false,
2032     result .value_required:n = true,
2033     silent-cmd .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
2034     silent-cmd .initial:n = false,
2035     silent-cmd .value_required:n = true,
2036     result-num .tl_set:c = { \__spintent_sp #1 _print_result_num_tl },
2037     result-num .initial:n = ,
2038     prefix .code:n =
2039     { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
2040     prefix .initial:n = ,
2041     prefix .value_required:n = true,
2042     sample-arg .bool_set:c = { \__spintent_sp #1 _sample_arg_bool },
2043     sample-arg .initial:n = false,
2044     sample-arg .value_required:n = true,
2045     unknown .code:n =
2046     {
2047       \str_set:Nn \__spintent_command_name_str { sp #1 }
2048       \__spintent_unknown_keys_cmd:n {##1}
2049     },
2050   }
2051 }

```

(End of definition for `result` and others.)

11.24.3 Definition of error messages for `\spnM` and `\spnD`

`md-invalid-argument` Exception triggered when the argument for multiples or divisors is invalid.

```

2052 \msg_new:nnnn { spintent } { md-invalid-argument }
2053 { Invalid~argument~provided~to~#1. }
2054 {
2055   The~argument~must~be~a~single~positive~integer. \
2056 }

```

(End of definition for `md-invalid-argument`.)

11.24.4 Internal functions for \spnM and \spnD

__spintenc_spnM_spnD_process:nnnn

The internal function `\__spintenc_spnM_spnD_process:nnnn` acts as the primary entry point for the multiples and divisors commands. It takes the command identifier ‘#1’, its printed mathematical symbol ‘#2’, its base reading string ‘#3’, the local key-value options ‘#4’, and the optional argument ‘#5’. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to `\__spintenc_spnM_and_D_exec:nnnn`.

```
2057 \cs_new_protected:Nn \__spintenc_spnM_spnD_process:nnnn
2058 {
2059   \keys_set:nn { spintenc / sp #1 } { #4 }
2060   \__spintenc_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
2061 }
```

(End of definition for __spintenc_spnM_spnD_process:nnnn.)

__spintenc_spnM_spnD_intent:nn

The internal function `\__spintenc_spnM_spnD_intent:nn` resolves the main reading intent string. It takes the command identifier ‘#1’ (such as “nM” or “nD”) and the base reading string ‘#2’. It automatically prepends the localized prefix if defined and handles specific string normalizations, such as ensuring the correct accentuation for “múltiplos”.

```
2062 \cs_new:Nn \__spintenc_spnM_spnD_intent:nn
2063 {
2064   \tl_if_empty:cF { l__spintenc_sp #1 _prefix_tl }
2065   { \tl_use:c { l__spintenc_sp #1 _prefix_tl } - }
2066   \str_case:nnF { #2 }
2067   { { multiplos } { múltiplos } }
2068   { #2 }
2069 }
```

(End of definition for __spintenc_spnM_spnD_intent:nn.)

__spintenc_spnM_spnD_silent:nn

The internal function `\__spintenc_spnM_and_D_silent:nn` prints the atomized notation structure in full silent mode. It takes the mathematical symbol ‘#1’ (such as “D” or “M”) and the variable or explicit argument ‘#2’. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules like “de”.

```
2070 \cs_new_protected:Nn \__spintenc_spnM_spnD_silent:nn
2071 {
2072   \MathMLintent { :silent } { \operatorname { #1 } }
2073   \MathMLintent { :silent } { ( }
2074   \MathMLintent { :silent } { \__spintenc_invisible_sep: }
2075   \MathMLintent { :silent } { #2 }
2076   \MathMLintent { :silent } { ) }
2077 }
```

(End of definition for __spintenc_spnM_spnD_silent:nn.)

__spintenc_spnM_spnD_result:n

The internal function `\__spintenc_spnM_and_D_result:n` handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier ‘#1’. It invokes the corresponding Lua backend calculation, maps the resulting clist into a sequence, and formats the final printed output utilizing custom accessibility intents for both separators and conjunctions. It also safely manages truncation indicators.

```
2078 \cs_new_protected:Nn \__spintenc_spnM_spnD_result:n
2079 {
2080   \use:c { __spintenc_luafun_calcular_ #1 :nn }
2081   { \l__spintenc_luaset_part_int_tl }
2082   { \tl_use:c { l__spintenc_sp #1 _print_result_num_tl } }
2083   \MathMLintent { son } { = \{ }
2084   \seq_set_from_clist:Ne
2085   \l__spintenc_md_out_seq { \tl_use:c { l__spintenc_sp #1 _luaset_result_tl } }
2086   \seq_use:Nnnn \l__spintenc_md_out_seq
2087   { \MathMLintent { _y } { , } }
2088   { \MathMLintent { :silent } { , } }
2089   { \MathMLintent { _y } { , } }
2090   \str_if_eq:vnT
2091   { l__spintenc_sp #1 _luaset_is_truncated_str } { true }
2092   {
2093     \MathMLintent { :silent } { , } ~ \dots
2094   }
2095   \MathMLintent { :silent } { \} }
2096 }
```

(End of definition for __spintenc_spnM_spnD_result:n.)

_spintent_spnM_spnD_exec:nnnn

The function `\\_spintent_spnM_and_D_exec:nnnn` implements the multiples and divisors commands. It takes the $\langle \text{argument} \rangle$ ‘#1’ (the command name), the $\langle \text{argument} \rangle$ ‘#2’ (the mathematical symbol), the base reading $\langle \text{argument} \rangle$ ‘#3’, and the optional $\langle \text{argument} \rangle$ ‘#4’. If no $\langle \text{argument} \rangle$ is provided, it evaluates the key `silent-cmd` using the default token ‘n’. If an $\langle \text{argument} \rangle$ is given, it evaluates the key `sample-arg`. If true, it processes the $\langle \text{argument} \rangle$ directly and evaluates the key `silent-cmd`. If false, it bypasses the *silent mode*, validates the $\langle \text{argument} \rangle$ as a natural number, and evaluates the key `result` optionally calculate and append the resulting set.

```

2097 \cs_new_protected:Nn \__spintent_spnM_spnD_exec:nnnn
2098 {
2099   \tl_if_novalue:nTF { #4 }
2100   {
2101     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
2102     { \__spintent_spnM_spnD_silent:nn { #2 } { n } }
2103     {
2104       \MathMLintend
2105       { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2106       { \operatorname { #2 } ( n ) }
2107     }
2108   }
2109   {
2110     \bool_if:cTF { l__spintent_sp #1 _sample_arg_bool }
2111     {
2112       \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
2113       { \__spintent_spnM_spnD_silent:nn { #2 } { #4 } }
2114       {
2115         \MathMLintend
2116         { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2117         { \operatorname { #2 } } ( #4 )
2118       }
2119     }
2120     {
2121       \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2122       \str_if_eq:VnTF \__spintent_luaset_has_natural_str { true }
2123       {
2124         \MathMLintend
2125         { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2126         { \operatorname { #2 } } ( #4 )
2127         \bool_if:cT { l__spintent_sp #1 _print_result_bool }
2128         { \__spintent_spnM_spnD_result:n { #1 } }
2129       }
2130       {
2131         \msg_error:nne { spintent } { md-invalid-argument } { \exp_not:c { sp #1 } }
2132       }
2133     }
2134   }
2135 }

```

(End of definition for `\\_spintent_spnM_spnD_exec:nnnn`.)

`\spnM` The commands `\spnM` and `\spnD` act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function `\\_spintent_spnM_spnD_process:nnnnn`, passing along the optional $\langle \text{keys} \rangle$ ‘#1’ and the numeric $\langle \text{argument} \rangle$ ‘#2’.

```

2136 \NewDocumentCommand \spnM { O{} d() }
2137 {
2138   \mode_if_math:TF
2139   {
2140     \group_begin:
2141     \__spintent_spnM_spnD_process:nnnnn { nM } { M } { multiplos } { #1 } { #2 }
2142     \group_end:
2143   }
2144   { \msg_error:nnn { spintent } { need-math-mode } { \spnM } }
2145 }
2146 \NewDocumentCommand \spnD { O{} d() }
2147 {
2148   \mode_if_math:TF
2149   {
2150     \group_begin:
2151     \__spintent_spnM_spnD_process:nnnnn { nD } { D } { divisores } { #1 } { #2 }
2152     \group_end:

```

```

2153     }
2154     { \msg_error:nnn { spintent } { need-math-mode } { \spnD } }
2155 }

```

(End of definition for `\spnM` and `\spnD`. These functions are documented on page 14.)

11.25 The command `\spang`

The command `\spang` is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function `\__spintent_luafun_spang_parse:n`. This separates the degrees, minutes, and seconds to construct a MathML `intent` tree with the correct postfixes for NVDA/MathCAT.

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```

2156 \str_new:N \__spintent_spang_luaset_grado_str
2157 \str_new:N \__spintent_spang_luaset_minuto_str
2158 \str_new:N \__spintent_spang_luaset_segundo_str
2159 \str_new:N \__spintent_spang_luaset_error_str

```

(End of definition for `\__spintent_spang_luaset_grado_str` and others.)

11.25.1 Definition of error messages for `\spang`

`invalid-angle-format`

The error message `invalid-angle-format` defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```

2160 \msg_new:nnnn { spintent } { invalid-angle-format }
2161 { Invalid~angle~format~in~#1. }
2162 {
2163   The~argument~'#2'~could~not~be~parsed. \
2164 }

```

(End of definition for `invalid-angle-format`.)

11.25.2 Internal functions for `\spang`

`\__spintent_spang_process:n`

The internal function `\__spintent_spang_process:n` parses the angle input using `\__spintent_luafun_spang_parse:n` after resetting the error flag in `\__spintent_spang_luaset_error_str`. If an invalid format is detected, it issues the `invalid-angle-format` error. Otherwise, it checks the extracted string variables `\__spintent_spang_luaset_grado_str`, `\__spintent_spang_luaset_minuto_str`, and `\__spintent_spang_luaset_segundo_str`. For each non-empty variable, it constructs the corresponding MathML `intent` structure with the correct measurement symbols and postfixes for NVDA/MathCAT.

```

2165 \cs_new_protected:Nn \__spintent_spang_process:n
2166 {
2167   \str_set:Nn \__spintent_spang_luaset_error_str { false }
2168   \__spintent_luafun_spang_parse:n { #1 }
2169   \str_if_eq:VnTF \__spintent_spang_luaset_error_str { true }
2170   {
2171     \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
2172   }
2173   {
2174     \str_if_empty:NF \__spintent_spang_luaset_grado_str
2175     {
2176       \spunit{ \__spintent_spang_luaset_grado_str °}
2177     }
2178     \str_if_empty:NF \__spintent_spang_luaset_minuto_str
2179     {
2180       \MathMLintent { minutos : postfix($m) }
2181       {
2182         \MathMLarg { m }
2183         {
2184           \spunit{ \__spintent_spang_luaset_minuto_str ' }
2185         }
2186       }
2187     }
2188     \str_if_empty:NF \__spintent_spang_luaset_segundo_str
2189     {
2190       \MathMLintent { segundos : postfix($s) }
2191       {
2192         \MathMLarg { s }
2193       }

```

```

2194         \spunit { \l__spintent_spang_luaset_segundo_str " }
2195     }
2196 }
2197 }
2198 }
2199 }

```

(End of definition for `\l__spintent_spang_process:n`.)

\spang The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\l__spintent_spang_process:n` to process the $\langle argument \rangle$.

```

2200 \NewDocumentCommand \spang { m }
2201 {
2202     \mode_if_math:TF
2203     {
2204         \group_begin:
2205         \l__spintent_spang_process:n { #1 }
2206         \group_end:
2207     }
2208     { \msg_error:nnn { spintent } { need-math-mode } { \spang } }
2209 }

```

(End of definition for `\spang`. This function is documented on page 17.)

11.26 The command `\spnZ`

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by MathCAT.

This command verifies that the $\langle argument \rangle$ is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

11.26.1 Definition of keys for `\spnZ`

`cifras-sep` Now we define the $\langle keys \rangle$ `cifras-sep` and `read-sign`. These are `.meta:nn` $\langle keys \rangle$ passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by MathCAT.

```

2210 \keys_define:nn { spintent / spnZ }
2211 {
2212     cifras-sep .meta:nn = { spintent / spnum } { cifras-sep = {#1} },
2213     read-sign .meta:nn = { spintent / spnum } { read-sign = {#1} },
2214     wrap-parens .bool_set:N = \l__spintent_spnz_wrap_parens_bool,
2215     wrap-parens .initial:n = false,
2216     wrap-parens .value_required:n = true,
2217     read-parens .bool_set:N = \l__spintent_spnz_read_parens_bool,
2218     read-parens .initial:n = true,
2219     read-parens .value_required:n = true,
2220     unknown .code:n =
2221     {
2222         \str_set:Nn \l__spintent_command_name_str { spnZ }
2223         \l__spintent_unknown_keys_cmd:n {#1}
2224     },
2225 }

```

(End of definition for `cifras-sep` and others.)

11.26.2 Definition of error messages

`spnz-not-integer` Raised when the $\langle argument \rangle$ contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```

2226 \msg_new:nnnn { spintent } { spnz-not-integer }
2227 { The~value~'~#1'~is~not~a~valid~integer. }
2228 {
2229     Command~\token_to_str:N \spnZ~only~accepts~integers.\\
2230 }

```

(End of definition for `spnz-not-integer`.)

11.26.3 Internal functions for \spnZ

__spintent_spnz_render_parens:

The function `\__spintent_spnz_render_parens:` first checks the state of the boolean variable `\__spintent_spnz` (controlled by the `read-parens` key). If it is `true`, it prints the number surrounded by parentheses. Otherwise, it applies the `:silent function intent` to the parentheses while printing the number.

```

2231 \cs_new_protected:Nn \__spintent_spnz_render_parens:
2232 {
2233   \bool_if:NTF \__spintent_spnz_read_parens_bool
2234   {
2235     ( \__spintent_spnz_print_num_start: )
2236   }
2237   {
2238     \MathMLintent { :silent } { ( }
2239     \__spintent_spnz_print_num_start:
2240     \MathMLintent { :silent } { ) }
2241   }
2242 }
```

(End of definition for __spintent_spnz_render_parens:.)

__spintent_spnz_process:nn

The function `\__spintent_spnz_process:nn` first processes the `<keys>` passed in ‘#1’ and then calls `\__spintent_luafun_clean_split_and_set:n` on ‘#2’. It verifies that the input is a valid integer by checking if the variable `\__spintent_luaset_has_integer_str` is `true`. If not `true`, an “error” is raised.

If it is `true`, it checks whether `\__spintent_luaset_part_int_tl` is *empty* and raises an “error” if so. Otherwise, it checks `\__spintent_spnz_wrap_parens_bool` set by `wrap-parens` and calls either the function `\__spintent_spnz_render_parens:` or `\__spintent_spnz_print_num_start:`.

```

2243 \cs_new_protected:Nn \__spintent_spnz_process:nn
2244 {
2245   \keys_set:nn { spintent / spnZ } {#1}
2246   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
2247   \str_if_eq:VnTF \__spintent_luaset_has_integer_str { true }
2248   {
2249     \tl_if_empty:NTF \__spintent_luaset_part_int_tl
2250     { \msg_error:nnn { spintent } { spnum-no-digits } {#2} }
2251     {
2252       \bool_if:NTF \__spintent_spnz_wrap_parens_bool
2253       { \__spintent_spnz_render_parens: }
2254       { \__spintent_spnz_print_num_start: }
2255     }
2256   }
2257   { \msg_error:nnn { spintent } { spnz-not-integer } {#2} }
2258 }
```

(End of definition for __spintent_spnz_process:nn.)

`\spnZ` Defines the user command `\spnZ`, which is executed within a *group* and strictly enforces *math mode*.

```

2259 \NewDocumentCommand \spnZ { O{} m }
2260 {
2261   \mode_if_math:TF
2262   {
2263     \group_begin:
2264     \__spintent_spnz_process:nn { #1 } { #2 }
2265     \group_end:
2266   }
2267   { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
2268 }
```

(End of definition for \spnZ. This function is documented on page 16.)

11.27 The command \spmQ

The command `\spmQ` typesets “mixed numbers” within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It delegates the scaling of the integer part to the Lua module `spintent.lua` via `\__spintent_luafun_spmQ_scale_int:Vn`, which measures both the integer glyph and the fraction box at the node level and computes a scaling factor to align their heights visually. Finally, it constructs the correct `MathML` intent for `MathCAT`.

11.27.1 Definition of variables for \spmQ

The variable `\l__spintrent_spmQ_join_word_str` stores the value set by the key `join-word` which will be verbalized by MathCAT as a grammatical connector between the integer and fractional parts.

The variable `\l__spintrent_spmQ_math_style_tl` stores the current mathematical style used by the function `\__spintrent_spmQ_set_wrap_parens_size:`.

The variable `\l__spintrent_spmQ_intent_read_sign_str` stores the `:intent` passed to `\MathMLintent` when the signs ‘+’ or ‘-’ are present in the integer part, used by the function `\__spintrent_spmQ_print_sacle_int:n`.

The variables `\l__spintrent_spmQ_wrap_parens_l_tl` and `\l__spintrent_spmQ_wrap_parens_r_tl` store the sized parentheses set by the function `\__spintrent_spmQ_set_wrap_parens_size:` and used by the key `wrap-parens`.

The integer variable `\l__spintrent_saved_luamml_flag_int` temporarily stores the current value of `\l__luamml_flag_int` before disabling it during the injection of the scaled integer node, and restores it afterwards. This prevents `luamml` from generating a spurious `<math>` element around the `sub_box` processed in isolation, before `\frac` has expanded.

```
2269 \str_new:N \l__spintrent_spmQ_join_word_str
2270 \tl_new:N \l__spintrent_spmQ_math_style_tl
2271 \str_new:N \l__spintrent_spmQ_intent_read_sign_str
2272 \tl_new:N \l__spintrent_spmQ_wrap_parens_l_tl
2273 \tl_new:N \l__spintrent_spmQ_wrap_parens_r_tl
2274 \int_new:N \l__spintrent_saved_luamml_flag_int
```

(End of definition for `\l__spintrent_spmQ_join_word_str` and others.)

11.27.2 Definition of error messages for \spmQ

The error message `spmQ-zero-denominator` is raised when a zero denominator is passed to `\spmQ`. The second error `spmQ-not-integer` is returned when the integer part passed in `{\langle argument \rangle}` is not actually an integer.

```
2275 \msg_new:nnnn { spintrent } { spmQ-zero-denominator }
2276 { Zero~denominator~in~\token_to_str:N \spmQ. }
2277 {
2278   The~denominator~of~a~fraction~cannot~be~zero.\l
2279 }
2280 \msg_new:nnnn { spintrent } { spmQ-not-integer }
2281 { The~value~'#1'~is~not~a~valid~integer. }
2282 {
2283   Command~\token_to_str:N \spmQ \c_space_tl only~accepts~integers.\l
2284 }
```

(End of definition for `spmQ-zero-denominator` and `spmQ-not-integer`.)

11.27.3 Definition of keys for \spmQ

Now we add the keys `join-word`, `use-spnZ`, `wrap-parens`, `read-parens` and `print-dfrac`.

```
join-word
use-spnZ
wrap-parens
read-parens
print-dfrac
unknown
2285 \keys_define:nn { spintrent / spmQ }
2286 {
2287   join-word .choices:nn =
2288     { entero , enteros , y }
2289     {
2290       \int_case:nn { \l_keys_choice_int }
2291       {
2292         {1} { \str_set:Nn \l__spintrent_spmQ_join_word_str { entero } }
2293         {2} { \str_set:Nn \l__spintrent_spmQ_join_word_str { enteros } }
2294         {3} { \str_set:Nn \l__spintrent_spmQ_join_word_str { y } }
2295       }
2296     },
2297   join-word .initial:n = y,
2298   join-word .value_required:n = true,
2299   join-word/unknown .code:n =
2300     \msg_error:nneee { spintrent } { unknown-choice }
2301     { join-word } { entero , ~enteros , ~y } { \exp_not:n {#1} },
2302   use-spnZ .bool_set:N = \l__spintrent_spmQ_use_spnZ_bool,
2303   use-spnZ .initial:n = true,
2304   use-spnZ .value_required:n = true,
2305   print-dfrac .bool_set:N = \l__spintrent_spmQ_print_dfrac_bool,
2306   print-dfrac .initial:n = false,
2307   print-dfrac .value_required:n = true,
2308   wrap-parens .bool_set:N = \l__spintrent_spmQ_wrap_parens_bool,
2309   wrap-parens .initial:n = false,
2310   wrap-parens .value_required:n = true,
```

```

2311     read-parens      .bool_set:N = \l__spintent_spmQ_read_parens_bool,
2312     read-parens      .initial:n = true,
2313     read-parens      .value_required:n = true,
2314     unknown          .code:n =
2315     {
2316         \str_set:Nn \l__spintent_command_name_str { spmQ }
2317         \__spintent_unknown_keys_cmd:n {#1}
2318     },
2319 }

```

(End of definition for join-word and others.)

11.27.4 Internal functions for \spmQ

__spintent_spmQ_set_wrap_parens_size:

The function `\__spintent_spmQ_set_wrap_parens_size:` captures the current mathematical style using the primitive `\mathstyle` from LaTeX and sets the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` with the appropriate size for the parentheses. Display styles (0 and 1) receive `\Bigl\lparen` and `\Bigr\rparen`; text styles (2 and 3) receive `\bigl\lparen` and `\bigr\rparen`; any other style falls back to `\lparen` and `\rparen`.

If the key `print-dfrac` is active, the size is always forced to `\Bigl\lparen`/`\Bigr\rparen` regardless of the current mathematical style, since `\dfrac` always produces a display-height fraction.

```

2320 \cs_new_protected:Nn \__spintent_spmQ_set_wrap_parens_size:
2321 {
2322     \int_case:nnF { \mathstyle }
2323     {
2324         { 0 }
2325         {
2326             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2327             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2328         }
2329         { 1 }
2330         {
2331             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2332             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2333         }
2334         { 2 }
2335         {
2336             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2337             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2338         }
2339         { 3 }
2340         {
2341             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2342             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2343         }
2344     }
2345     {
2346         \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \lparen }
2347         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \rparen }
2348     }

```

If the key `print-dfrac` is active, we must reset the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` to `\Bigl\lparen`/`\Bigr\rparen` unconditionally.

```

2349     \bool_if:NT \l__spintent_spmQ_print_dfrac_bool
2350     {
2351         \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2352         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2353     }
2354 }

```

(End of definition for __spintent_spmQ_set_wrap_parens_size:.)

__spintent_spmQ_start_wrap_parens:

__spintent_spmQ_stop_wrap_parens:

The functions `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:` open and close the parentheses around the “mixed number” when the key `wrap-parens` is active. They first call `\__spintent_spmQ_set_wrap_parens_size:` to determine the correct size for the current mathematical style, then print the appropriate parenthesis stored in `\l__spintent_spmQ_wrap_parens_l_tl` or `\l__spintent_spmQ_wrap_parens_r_tl`.

If the variable `\l__spintent_spmQ_read_parens_bool`, controlled by the key `read-parens`, is *false*, each parenthesis is wrapped inside `\MathMLintent{:silent}` so that NVDA/MathCAT does not verbalize them.

```

2355 \cs_new_protected:Nn \__spintent_spmQ_start_wrap_parens:
2356 {

```

```

2357 \bool_if:NT \l__spintenc_spmQ_wrap_parens_bool
2358 {
2359   \__spintenc_spmQ_set_wrap_parens_size:
2360   \bool_if:NTF \l__spintenc_spmQ_read_parens_bool
2361     { \l__spintenc_spmQ_wrap_parens_l_tl }
2362     { \MathMLintent { :silent } { \l__spintenc_spmQ_wrap_parens_l_tl } }
2363 }
2364 }
2365 \cs_new_protected:Nn \__spintenc_spmQ_stop_wrap_parens:
2366 {
2367   \bool_if:NT \l__spintenc_spmQ_wrap_parens_bool
2368   {
2369     \__spintenc_spmQ_set_wrap_parens_size:
2370     \bool_if:NTF \l__spintenc_spmQ_read_parens_bool
2371       { \l__spintenc_spmQ_wrap_parens_r_tl }
2372       { \MathMLintent { :silent } { \l__spintenc_spmQ_wrap_parens_r_tl } }
2373   }
2374 }

```

(End of definition for __spintenc_spmQ_start_wrap_parens: and __spintenc_spmQ_stop_wrap_parens:.)

__spintenc_spmQ_set_join_word:

The function `\__spintenc_spmQ_set_join_word:` injects the grammatical connector between the integer part and the fraction by dispatching on the value stored in `\l__spintenc_spmQ_join_word_str`, set by the key `join-word`. In all three cases the visual output is the Unicode invisible separator U+2063 via `\c__spintenc_invisible_sep_tl`, which produces a `<mi>` node in the MathML structure; what differs is the `:intent` text attached to it via `\MathMLintent`, which NVDA/MathCAT will verbalize as «entero», «enteros» or «y» respectively.

```

2375 \cs_new_protected:Nn \__spintenc_spmQ_set_join_word:
2376 {
2377   \str_case:Nn \l__spintenc_spmQ_join_word_str
2378   {
2379     { entero } { \MathMLintent { _entero } { \c__spintenc_invisible_sep_tl } }
2380     { enteros } { \MathMLintent { _enteros } { \c__spintenc_invisible_sep_tl } }
2381     { y } { \MathMLintent { _y- } { \c__spintenc_invisible_sep_tl } }
2382   }
2383 }

```

(End of definition for __spintenc_spmQ_set_join_word:.)

__spintenc_spmQ_print_scale_int:

The function `\__spintenc_spmQ_print_scale_int:` prints the integer part of the “mixed number” scaled to match the visual height of the accompanying fraction. Before calling the Lua function `\__spintenc_luafun_spmQ_scale_int` it temporarily disables `luamml` by saving the current value of `\l__luamml_flag_int` into `\l__spintenc_saved_luamml_flag_int` and setting it to `0` (skip completely). This prevents `luamml` from generating a spurious `<math>` wrapper around the `sub_box` node that holds the scaled integer, which is processed in isolation before `\frac` has expanded. Once the Lua function returns, the original flag value is restored via `\l__spintenc_saved_luamml_flag_int`.

The Lua function receives the integer digits from `\l__spintenc_luaset_part_int_tl` and the fraction command name (`\frac` or `\dfrac`, determined by the key `print-dfrac`) as its two arguments, and inserts the appropriately scaled `sub_box` node directly into the current horizontal list.

```

2384 \cs_new_protected:Nn \__spintenc_spmQ_print_scale_int:
2385 {
2386   \int_set_eq:Nc \l__spintenc_saved_luamml_flag_int { \l__luamml_flag_int }
2387   \int_set:Nn \l__luamml_flag_int { 0 }
2388   \bool_if:NTF \l__spintenc_spmQ_print_dfrac_bool
2389     { \__spintenc_luafun_spmQ_scale_int:Nn \l__spintenc_luaset_part_int_tl { dfraction } }
2390     { \__spintenc_luafun_spmQ_scale_int:Nn \l__spintenc_luaset_part_int_tl { fraction } }
2391   \int_set_eq:cn { \l__luamml_flag_int } \l__spintenc_saved_luamml_flag_int
2392 }

```

(End of definition for __spintenc_spmQ_print_scale_int:.)

__spintenc_spmQ_print_sacle_int:n

The function `\__spintenc_spmQ_print_sacle_int:n` takes the integer part of the “mixed number” passed in `#1`. It first calls `\__spintenc_luafun_clean_split_and_set:n` on `#1` to parse and populate the shared Lua variables, then checks the status of `\l__spintenc_luaset_has_integer_str`.

If the status is `true`, it inspects `\l__spintenc_luaset_sign_tl`. When the sign is *empty*, it delegates directly to `\__spintenc_spmQ_print_scale_int:` to render the scaled integer. When a sign is present, it sets `\l__spintenc_spmQ_intent_read_sign_str` to either `_más:prefix($e)` or `_menos:prefix($e)` and wraps the sign glyph together with a MathML argument (`$e`) containing the scaled integer inside a `\MathMLintent`, so that NVDA/MathCAT verbalizes the sign as a prefix before the integer value. An

`\__spintent_invisible_sep:` call between the sign and the `MathMLarg` forces the opening of a new `<mrow>` in the MathML structure, which is required for the intent tree to be well-formed.

If the status is `false`, the `spmQ-not-integer` “error” is raised immediately.

```

2393 \cs_new_protected:Nn \__spintent_spmQ_print_sacle_int:n
2394 {
2395   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2396   \str_if_eq:VnTF \__spintent_luaset_has_integer_str { true }
2397   {
2398     \tl_if_empty:NTF \__spintent_luaset_sign_tl
2399     {
2400       \__spintent_spmQ_print_scale_int:
2401     }
2402     {
2403       \str_if_eq:VnTF \__spintent_luaset_sign_tl { + }
2404       {
2405         \str_set:Nn
2406           \__spintent_spmQ_intent_read_sign_str { _más:prefix($e) }
2407       }
2408       {
2409         \str_set:Nn
2410           \__spintent_spmQ_intent_read_sign_str { _menos:prefix($e) }
2411       }
2412       \MathMLintent { \__spintent_spmQ_intent_read_sign_str }
2413       {
2414         \__spintent_luaset_sign_tl \__spintent_invisible_sep:
2415         \MathMLarg { e } { \__spintent_spmQ_print_scale_int: }
2416       }
2417     }
2418   }
2419   { \msg_error:nn { spintent } { spmQ-not-integer } }
2420 }

```

(End of definition for `\__spintent_spmQ_print_sacle_int:n`.)

`\__spintent_spmQ_print_spnZ_dfrac:nn`

The function `\__spintent_spmQ_print_spnZ_dfrac:nn` typesets the fractional part of the “mixed number”. It takes the numerator in ‘#1’ and the denominator in ‘#2’. It first calls `\__spintent_luafun_clean_split_and_set:` on ‘#1’ to validate that the numerator is an integer; if `\__spintent_luaset_has_integer_str` is `false`, the `spmQ-not-integer` “error” is raised immediately.

If the validation passes, the function dispatches on the two boolean variables `\__spintent_spmQ_use_spnZ_bool` and `\__spintent_spmQ_print_dfrac_bool`, controlled respectively by the keys `use-spnZ` and `print-dfrac`. When `use-spnZ` is `true`, both the numerator and denominator are passed through `\spnZ` to gain MathML integer semantics; otherwise, the raw token lists ‘#1’ and ‘#2’ are used directly. In both branches, the fraction command is selected dynamically via `\use:c`: if `print-dfrac` is `true`, `dfrac` is assembled; otherwise, `frac` is used.

```

2421 \cs_new_protected:Nn \__spintent_spmQ_print_spnZ_dfrac:nn
2422 {
2423   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2424   \str_if_eq:VnTF \__spintent_luaset_has_integer_str { false }
2425   { \msg_error:nn { spintent } { spmQ-not-integer } }
2426   \bool_if:NTF \__spintent_spmQ_use_spnZ_bool
2427   {
2428     \use:c { \bool_if:NTF \__spintent_spmQ_print_dfrac_bool { d } frac }
2429     { \spnZ { #1 } } { \spnZ { #2 } }
2430   }
2431   {
2432     \use:c { \bool_if:NTF \__spintent_spmQ_print_dfrac_bool { d } frac }
2433     { #1 } { #2 }
2434   }
2435 }

```

(End of definition for `\__spintent_spmQ_print_spnZ_dfrac:nn`.)

`\__spintent_spmQ_print:nnn`

The function `\__spintent_spmQ_print:nnn` is the main rendering orchestrator for `\spmQ`. It takes three arguments: the integer part ‘#1’, the numerator ‘#2’, and the denominator ‘#3’. It opens the optional parentheses via `\__spintent_spmQ_start_wrap_parens:`, then calls `\__spintent_spmQ_print_sacle_int:n` to scale and render the integer part. Next, it injects the grammatical connector between the integer and the fraction via `\__spintent_spmQ_set_join_word:`. Then it calls `\__spintent_spmQ_print_spnZ_dfrac:nn` to typeset the fraction using the keys `print-dfrac` and `use-spnZ`. Finally, it closes the optional parentheses via `\__spintent_spmQ_stop_wrap_parens:`.

```

2436 \cs_new_protected:Nn \__spintent_spmQ_print:nnn

```

```

2437 {
2438   \__spintent_spmQ_start_wrap_parens:
2439   \__spintent_spmQ_print_sacle_int:n { #1 }
2440   \__spintent_spmQ_set_join_word:
2441   \__spintent_spmQ_print_spnZ_dfrac:nn { #2 } { #3 }
2442   \__spintent_spmQ_stop_wrap_parens:
2443 }

```

(End of definition for __spintent_spmQ_print:nnn.)

__spintent_spmQ_exec:nnnn

The internal function __spintent_spmQ_exec:nnnn takes four arguments: the optional *(keys)* ‘#1’, the integer ‘#2’, the numerator ‘#3’, and the denominator ‘#4’. First sets the optional *(keys)* from ‘#1’, then processes the denominator input in ‘#4’ using __spintent_luafun_clean_split_and_set:n to validate it is a non-zero integer.

It checks the status of \l__spintent_luaaset_has_integer_str. If **true** and the integer part is exactly zero, it issues the spmQ-zero-denominator “error”. Otherwise, whether the status is **false** or the integer part is *non-zero*, it calls __spintent_spmQ_print:nnn to format the “mixed number”.

```

2444 \cs_new_protected:Nn \__spintent_spmQ_exec:nnnn
2445 {
2446   \keys_set:nn { spintent / spmQ } { #1 }
2447   \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2448   \str_if_eq:VnTF \l__spintent_luaaset_has_integer_str { true }
2449   {
2450     \int_compare:nNnTF { \l__spintent_luaaset_part_int_tl } = { 0 }
2451     { \msg_error:nn { spintent } { spmQ-zero-denominator } }
2452     { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2453   }
2454   { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2455 }

```

(End of definition for __spintent_spmQ_exec:nnnn.)

\spmQ

The command \spmQ is the public interface for formatting “mixed numbers”. It first checks if it is being used in *math mode*, issuing an “error” if not. Inside a local group, it calls __spintent_spmQ_exec:nnnn to process the optional *(keys)* and the *{(arguments)}*.

```

2456 \NewDocumentCommand \spmQ { O{ } m m m }
2457 {
2458   \mode_if_math:TF
2459   {
2460     \group_begin:
2461     \__spintent_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
2462     \group_end:
2463   }
2464   { \msg_error:nnn { spintent } { need-math-mode } { \spmQ } }
2465 }

```

(End of definition for \spmQ. This function is documented on page 16.)

11.28 Finish package

Finish package implementation.

```

2466 \file_input_stop:
2467 \</package>

```

12 Índice de la Implementación

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

C	
cifras-sep	<u>220</u> , <u>505</u> , <u>778</u> , <u>1727</u> , <u>1798</u> , <u>2210</u>
cmd-key-unknown	<u>140</u>
cmd-key-value-unknown	<u>140</u>
Commands provide by <i>spintent</i> :	
\setspintent	24
\spmoney	38
\spnum	27
\spsiglo	47, 49
\spunit	32
currency	<u>778</u>
currency-no-subunits	<u>747</u>
D	
dolar-math	<u>778</u>
I	
inline-numeric-denominator	<u>483</u>
invalid-angle-format	<u>2160</u>
invalid-century-format	<u>1174</u>
invalid-currency	<u>747</u>
invalid-date-format	<u>1404</u>
invalid-time-format	<u>1325</u>
J	
join-word	<u>2285</u>
K	
Keys for \spdiv provide by <i>spintent</i> :	
cifras-sep	58
notation-sym	58
read-parens	58
read-period-sep	58
read-sign	58
read-sym	58
set-sym	58
wrap-parens	58
Keys for \spdsym provide by <i>spintent</i> :	
prefix	56
read-sym	56
set-sym	56
suffix	56
Keys for \spmcd provide by <i>spintent</i> :	
prefix	62, 64
read-cmd	62
result-num	64
result	62
sample-arg	62
silent-cmd	62
upper-case	62
Keys for \spmcm provide by <i>spintent</i> :	
prefix	62, 64
read-cmd	62
result-num	64
result	62
sample-arg	62
silent-cmd	62
upper-case	62
Keys for \spmoney provide by <i>spintent</i> :	
cifras-sep	39
currency	39
dolar-math	39
sub-units	39
Keys for \spmQ provide by <i>spintent</i> :	
join-word	70, 72
print-dfrac	70-73
read-parens	70, 71
use-spnZ	70, 73
wrap-parens	70, 71
Keys for \spmsym provide by <i>spintent</i> :	
not-print	57
prefix	57
read-sym	57
set-sym	57
suffix	57
Keys for \spmul provide by <i>spintent</i> :	
cifras-sep	60
notation-sym	60
read-parens	60
read-period-sep	60
read-sign	60
read-sym	60
set-sym	60
wrap-parens	60
Keys for \spnD provide by <i>spintent</i> :	
result	64
silent-cmd	64
Keys for \spnM provide by <i>spintent</i> :	
result	64
silent-cmd	64
Keys for \spnum provide by <i>spintent</i> :	
cifras-sep	26, 28, 29
notation-sym	27, 28
read-period-sep	27-29
read-sign	27, 28
Keys for \spnZ provide by <i>spintent</i> :	
cifras-sep	68
read-parens	68, 69
read-sign	68
wrap-parens	68, 69
Keys for \spshort provide by <i>spintent</i> :	
read-arg	44
small-caps	44
Keys for \spsiglo provide by <i>spintent</i> :	
print-era	47, 49
Keys for \spunit provide by <i>spintent</i> :	
cifras-sep	33
notation-sym	33
print-cdot	33
read-cdot	33
read-period-sep	33
read-slash	33
Keys for \spusep provide by <i>spintent</i> :	
silent	53
size	53
M	
mc-invalid-arguments	<u>1880</u>
md-invalid-argument	<u>2052</u>

Lua module provide by [spintent](#):

`spintent.lua` . [22](#), [23](#), [26](#), [27](#), [32](#), [33](#), [36](#), [38](#), [39](#), [44](#), [45](#),
[47-51](#), [61](#), [64](#), [67](#), [69](#)

N

`not-print` [1653](#)
`notation-sym` [220](#), [505](#), [1727](#), [1798](#)

P

Packages:

`expl3` [22](#), [26](#), [27](#)
`l3doc` [22](#)
`lua-unicode-math` [22](#)
`luamml` [23](#), [70](#), [72](#)
`tagpdf` [44](#), [48](#)
`unicode-math` [22](#), [54](#)
`prefix` [1583](#), [1653](#), [1887](#), [2026](#)
`print-dfrac` [2285](#)
`print-era` [1182](#)

R

`read-arg` [1052](#)
`read-cdot` [505](#)
`read-cmd` [1887](#)
`read-parens` [1727](#), [1798](#), [2210](#), [2285](#)
`read-period-sep` [220](#), [505](#), [1727](#), [1798](#)
`read-sign` [220](#), [1727](#), [1798](#), [2210](#)
`read-slash` [505](#)
`read-sym` [1583](#), [1653](#), [1727](#), [1798](#)
`result` [1887](#), [2026](#)
`result-num` [2026](#)

S

`sample-arg` [1887](#), [2026](#)
seq commands:
`\seq_set_from_clist:Nn` [41](#)
`set-sym` [1583](#), [1653](#), [1727](#), [1798](#)
`\setspintent` [3](#), [24](#), [128](#)
`silent` [1448](#)
`silent-cmd` [1887](#), [2026](#)
`size` [1448](#)
`small-caps` [1052](#)
`\spang` [17](#), [67](#), [2200](#)
`\spdate` [11](#), [51](#), [1428](#)
`\spdiv` [13](#), [58](#), [1788](#)
`spdiv-empty-argument` [1747](#)
`\spdsym` [12](#), [55](#), [1630](#)
spintent internal commands:
`\__spintent_collapse_hyphens:N` [65](#)
`\l__spintent_command_name_str` [140](#)
`\__spintent_invisible_sep:` [51](#)
`\c__spintent_invisible_sep_tl` [72](#), [51](#)
`\__spintent_luafun_clean_split_and_set:n` [46](#),
[177](#)
`\__spintent_luafun_spang_parse:n` [67](#)
`\__spintent_luafun_spdate_parse:n` [52](#)
`\__spintent_luafun_spmoney_lookup_data:n` [46](#)
`\__spintent_luafun_spmQ_scale_int:nn` [46](#)
`\__spintent_luafun_sptime_parse:n` [51](#)
`\__spintent_luafun_spunit_lookup_alias:n` [46](#)
`\l__spintent_luaset_arg_above_tl` [196](#)
`\l__spintent_luaset_arg_below_tl` [196](#)

`\l__spintent_luaset_dec_is_all_zeros_-str` [189](#)
`\l__spintent_luaset_dec_sep_tl` [177](#)
`\l__spintent_luaset_decimal_period_str` [189](#)
`\l__spintent_luaset_denom_has_number_-str` [196](#)
`\l__spintent_luaset_exponent_tl` [189](#)
`\l__spintent_luaset_format_part_dec_str` [184](#)
`\l__spintent_luaset_format_part_int_str` [184](#)
`\l__spintent_luaset_has_exponent_str` . . [189](#)
`\l__spintent_luaset_has_integer_str` . . [72-74](#),
[177](#)
`\l__spintent_luaset_has_natural_str` . . . [177](#)
`\l__spintent_luaset_has_units_str` [196](#)
`\l__spintent_luaset_millions_str` [189](#)
`\l__spintent_luaset_not_decimal_not_-period_str` [189](#)
`\l__spintent_luaset_not_decimal_period_-str` [189](#)
`\l__spintent_luaset_only_part_dec_str` . . [184](#)
`\l__spintent_luaset_only_part_int_str` . . [184](#)
`\l__spintent_luaset_only_part_period_-str` [184](#)
`\l__spintent_luaset_part_dec_tl` [177](#)
`\l__spintent_luaset_part_int_tl` [72](#), [177](#)
`\l__spintent_luaset_part_period_tl` [177](#)
`\l__spintent_luaset_sign_tl` [72](#), [177](#)
`\l__spintent_luaset_status_str` [196](#)
`\l__spintent_mc_args_seq` [1869](#)
`\__spintent_mc_process_args:nn` [1928](#)
`\l__spintent_md_out_seq` [2016](#)
`\__spintent_print_spunit_hierarchy:` . . . [606](#)
`\__spintent_sanitize_affix:Nn` [73](#)
`\l__spintent_saved_luamml_flag_int` [72](#), [2269](#)
`\l__spintent_setspintent_input_arg_cmd_-tl` [91](#)
`\__spintent_setspintent_parse_args:nn` . . [111](#)
`\__spintent_setspintent_set_keys:nn` . . . [100](#)
`\l__spintent_spang_luaset_error_str` . . . [2156](#)
`\l__spintent_spang_luaset_grado_str` . . . [2156](#)
`\l__spintent_spang_luaset_minuto_str` . . [2156](#)
`\l__spintent_spang_luaset_segundo_str` . . [2156](#)
`\__spintent_spang_process:n` [2165](#)
`\l__spintent_spdate_luaset_error_str` . . [1402](#)
`\l__spintent_spdate_luaset_output_str` . . [1402](#)
`\__spintent_spdate_process:n` [52](#), [1413](#)
`\__spintent_spdiv_process:nnn` [1773](#)
`\__spintent_spdiv_render_op:nn` [1752](#)
`\__spintent_spdiv_render_parens:nn` . . . [1761](#)
`\l__spintent_spdsym_prefix_tl` [1579](#)
`\l__spintent_spdsym_read_sym_tl` [1579](#)
`\l__spintent_spdsym_suffix_tl` [1579](#)
`\l__spintent_spdsym_symbol_tl` [1579](#)
`\__spintent_spmc_process:nnnn` [1950](#)
`\l__spintent_spmcd_prefix_tl` [1869](#)
`\l__spintent_spmcd_print_result_bool` . . [1869](#)
`\l__spintent_spmcd_print_upper_bool` . . [1869](#)
`\l__spintent_spmcd_read_terse_bool` . . . [1869](#)
`\l__spintent_spmcd_suffix_tl` [1869](#)

__spintent_spmcm_prefix_tl	1869	__spintent_spmQ_stop_wrap_parens: ...	2355
\l__spintent_spmcm_print_result_bool ..	1869	\l__spintent_spmQ_use_spnZ_bool	73
\l__spintent_spmcm_print_upper_bool ...	1869	\l__spintent_spmQ_prefix_tl	1649
\l__spintent_spmcm_read_terse_bool ...	1869	\l__spintent_spmQ_read_sym_tl	1649
\l__spintent_spmcm_spmcd_luaset_error_-str	1869	\l__spintent_spmQ_suffix_tl	1649
\l__spintent_spmcm_suffix_tl	1869	\l__spintent_spmQ_symbol_tl	1649
__spintent_spmoney_build_tree:	957	__spintent_spmul_process:nnn	1844
__spintent_spmoney_check_sub_units: ..	910	__spintent_spmul_render_op:nn	1823
\l__spintent_spmoney_extracted_curr_str	739	\l__spintent_spmul_render_parens:nn ...	1832
\l__spintent_spmoney_extracted_num_tl .	739	\l__spintent_spnD_luaset_is_truncated_-str	2016
__spintent_spmoney_intent_int:	846	\l__spintent_spnD_luaset_result_tl ...	2016
\l__spintent_spmoney_intent_money_str .	739	\l__spintent_spnD_prefix_tl	2016
\l__spintent_spmoney_intent_sign_str ..	739	\l__spintent_spnD_print_result_bool ...	2016
\l__spintent_spmoney_intent_sub_unit_-str	739	\l__spintent_spnD_print_result_num_tl .	2016
\l__spintent_spmoney_luaset_conde_str .	729	\l__spintent_spnD_silent_cmd_bool ...	2016
\l__spintent_spmoney_luaset_currency_arg_-str	729	\l__spintent_spnM_luaset_is_truncated_-str	2016
\l__spintent_spmoney_luaset_plur_str ..	729	\l__spintent_spnM_luaset_result_tl ...	2016
\l__spintent_spmoney_luaset_position_-str	729	\l__spintent_spnM_prefix_tl	2016
\l__spintent_spmoney_luaset_print_iso_-str	729	\l__spintent_spnM_print_result_bool ...	2016
\l__spintent_spmoney_luaset_sing_str ..	729	\l__spintent_spnM_print_result_num_tl .	2016
\l__spintent_spmoney_luaset_status_str	729	\l__spintent_spnM_silent_cmd_bool ...	2016
\l__spintent_spmoney_luaset_subplur_str	729	__spintent_spnM_spnD_exec:nnnn	2097
\l__spintent_spmoney_luaset_subsing_str	729	__spintent_spnM_spnD_intent:nn	2062
\l__spintent_spmoney_luaset_symbol_str	729	__spintent_spnM_spnD_process:nnnnn ...	2057
__spintent_spmoney_normalize_key:n ...	764	__spintent_spnM_spnD_result:n	2078
__spintent_spmoney_parse_and_route:n .	1005	__spintent_spnM_spnD_silent:nn	2070
__spintent_spmoney_print_dec_sub_unit:	957	\l__spintent_spnQ_intent_read_sign_str	2269
__spintent_spmoney_print_int:	846	\l__spintent_spnQ_read_parens_bool ...	71
__spintent_spmoney_print_integer_-continue:	957	\l__spintent_spnQ_wrap_parens_l_tl	71, 2269
__spintent_spmoney_print_sep_con: ...	957	\l__spintent_spnQ_wrap_parens_r_tl	71, 2269
__spintent_spmoney_print_symbol: ...	812	\l__spintent_spnnum_notation_sym_tl ...	27
__spintent_spmoney_render_layout: ...	825	\l__spintent_spnnum_intent_read_dec_sep_-str	28, 30, 215
__spintent_spmoney_render_number_node:	837	\l__spintent_spnnum_intent_read_sign_str	28, 215
__spintent_spmoney_render_only_integer_-node:	837	\l__spintent_spnnum_notation_sym_tl ...	215
\l__spintent_spmoney_res_main_voice_str	739	__spintent_spnnum_print_integer:	382
__spintent_spmoney_set_and_print_sub_units:	910	__spintent_spnnum_print_num_start: ...	382
__spintent_spmoney_set_currency:n ...	764	__spintent_spnnum_print_number_core: ..	382
\l__spintent_spmoney_sub_units_bool ...	739	__spintent_spnnum_print_part_dec: ...	361
\l__spintent_spmoney_subnum_int	739	__spintent_spnnum_print_part_period: ..	361
__spintent_spmoney_validate_money: ...	796	__spintent_spnnum_print_period_bar: ...	290
__spintent_spmQ_exec:nnnn	2444	\l__spintent_spnnum_read_period_sep_str .	27, 215
\l__spintent_spmQ_join_word_str ...	72, 2269	\l__spintent_spnnum_read_terse_bool .	27, 215
\l__spintent_spmQ_math_style_tl	2269	__spintent_spnnum_render_dec:	273
__spintent_spmQ_print:nnn	2436	__spintent_spnnum_render_decimal_sep: .	327
\l__spintent_spmQ_print_dfrac_bool ...	73	__spintent_spnnum_render_int:	273
__spintent_spmQ_print_sacle_int:n ...	2393	__spintent_spnnum_render_only_period: .	290
__spintent_spmQ_print_scale_int: ...	2384	__spintent_spnnum_render_part:n	273
__spintent_spmQ_print_spnZ_dfrac:nn ..	2421	__spintent_spnnum_render_period_sep: ..	290
__spintent_spmQ_set_join_word:	2375	__spintent_spnz_process:nn	2243
__spintent_spmQ_set_wrap_parens_size:	2320	__spintent_spnz_render_parens:	2231
__spintent_spmQ_start_wrap_parens: ...	2355	__spintent_spread_exec:nn	1560
		\l__spintent_spread_intent_tl	1559
		__spintent_spshort_execute_layout: ...	1113
		\l__spintent_spshort_luaset_base_str ..	1030

\l__spintent_spshort_luaset_expanded_-str	1030	__spintent_spusep_convert:n	1507
\l__spintent_spshort_luaset_layout_str	1030	\l__spintent_spusep_left_tl	53, 54, 1445
\l__spintent_spshort_luaset_output_str	1030	\l__spintent_spusep_right_tl	53, 54, 1445
\l__spintent_spshort_luaset_status_str	1030	__spintent_spusep_set_parens:nn	1496
\l__spintent_spshort_luaset_suffix_str	1030	\l__spintent_spusep_silent_bool	54
__spintent_spshort_print:nn	1066	__spintent_unknown_keys:n	167
__spintent_spshort_process_and_-render:n	1078	__spintent_unknown_keys:nn	167
__spintent_spshort_render_bypass:n	1089	\spmcD	15, 61, 1996
__spintent_spshort_render_error:n	1089	\spmcM	15, 61, 1996
__spintent_spshort_render_fallback:	1089	\spmoney	6, 38, 1019
__spintent_spshort_render_success:	1089	\spmQ	16, 69, 2456
__spintent_spshort_tag_span:nn	1039	spmQ-not-integer	2275
\l__spintent_spsiglo_luaset_arabic_str	1170	spmQ-zero-denominator	2275
\l__spintent_spsiglo_luaset_error_str	1170	\spmsym	12, 57, 1702
\l__spintent_spsiglo_luaset_roman_min_-str	1170	\spmul	13, 59, 1859
\l__spintent_spsiglo_print_era_str	1170	spmul-empty-argument	1818
__spintent_spsiglo_print_math:n	1254	\spnD	14, 64, 2136
__spintent_spsiglo_print_text:n	1254	\spnewunit	6, 36, 697
__spintent_spsiglo_render_simple:nn	1205	spnewunit-duplicate	676
__spintent_spsiglo_render_with_-era:nnn	1205	\spnM	14, 64, 2136
__spintent_spsiglo_tag_span:nn	1241	\spnum	3, 27, 29, 451
__spintent_sptime_build_seconds:nn	1334	spnum-has-units	437
__spintent_sptime_build_tree:	51, 1352	spnum-no-digits	437
\l__spintent_sptime_luaset_base_str	1320	\spnZ	16, 68, 2259
\l__spintent_sptime_luaset_error_str	1320	spnz-not-integer	2226
\l__spintent_sptime_luaset_has_seconds_-str	1320	\spread	11, 55, 1569
\l__spintent_sptime_luaset_is_fraction_-str	1320	spread-empty-args	1556
\l__spintent_sptime_luaset_seconds_str	1320	\spshort	7, 44, 1157
\l__spintent_spunit_exp_seq	476	\spsiglo	11, 47, 1307
\l__spintent_spunit_exponent_tl	476	\sptime	11, 50, 1384
__spintent_spunit_format_canonical:	539	\spunit	4, 32, 33, 36, 664
\l__spintent_spunit_is_mult_bool	476	spunit-invalid-unit	483
\l__spintent_spunit_luaset_canonical_-str	469	spunit-no-units	483
\l__spintent_spunit_luaset_compact_exp_-str	469	\spunitalias	6, 36, 711
\l__spintent_spunit_luaset_is_compact_-str	469	spunitalias-duplicate	676
\l__spintent_spunit_luaset_is_-sexagesimal_str	469	spunitalias-invalid-expr	676
\l__spintent_spunit_luaset_lookup_status_-str	469	\spusep	12, 53, 1545
\l__spintent_spunit_luaset_read_str	469	spusep-empty-arg	1438
\l__spintent_spunit_luaset_status_str	469	spusep-invalid-arg	1438
\l__spintent_spunit_mult_seq	476	str commands:	
__spintent_spunit_process_base_exp:n	539	\str_if_eq:nnTF	41
__spintent_spunit_process_mult:n	599	\str_lowercase:n	41
\l__spintent_spunit_read_cdot_bool	476	\str_uppercase:n	41
\l__spintent_spunit_read_slash_str	476	sub-units	778
__spintent_spunit_safe_exec:n	618	suffix	1583, 1653
\l__spintent_spunit_unit_name_tl	476		
\l__spintent_spusep_arg_tl	53, 54, 1445		
		U	
		unknown	1448, 1583, 1653, 2285
		unknown-abbreviation	1036
		unknown-choice	140
		unknown-command-setup	91
		upper-case	1887
		use-spnZ	2285
		W	
		wrap-parens	1727, 1798, 2210, 2285